

José Augusto N. G. Manzano

Linguagem HOPE

Programação funcional

2ª Edição

Revisão 0

São Paulo

2019 – Propes Vivens

Copyright © 2018 de José Augusto N. G. Manzano.

Todos os direitos reservados. Proibida a reprodução total ou parcial, por qualquer meio ou processo, especialmente por sistemas gráficos, microfilmicos, fotográficos, reprográficos, fonográficos, videográficos, internet, e-books. Vedada a memorização e/ou recuperação total ou parcial em qualquer sistema de processamento de dados e a inclusão de qualquer parte da obra em qualquer programa juscibernético. Essas proibições aplicam-se também às características gráficas da obra e à sua editoração. A violação dos direitos autorais é punível como crime (art. 184 e parágrafos, do Código Penal, conforme Lei nº 10.695, de 07.01.2003) com pena de reclusão, de dois a quatro anos, e multa, conjuntamente com busca e apreensão e indenizações diversas (artigos 102, 103 parágrafo único, 104, 105, 106 e 107 itens 1, 2 e 3 da Lei nº 9.610, de 19.06.1998, Lei dos Direitos Autorais).

O Autor acredita que todas as informações aqui apresentadas estão corretas e podem ser utilizadas para qualquer fim legal. Entretanto, não existe qualquer garantia, explícita ou implícita, de que o uso de tais informações conduzirá sempre ao resultado desejado. Os nomes de sites e empresas, porventura mencionados, foram utilizados apenas para ilustrar os exemplos, não tendo vínculo nenhum com o livro, não garantindo a sua existência nem divulgação.

Conteúdo adaptado ao Novo Acordo Ortográfico da Língua Portuguesa, em execução desde 1º de janeiro de 2009.

Dados Internacionais de Catalogação na Publicação (CIP)
(Ficha catalográfica confeccionada pelo autor)

M296l Manzano, José Augusto Navarro Garcia
Linguagem HOPE: programação funcional / José Augusto N. G. Manzano --
2. ed. -- São Paulo : Propes Vivens, 2019.
170 p.

Bibliografia.

ISBN: 978-85-923720-5-7

1. HOPE (Linguagem de programação para computadores)
I. Título.

13-08576

CDD-005.133

Índices para catálogo sistemático:

1. HOPE : Linguagem de programação : Computadores : Processamento de dados 005.133

Produção e Editoração:
Cover image:

José Augusto Navarro Garcia Manzano
canvas.com

FABRICANTE

Linguagem: **HOPE**
Autores: **David B. Macqueen**
Donald T. Sannella
Rod M. Burstall

Interpretador: **Hope Interpreter (RP-HOME)**
Fabricante: **Ross Paterson**
Sítio: **<http://www.soi.city.ac.uk/~ross/Hope/> (descontinuado)**
Sistema: **FreeBSD**
Distribuidor: **Daniil Baturin**
Sítio: **<https://github.com/dmbaturin/hope>**

Interpretador: **Hope for Windows (MA-HOPE)**
Fabricante: **Marco Alfaro**
Sítio: **<http://hopelang.blogspot.com/> (descontinuado)**
Sistema: **Windows**
Distribuidor: **Augusto Manzano**
Sítio: **<http://hope.manzano.pro.br/>**

AGRADECIMENTOS

À minha esposa Sandra e à minha filha Audrey, motivos de inspiração ao meu trabalho.

Aos meus alunos por me incentivarem continuamente quando me questionam sobre temas que ainda desconheço e me levam a pesquisar muito mais.

Vida longa e próspera!

Posso deitar-me, dormir e despertar,
pois é o Senhor quem me ampara.

Sl 3, 6

SUMÁRIO

Capítulo 1 - Introdução Funcional

1.1 Fundamentação	13
1.2 Paradigma funcional	15
1.3 Linguagens funcionais	17
1.4 Linguagem funcional: HOPE	18
1.5 Obtenção e instalação	21

Capítulo 2 - Linguagem HOPE

2.1 Interpretadores HOPE	23
2.2 Tipos de dados primitivos	27
2.3 Palavras chave e funcionalidades	29
2.4 Operadores funcionais	32
2.5 Estrutura sintática básica	34

Capítulo 3 - Programação funcional básica

3.1 Definição de constante implícita	41
3.2 Operações aritméticas	46
3.3 Operações lógicas	48
3.4 Tomada de decisão	51
3.5 Recursão como laços	54
3.6 Funções internas	60
3.7 Funções definidas pelo programador	66
3.8 Tipos de dados declarados	71
3.9 Função polimórfica	75

Capítulo 4 - Programação funcional avançada

4.1 Funções pré-fixada e pós-fixada	79
4.2 Operações básicas com estruturas	81
4.3 Funções de ordem superior	88
4.4 Funções anônimas	90
4.5 Simplificações de expressões	91
4.6 Definição de módulos	95

Capítulo 5 - Programação com listas

5.1 Listas como estruturas de dados	103
5.2 Operações básicas com conjuntos	120
5.3 Processamento de conjuntos	124
5.4 Relações de inclusão	133
5.5 Módulo mylist.hop	135

Capítulo 6 - Programação complementar

6.1 Currificação	143
6.2 Funções complementares	146
6.3 Manipulação de caracteres	148
6.4 Manipulação de cadeias	153
6.5 Criptografia Caesar	159

Apêndice

Tabela ASCII	163
--------------------	-----

Bibliografia	167
---------------------------	-----

PREFÁCIO

Descobri HOPE durante estudo realizado a respeito do paradigma funcional e das linguagens puras que suportam este modelo de programação. O que chamou a atenção nesta linguagem foi sua simplicidade e o fato de não ter nenhuma funcionalidade de gerenciamento de dados predefinida, tornando-a uma excelente ferramenta para o ensino do paradigma funcional para novatos neste contexto.

HOPE não é usada comercialmente ou industrialmente, ficou restrita ao universo acadêmico quando da apresentação como trabalho de conclusão de curso em 1978 na Universidade de Endiburgo. Apesar de aparentemente “esquecida” a linguagem motivou o desenvolvimento de alguns interpretadores, tais como: IC-HOPE (Imperial College) em 1984 para PC-DOS, HOPE Interpreter (Ross Paterson) em 1998 para UNIX (FreeBSD), Hopeless (Alexander Sharbarshin) para MacOS e Hope for Windows (Marco Alfaro) em 2013 para Windows.

Dentro deste cenário o que motivou a escrita deste trabalho foi o fato de existir pouca documentação disponível para a linguagem e de não ter sido escrito até este lançamento um livro sobre a linguagem tanto no Brasil como no restante do mundo.

São apresentados nesta obra 100% dos comandos disponíveis na linguagem. Nos capítulos apresentados são operacionalizados temas como: tipos de dados; definição de constantes e tipos de dados; operações aritméticas e lógicas; tomada de decisões, recursão simples e de cauda; funções internas; funções definidas pelo programador; tipos de dados declarados; criação e uso de módulos; operações com listas de dados; manipulação de caracteres e cadeias, além de uma introdução simplificada de ações criptográficas.

Desejo um grande abraço e um excelente começo na aprendizagem do tema lógica de programação funcional. Espero que o investimento ora realizado na aquisição legal deste livro lhe seja útil e valha cada centavo investido.

Augusto Manzano

SOBRE O AUTOR

José Augusto Navarro Garcia Manzano é brasileiro, nascido no estado de São Paulo, capital, em 26 de abril de 1965; professor e mestre com formação acadêmica em Análise de Sistemas, Ciências Econômicas e licenciatura em Matemática. Atua na área de Tecnologia da Informação (desenvolvimento de software, ensino e treinamento) desde 1986. Participou do desenvolvimento de aplicações computacionais para áreas de telecomunicações e comércio. Na carreira docente, iniciou suas atividades em cursos livres, trabalhando, posteriormente, em empresas de treinamento e nos ensinos técnico e superior. Trabalhou em empresas da área, como Abak, Servimec, Montreal Informática, CompuCenter, Cebel, SPCI, BEPE, Origin, OpenClass, entre outras.

Atualmente, é professor com dedicação exclusiva ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), antiga Escola Técnica Federal. Em sua carreira docente, possui condições de ministrar componentes curriculares de Lógica de Programação (Algoritmos), Estrutura de Dados, Microinformática, Informática, Linguagens de Programação Estruturada, Linguagens de Programação Orientada a Objetos, Engenharia de Software, Tópicos Avançados em Processamento de Dados, Sistemas de Informação, Engenharia da Informação, Arquitetura de Computadores e Tecnologias Web. Tem conhecimento das linguagens de programação Classic BASIC, COMAL, Logo, PASCAL, Assembly, FORTRAN, C, C++, D, Java, Modula-2, Structured BASIC, C#, Lua, HTML, XHTML, VBA, Ada, Rust, Hope, Python, Haskell, OCaml, Groovy, Julia e JavaScript/JScript. Possui mais de uma centena de obras publicadas, além de artigos no Brasil e no exterior.

1

Introdução Funcional

Este capítulo apresenta alguns conceitos iniciais importantes. Destacam-se nesta exposição alguns detalhes que fundamentam o paradigma declarativo usado por linguagens funcionais. São fornecidos detalhes históricos que influenciaram o desenvolvimento do paradigma funcional e apontamentos sobre o surgimento da linguagem de programação HOPE, sua instalação, chamada para operação e encerramento. São detalhados instruções de obtenção e instalação da linguagem nos sistemas operacionais FreeBSD e Windows.

1.1 FUNDAMENTAÇÃO

Os fundamentos que norteiam a programação funcional ocorrem a partir da década de 1940 quando do lançamento do primeiro computador eletrônico ENIAC (***Elec-tronic Numerical Integrator and Computer***) em 1946. O ENIAC começou a ser desenvolvido no ano de 1943 com o objetivo de ser usado para o cálculo de balística durante a II Guerra Mundial (1939-1945), o que acabou por não acontecer.

Com o término da II Guerra Mundial os Estados Unidos da América (EUA) inicia com a União das Repúblicas Socialistas Soviéticas (URSS) a chamada Guerra Fria que durou até o ano de 1991 quando a URSS deixou de existir ocorrendo a separação dos países membros. O período da Guerra Fria caracterizou-se por rivalidades nas áreas: militar, econômica, política, ideológica e principalmente científica o que trouxe, inclusive, grandes avanços na área de computação culminando no que se tem na atualidade.

Um dos grandes episódios durante a Guerra Fria foi o desenvolvimento da corrida espacial entre EUA e URSS na busca da supremacia na exploração espacial durante os anos de 1957 a 1975. Isso fez com que em 1957 a URSS colocasse na órbita terrestre o primeiro satélite artificial chamado Sputnik 1 e em 1961 patrocini-

nasceu o primeiro voo orbital tripulado pelo cosmonauta Yuri Alekseyevitch Gagarin com a nave Vostok I. Em resposta, os EUA em 1969 patrocinam a operação de primeira alunagem com a nave Apolo 11 tripulada pelos astronautas Neil Armstrong, Edwin Aldrin e Michael Collins.

O uso de computadores por empresas tem sua origem durante a década de 1950. Nesse período o desenvolvimento de programas, ainda mesmo que precários, eram produzidos com linguagens de baixo nível (linguagens de máquina e assemblys). Durante 1954 é lançada pela equipe de cientista da computação da empresa IBM (*International Business Machines*) chefiada por John Backus a primeira linguagem de alto nível em estilo imperativo semiestruturado FORTRAN (*FORMula TRANslation*) para aplicações matemáticas. Depois em 1958 (após o fim da II Guerra Mundial e início da Guerra Fria) é lançada a primeira linguagem funcional em estilo declarativo no MIT (*Massachusetts Institute of Technology*) por John McCarthy chamada Lisp.

A linguagem Lisp no seu desenvolvimento tem como influência diversos trabalhos produzidos por vários cientistas, destacando-se: Haskell Brooks Curry (teoria da lógica combinatória, 1927), Kurt Gödel (teoria das funções recursivas, 1931), Alan Turing (máquina de Turing, 1936), Alonzo Church (teoria do cálculo lambda, 1936), Stephen Cole Kleene (teoria da computabilidade: funções computáveis, 1938) e Emil Leon Post (máquina de estado finito, 1943), que por sua vez são a base de fundamentação do paradigma declarativo funcional. O desenvolvimento de Lisp decorreu das necessidades de alguns cientistas da computação estarem realizando pesquisas na área de inteligência artificial, cálculo simbólico, comprovações de teoremas, sistemas baseados em regras e processamento de linguagem natural.

Apesar de ser o paradigma declarativo funcional o segundo paradigma mais antigo desenvolvido, este acabou não sendo muito utilizado devido a um problema colateral de sua funcionalidade: o alto consumo de memória, tornando seu uso viável mais recentemente. Este problema decorre do fato de que no ano de 1957, início da corrida espacial, 1 megabyte de memória custava US\$ 400.000.000,00. Quando em 1975 com a popularização dos microprocessadores e o fim da Guerra Fria 1 megabyte de memória custava US\$ 50.000,00. Em 1985 1 megabyte de memória custava por volta de US\$ 1.500,00 chegando ao preço médio de US\$ 10,00 em 2018.

A queda de preço no custo das memórias proporcionou computadores com maior capacidade de processamento tornando o uso de linguagens funcionais mais viáveis, o que justifica a popularidade e interesse no uso do paradigma declarativo funcional. No entanto, o problema maior atualmente ocorre em relação ao nível de

processamento por ser este um recurso finito e para neutralizar essa característica muitas linguagens funcionais usam ações de concorrência.

Outro fato curioso em relação ao uso de linguagens funcionais é que devido ao seu apelo matemático além do uso em aplicações científicas, essas linguagens foram, por muitos anos, usadas no contexto acadêmico e mais recentemente estão sendo usadas em empreendimentos comerciais e industriais. Isso vem decorrendo, não só devido ao baixo custo das memórias, mas a capacidade dos microprocessadores mais recentes terem suporte as ações de processamento com paralelismo e a necessidade de desenvolver programas mais dinâmicos e com maior facilidade de manutenção, características oferecidas por linguagens funcionais. Mas há ainda, o fato de que diversas linguagens de programação mais tradicionais como C++, Java, C#, Javascript, Ruby, Go, Python, entre outras vem ao longo de seus desenvolvimentos adotando capacidades de operações funcionais. Além das linguagens clássicas diversas linguagens vêm sendo lançadas com adoção direta do paradigma declarativo funcional como: Elixir, Clojure, Scala, Rust e Elm, entre outras.

1.2 PARADIGMA FUNCIONAL

A programação funcional é uma forma de ação lógica baseada no paradigma declarativo funcional, é em si uma metodologia de programação e não necessariamente uma categoria de linguagens para programação que tem por finalidade escrever programas na forma de expressões matemáticas. Neste paradigma é considerado o uso e avaliação de funções que evitam estados mutáveis de dados (funções puras) sem efeitos colaterais, pois o resultado de retorno de uma função será sempre o mesmo se a entrada também o for (transparência referencial), auxiliando o desenvolvimento de programas mais robustos, de fácil manutenção e fáceis de testar. Este paradigma define a computação como uma série de expressões matemática baseadas no uso de funções e no auxílio de estruturas de conjuntos. Os programas são escritos, neste paradigma, de forma concisa seguindo uma especificação matemática, uma vez que a estrutura principal de controle de um programa é a função (ponto de vista matemático) e o foco é direcionado no que se deve fazer e não em como deve ser feito.

É um modelo que baseia a lógica de programação a partir de um conjunto de funções ou de expressões matemáticas. Assim sendo, a função " $f(x) = x + 1$ " diz que dado um argumento " x " em " $f(x)$ ", seu domínio, há sempre o retorno do valor sucessor de " x ", sendo este " $x + 1$ ", seu contradomínio. Se fornecido para a função o argumento " 5 " o retorno será " 6 ", pois " $f(5) = 6$ ". Desta forma, o código de

um programa funcional segue uma estrutura declarativa e não imperativa, onde cada elemento de um conjunto domínio usado como entrada de uma função " $f(x)$ " terá como saída um valor de elemento do conjunto contradomínio (imagem) " $x + 1$ ".

A ação de aplicação lógica na programação caracteriza-se por ser a aplicação de métodos de raciocínio, ou seja, fornece regras e técnicas para definir se certo argumento é válido ou não. O raciocínio lógico usado na ciência matemática serve para verificar e validar teoremas e o raciocínio lógico usado na ciência da computação serve para verificar e testar se os programas estão ou não corretos.

Na programação funcional uma função pode ser usada como argumento de outra função. Devido a esta característica a programação funcional não é operada com variáveis de estado (variáveis que sofrem alterações de seus valores ao longo da execução de um programa a qualquer momento e por qualquer motivo). Isto posto, significa que se um valor é associado a uma variável no contexto funcional será imutável até a conclusão do processamento do programa.

No paradigma declarativo funcional a programação é feita a partir da separação das estruturas de dados e das funções que operam sobre as estruturas de dados definidas, normalmente na forma de listas ao estilo das máquinas de estado finito como conjuntos de dados. As funções são elementos de primeira ordem, ou seja, podem receber como argumento outras funções. Isso permite, entre outras coisas, realizar o armazenamento de funções em listas. As funções que recebem ou retornam como argumentos outras funções são chamadas de funções de ordem superior (forma funcional), sendo esta uma maneira de se fazer a reutilização de código dentro do paradigma descritivo funcional.

A programação funcional não se utiliza de laços como se faz no paradigma imperativo. Para ações de repetições usa-se o conceito de recursividade, onde uma função pode chamar a si mesma que será em momento oportuno melhor explorado.

No paradigma imperativo os programas são baseados no uso de variáveis mutáveis, atribuições de valores em variáveis que podem ser alteradas a qualquer instante e estruturas de blocos com execução de laços e a definição de sub-rotinas que operam ao estilo procedimento e função. Já na programação funcional os programas não se utilizam de atribuições em variáveis, as variáveis são imutáveis e os programas possuem unicamente a definição de funções.

1.3 LINGUAGENS FUNCIONAIS

A partir do entendimento que o paradigma declarativo funcional é um método de operação no desenvolvimento de programas de computadores e não é em si a linguagem de programação usada para esta finalidade cabe descrever sobre as ferramentas que atendem a este paradigma. É importante ressaltar que a programação funcional é baseada basicamente em duas perspectivas: uso de funções puras e a eliminação de efeitos colaterais ocorridos com o uso de variáveis mutáveis, acrescentando-se a isso o fato de as linguagens funcionais fazerem uso de estruturas de dados baseadas em listas (mapas, dicionário, tuplas, entre outras que podem ser ou não consideradas). O objetivo de uma linguagem funcional é imitar as funções matemáticas ao máximo possível (SEBESTA, 2018, p. 636).

Em relação à disponibilidade de linguagens funcionais encontram-se as chamadas linguagens puras e as linguagens impuras (ou híbridas) que operam com múltiplos paradigmas e que dão suporte ao modelo funcional. O problema da aprendizagem de qualquer linguagem de programação é o desenvolvimento do chamado preconceito tecnológico que é adquirido pelo estudante, muitas vezes dentro da sala de aula, ao achar que a linguagem que está aprendendo é a melhor que existe e que o paradigma em uso é melhor que os demais. No contexto de desenvolvimento é necessário na maior parte das vezes ter a mente aberta para combinar paradigmas e tirar o máximo proveito deles, neutralizando os pontos negativos que possam existir entre esses paradigmas. Lembre-se de que na área de desenvolvimento de software se aplica sem exceção a celebre frase de Aristóteles: "A soma das partes é maior que o todo".

As linguagens funcionais puras possuem como característica o uso de funções operacionalizadas a partir de variáveis imutáveis não gerando efeitos colaterais, não dependendo de variáveis globais ou locais, utilizando-se de recursividade para a execução de laços e realizando iteração em coleções de dados. Em contra posição linguagens funcionais impuras operam com entrada e saída de dados de forma explícita utilizando-se de variáveis mutáveis aceitando ações imperativas devido ao fato de serem híbridas e adotarem mais de um paradigma para a produção de programas.

Para o atendimento da programação funcional seja ela pura ou impura há um extenso conjunto de linguagens de programação. Uma linguagem funcional é considerada pura quando não há a alocação explícita de memória e nem declaração de variáveis mutáveis. As operações de acesso a memória ocorrem automaticamente

quando uma função é chamada. Desta forma, uma linguagem funcional típica deve dar suporte ao menos aos recursos:

- Funções de primeira classe;
- Funções de primeira ordem;
- Funções puras;
- Recursividade;
- Imutabilidade.

O paradigma funcional prevê ainda outras ações para tratamento de dados, os quais podem ter comportamentos diferentes dependendo da linguagem em uso, sendo:

- Currificação;
- Composição.

Resumidamente pode-se dizer que, do ponto de vista funcional, um programa é um conjunto de definições, que por sua vez são formadas por associações de um rótulo de identificação e um valor que representam em si certa função.

O conjunto de linguagens funcionais disponíveis para uso é muito grande, desde linguagens tradicionais até linguagens com lançamentos mais recentes. Segue breve lista de linguagens funcionais puras e impuras: linguagens puras e ano de lançamento: Agda (2007), Elm (2012), Haskell (1990), ML (1980), HOPE (1978), Miranda (1985), LambdaScript (2017) e SASL (1972); linguagens impuras e ano de lançamento: Erlang (1986), Elixir (2011), F# (2005), Lisp (1958), Logo (1967), Python (1990) e Rust (2010).

1.4 LINGUAGEM FUNCIONAL: HOPE

HOPE (oficialmente grafada com todos os caracteres no formato maiúsculo) é uma linguagem de programação experimental e minimalista que opera sob o paradigma funcional puro com definição de tipos para dados polimórficos, tipos de dados algébricos, correspondência de padrões e uso de listas preguiçosas (lista que atrasa a avaliação de uma expressão até ela ser necessária evitando repetir avaliações utilizadas anteriormente) com funções de ordem superior, sendo uma linguagem de programação bastante poderosa dentro dos limites que opera. A lin-

guagem foi desenvolvida por Rod M. Burstall, David B. Macqueen e Donald T. Sannella como trabalho apresentado para a Universidade de Edinburg na Escócia em 1978. Seu nome advém do local onde se situava o Departamento de Ciência da Computação e Inteligência Artificial da Universidade de Edinburgh: Hope Park Square (INCE & ANDREWS, 2014, p. 110).

O objetivo da linguagem **HOPE** é ser simples, permitindo a construção de programas claros e de fácil manutenção, tendo sido baseada a partir das linguagens LISP e ISWIM com influências das linguagens PROLOG, SCRATCHPAD, ML, SASL, OBJ, além das linguagens de programação produzidas por Burge e Backus (BURSTALL, MACQUEEN & SANNELLA, 1978). É a primeira linguagem a fazer uso do recurso **call-by-pattern** (chamada por padrão) que é um mecanismo de controle não determinístico que se caracteriza por ser um conjunto de condições de aplicabilidade definidas ao domínio de uma função a partir de seus argumentos.

Apesar da linguagem **HOPE** nunca ter sido usada profissionalmente por ter sido centrada dentro do contexto acadêmico, ela pode e deve ser usada para o ensino do paradigma declarativo funcional de forma descontraída e com muita comodidade.

A linguagem **HOPE** não opera com declaração de atribuição, estratégia comum em linguagens declarativas, uma vez que trabalha com ações de asserção e correspondência de padrões. Essa característica permite ao programador maior liberdade na definição de tipos de dados derivados. Por ser uma linguagem funcional, **HOPE** não possui comandos específicos para a execução de laços que quando necessários dão produzidos com ações de recursividade.

As ações operacionais da linguagem são produzidas a partir da definição de funções que representam equações matemáticas, onde o lado esquerdo da função define uma correspondência de padrão usada pela ação indicada a sua direita. A propósito, uma variável, constante, comando ou função (como uma expressão de sub-rotina) em **HOPE** é sempre uma função.

Um detalhe curioso é que a ideia de variáveis ocorre apenas dentro de funções que efetuam alguma manipulação de dado. Não existe a definição de variáveis fora do escopo de uma função. Ao mesmo tempo em que isso parece uma desvantagem, é um bom reforço na forma de pensar a solução dos programas, pois quando fora do escopo de uma função nada existe na memória. **HOPE** opera apenas com variáveis locais.

Na atualidade há duas possibilidades para uso da linguagem *HOPE* a partir de duas versões de interpretadores, muito parecidos, disponibilizados para sistemas operacionais padrão UNIX (sistema operacional FreeBSD) e Windows.

A partir da introdução da linguagem em 1978 não se tem um histórico do desenvolvimento do interesse no uso da linguagem *HOPE* e no desenvolvimento de ambientes para sua programação. Pelo que tudo indica não houve um grande interesse da comunidade computacional em dar espaço para a linguagem, o que pode vir a ser mudado por ser uma linguagem ideal para a introdução e aprendizagem do paradigma funcional por iniciantes.

Apesar dessas ocorrências não muito animadoras em agosto de 1985, Roger Baley publicou na revista BYTE um artigo intitulado "*A HOPE TUTORIAL*", o qual despertou certo interesse do público na época. Nesta ocasião foi distribuído um interpretador *HOPE* para sistema operacional PC-DOS 2.0 que permitia efetuar os testes práticos propostos no artigo.

Um dos primeiros interpretadores para a linguagem foi o programa *ICHOPE* escrito em 1984 por Victor Wu Way Hung do Imperial College de Londres. Após o artigo publicado por Roger Baley, entre os anos de 1998 e 1999 o professor Ross Paterson da Universidade City em Londres manteve a distribuição de um interpretador *HOPE* escrito em linguagem C para sistemas operacionais padrão UNIX que se tornou referência de uso no sistema operacional FreeBSD e passou a ser usado como base para outros interpretadores descendentes.

Entre os anos de 2007 e 2012, Alexander A. Sharbarshin estendeu diversas funcionalidades do interpretador *HOPE* escrito por Ross Paterson chamando seu produto como *Hopeless* sendo disponibilizado para os sistemas operacionais MacOS para plataforma PowerPC e Windows com o uso do programa Cigwin que pode ser encontrado no endereço Web <http://shabarshin.com/funny/>.

O sistema operacional Windows possui uma versão do interpretador *HOPE* chamado *Hope for Windows* desenvolvido por Marco Alfaro apresentada no ano de 2012 a partir do código fonte produzido pelo professor Ross Paterson e produzida no ambiente de desenvolvimento Visual Studio, encontrando-se sem atualização desde 2014 como pode ser constado no site do autor no endereço Web <http://hopelang.blogspot.com/>.

Em relação ao exposto nota-se que de tempos em tempos algumas pessoas procuram desenvolver trabalhos para continuação de uma linguagem de programação que possui potencial para ser usada como tantas outras linguagens funcionais.

1.5 OBTENÇÃO E INSTALAÇÃO

Considerando a disponibilidade do interpretador de linguagem **HOPE** para os sistemas operacionais FreeBSD e Windows apresenta-se a seguir as instruções de obtenção e instalação dos interpretadores.

Considerando o sistema operacional FreeBSD basta como usuário root executar a instrução a seguir e aguardar o término da operação.

```
pkg install hope
```

Assim que a instalação é concluída para fazer uso da linguagem **HOPE** basta no *prompt* de qualquer usuário do sistema executar o comando.

```
hope
```

Considerando o sistema operacional Windows sugere-se acessar o endereço Web.

```
http://hope.manzano.pro.br/
```

Localizar no texto do sítio "**Language HOPE**" a indicação "**hope.zip**" ([clique aqui](#)) e com o ponteiro do *mouse* acione o link *clique aqui* para copiar para seu computador o arquivo "**home.zip**".

A partir da raiz do disco rígido crie um diretório chamado "**hope**" e descompacte dentro deste diretório o conteúdo do arquivo "**hope.zip**". Para fazer uso da linguagem **HOPE** no sistema operacional Windows entre do diretório "**hope**" e execute o programa "**hope.exe**".

A partir da execução do ambiente interativo da linguagem **HOPE** é apresentado como *prompt* operacional de trabalho da linguagem o símbolo ">:". Para encerrar o ambiente interativo basta executar o comando "**exit**" seguido de ponto e vírgula.

```
exit;
```

A linguagem **HOPE** instalada no FreeBSD possui além do interpretador e do módulo padrão principal "**Standard.hop**" um conjunto de módulos com diversos recursos escritos pelo professor Ross Paterson. A instalação no Windows possui apenas o módulo padrão principal "**Standard.hop**".

Caso queira obter o conjunto de módulos do professor Ross Paterson para o interpretador *HOPE* para Windows acesse o endereço Web <https://github.com/dmbaturin/hope/tree/master/lib>, baixe o conteúdo do diretório "*lib*" e copie este conteúdo para o diretório "*hope*".

Apesar da possibilidade de uso dos módulos suplementares que podem ser usados como fonte de estudo os recursos abordados nesta obra levam em consideração apenas o uso do módulo "*Standard.hop*".

2

Linguagem HOPE

Este capítulo descreve estruturalmente como a linguagem HOPE é definida e indica as principais diferenças existentes entre os interpretadores utilizados.

2.1 INTERPRETADORES HOPE

Como comentado no capítulo anterior os interpretadores usados para a realização deste trabalho são os disponibilizados para os sistemas operacionais FreeBSD (Ross Paterson) e Windows (Marco Alfaro). Apesar de o interpretador Marco Alfaro ser baseado no interpretador Ross Paterson há entre eles algumas diferenças e certas funcionalidades que necessitam ser consideradas e, por vezes, ajustadas. O código fonte do projeto Ross Paterson é facilmente obtido (como já comentado) e pelo que tudo indica está em conformidade com o projeto original da linguagem, mas o código fonte Marco Alfaro não se encontra disponível e possui algumas diferenças (não documentadas no sítio do autor) em relação à forma original da linguagem e em desconformidade com a versão Ross Paterson.

Para ser mais facilmente identificado nesta obra sobre qual interpretador o foco está sendo indicado, será usada a nomenclatura **RP-HOPE** (para referenciar o projeto Ross Paterson) e **MA-HOPE** (para referenciar o projeto Marco Alfaro). Partindo-se deste escopo a versão da linguagem **HOPE** apresentada nesta obra é a que está em conformidade com os interpretadores **RP-HOPE** e **MA-HOPE**.

A principal diferença entre os interpretadores está na apresentação visual. O interpretador **MA-HOPE** mostra-se em tela colorida e o interpretador **RP-HOME** mostra-se em tela monocromática.

Quando da chamada de execução dos interpretadores o interpretador *MA-HOPE* apresenta uma imagem de *splash* com o nome do interpretador, versão e a indicação de uso do comando "*help*;" não existente no interpretador *RP-HOME*. As figuras 2.1 e 2.2 apresentam respectivamente as telas de acesso aos interpretadores *MA-HOPE* e *RP-HOME* nos sistemas operacionais Windows e FreeBSD após o uso da chamada do ambiente interativo por meio do comando "*hope*".

```
** Hope for Windows Version 1.2.2 <c> 2012 ***
**                                     **
**      Welcome to the HOPE interpreter      **
**                                     **
*****
Manzano at NOTEAUGUSTO Windows_NT on Sun Jun 17 20:31:01 2018.
help; for help.    http://www.hopenachine.co.uk
>: _
```

Figura 2.1 – Tela inicial do interpretador MA-HOPE.

```
$ hope
>: █
```

Figura 2.2 – Tela inicial do interpretador RP-HOPE.

Há certas funções internas, para certos cálculos matemáticos, existentes no interpretador *RP-HOME* que não estão disponibilizadas no interpretador *MA-HOPE* e vice-versa. Neste caso, essas diferenças serão comentadas na medida em que ocorrem sobre a apresentação deste trabalho. Além disso, há nos interpretadores a falta de eventuais funcionalidades comuns em diversas linguagens de programação

que serão desenvolvidas em módulos a serem usados juntamente da linguagem junto aos interpretadores.

Ao ser executado o comando "**help**;" no interpretador **MA-HOPE** ocorre a apresentação de uso de alguns poucos comandos do interpretador como aponta a figura 2.3. A mesma tentativa no interpretador **RP-HOPE** apresenta uma mensagem de erro como indica a figura 2.4.

```
*** Hope for Windows Version 1.2.2 <c> 2012 ***
***      Welcome to the HOPE interpreter      ***
***
*****
Manzano at NOTEAUGUSTO Windows_NT on Sun Jun 17 20:31:01 2018.
help; for help.  http://www.hopemachine.co.uk

>: help;
Hope for Windows 1.2.2
<expression>;
write <expression>;
write <expression> to "file";
display;
save filename;
exit;
>:
```

Figura 2.3 – Tela com detalhes sobre o uso do comando help MA-HOPE.

```
$ hope
>: help;
semantic error - help: undefined variable
>: █
```

Figura 2.4 – Tela com erro na execução do comando help RP-HOPE.

Independentemente de algumas diferenças existentes entre os interpretadores as ferramentas funcionam de forma semelhante atendendo em média a originalidade da linguagem **HOPE**.

Os ambientes interativos **HOPE** possuem um conjunto de comandos (sempre definidos com caracteres minúsculos) que podem ser usados para operacionalizar algumas ações do ambiente independentemente dos comandos da linguagem de programação em si, destacando-se:

- **display** mostra as definições existentes na seção atual de trabalho;
- **edit** edita módulo com editor padrão (somente UNIX com RP-HOPE);
- **exit** permite saída do ambiente interativo da linguagem;
- **help** modo ajuda – disponível apenas em MA-HOPE;
- **id** identifica e apresenta o tipo de dado de um elemento;
- **read** carrega e apresenta o conteúdo de arquivo externo;
- **save** salva na forma de módulo as definições da seção atual;
- **uses** carrega para uso um arquivo de módulo externo.

Dos comandos apresentados para o gerenciamento de ações nos ambientes interativos os comandos "**display**", "**uses**" e "**exit**" são comandos considerados padrões originais como apresentados no manual de referência "[ref_man.pdf](#)" publicado pelo professor Ross Paterson. Os demais comandos são recursos adicionais incorporados ou por Ross Paterson ou por Marco Alfaro em seus interpretadores.

O conteúdo da memória dos interpretadores pode ser listado a qualquer momento com o uso do comando "**display**".

O comando "**id**" apresenta o tipo de dado de um valor indicado. Por exemplo, se executada a instrução "**id 1**;" será apresentada como resposta "**1 : num**" indicando ser o valor "**1**" um dado do tipo "**num**" (numérico).

Todo o conteúdo definido na memória no uso dos interpretadores pode ser gravado com o comando "**save**" seguido do nome que se deseja dar ao arquivo de módulo externo. Quando um nome de arquivo é fornecido ao comando "**save**" a extensão "**.hop**" é acrescida automaticamente ao arquivo.

Para fazer uso de um módulo externo basta executar o comando "**uses**" seguido do nome do módulo ou dos módulos a serem usados separados por vírgulas sem a menção da extensão "**.hop**".

Um arquivo externo ao ambiente pode nele ser listado com o uso do comando "**read**", desde que se coloque o nome do arquivo e extensão se houver dentro de aspas inglesas.

O comando "**edit**" está disponível apenas no interpretador **RP-HOPE**, tendo como finalidade executar externamente o editor de texto padrão do sistema operacional UNIX (normalmente executado o editor de texto **vi**, como ocorre no sistema operacional FreeBSD) para editar um arquivo de módulo indicado. Os arquivos de módulo são gravados automaticamente no sistema com a extensão "**.hop**".

Para sair do ambiente interativo basta fazer uso do comando "**exit**".

2.2 TIPOS DE DADOS PRIMITIVOS

Todo computador tem como princípio básico a capacidade de processar os dados advindos do mundo externo e transformá-los, de alguma forma, em informações que sejam úteis para seus usuários. Os dados são representados por informações processadas por um computador as quais são armazenadas em memória, seja ela primária ou secundária. A linguagem **HOPE** opera com o reconhecimento automático dos tipos de dados a partir da definição de certo valor. Desta forma, a linguagem **HOPE** suporta os seguintes tipos de dados:

- **num**: tipo de dado explícito usado para representar valores numéricos inteiros ou reais com ponto flutuante;
- **char**: tipo de dado explícito usado para representar valores alfabéticos e/ou alfanuméricos delimitados entre aspas simples quando for o uso de um único caractere, podendo ser formado por mais de um caractere delimitado entre aspas inglesas;
- **truval/bool**: tipo de dado explícito usado para representar valores lógicos falso (*false*) e verdadeiro (*true*);
- **tuple**: tipo de dado implícito usado para representar conjuntos de dados delimitados entre parênteses e separados por vírgula;
- **list**: tipo de dado explícito usado para representar listas de dados delimitados entre colchetes e separados por vírgulas na forma de conjuntos, podendo ser "**list char**" para cadeia de caracteres ou "**list num**" para sequências de valores numéricos.

O tipo de dado "**truval**" é o tipo lógico padrão para a linguagem **HOPE** e disponibilizado no interpretador **MA-HOPE**. No entanto, no interpretador **RP-HOPE** este tipo de dado é referenciado como "**bool**", determinando-se mais um ponto de diferença entre os dois interpretadores.

Uma forma rápida de realizar testes para cada tipo de dado suportado pela linguagem **HOPE** é executar as instruções indicadas que após cada acionamento da tecla "**<Enter>**" apresenta o tipo de dado em uso.

Valor numérico inteiro.

```
>: 1;  
>> 1 : num
```

Valor numérico real.

```
>: 1.1;  
>> 1.1 : num
```

Valor caractere.

```
>: 'a';  
>> 'a' : char
```

Valor cadeia de caracteres.

```
>: "abc";  
>> "abc" : list char
```

Valor de uma tupla.

```
>: (1, 2, 3, 'a');  
>> (1, 2, 3, 'a') : num # num # num # char
```

Valor de uma lista.

```
>: [1, 2, 3];  
>> [1, 2, 3] : list num
```

Valor nulo.

```
>: "";  
>> nil : list char
```

O valor identificado como "**nil**" refere-se a definição de valor nulo, quando um resultado não pode representar um valor em específico, seja este valor de qualquer tipo de dado suportado pela linguagem.

Vale salientar que em relação a definição de tipos de dados na linguagem **HOPE**, tuplas aceitam dados heterogêneos, mas o mesmo não ocorre com listas, onde os dados devem ser de tipos homogêneos.

2.3 PALAVRAS CHAVE E FUNCIONALIDADES

Apesar da linguagem de programação funcional **HOPE** ser considerada minimalista ela possui uma quantidade de comandos significativos para a programação de suas ações. Segue lista dos comandos padrão da linguagem:

- **---** inicia a definição de uma função;
- **** define sinônimo para comando "lambda";
- **abstype** define tipo de dado abstrato;
- **data** declara tipo de dado derivado;
- **dec** declara uma função;
- **else** complemento ao comando "if" para ação falsa;
- **error** define mensagem de erro pelo usuário;
- **if** comando usado para a tomada de decisões;
- **in** complemento aos comandos "let" e "letrec";
- **infix** define precedência de operador esquerdo;
- **infixr** define precedência de operador direito;
- **infixrl** define sinônimo para comando "infixr";
- **lambda** define função anônima;

- **let** simplificação de expressões;
- **letrec** simplificação de expressões (restritivo);
- **mu** define simplificação de tipos de dados recursivos;
- **nonop** trata como função operadores aritmético/lógico;
- **private** define componentes privados de um módulo;
- **then** complemento ao comando "if" para ação verdadeira;
- **type** define sinônimo para um tipo de dados específico;
- **typevar** define tipo variável de dado a ser usado com "dec";
- **use** define sinônimo para uso do comando "uses";
- **uses** coloca em uso módulo externo;
- **where** simplificação alternativa de expressões;
- **whererec** simplificação alternativa de expressões (restritivo);
- **write** apresenta os dados de uma expressão.

Além dos recursos apresentados há um conjunto suplementar de comandos que são encontrados em outras distribuições da linguagem **HOPE**, como aponta Ross Paterson em seu manual de referência.

Esses comandos são encontrados nos interpretadores **MA-HOPE** e **RP-HOME** a título de compatibilidade com outras implementações da linguagem, destacando-se os comandos (que não são usados neste trabalho):

- **end** finaliza definição de módulos;
- **module** define módulos de operação;
- **pubconst** define constante dentro de módulo;
- **pubfun** define função dentro de módulo;
- **pubtype** define tipo de dado dentro de módulo.

Para auxiliar as ações dos comandos os interpretadores **MA-HOPE** e **RP-HOME** disponibilizam um conjunto de funções matemáticas e de conversão de dados. Veja as funções matemáticas existentes nos interpretadores usados.

- **abs(x)** valor absoluto de "x" (sempre positivo);
- **acos(x)** arco cosseno de "x";
- **asin(x)** arco seno de "x";
- **atan(x)** arco tangente de "x";
- **atan2(x, y)** arco tangente do quociente de "x" e "y";
- **ceil(x)** arredonda para cima valor de "x";
- **cos(x)** cosseno de "x";
- **cosh(x)** cosseno hiperbólico de "x";
- **exp(x)** exponencial de "x";
- **floor(x)** arredonda para baixo valor de "x";
- **hypot(x, y)** raiz quadrada do comprimento da hipotenusa;
- **log(x)** logaritmo natural de "x";
- **log10(x)** logaritmo de "x" na base 10;
- **pow(b, i)** potência de "b" sobre "i";
- **sin(x)** seno de "x";
- **sinh(x)** seno hiperbólico de "x";
- **sqrt(x)** raiz quadrada de "x";
- **succ(x)** mostra valor sucessor de "x";
- **tanh(x)** tangente hiperbólica de "x";

Algumas das funções apresentadas podem retornar tipos de respostas diferentes, dependendo do interpretador em uso. Por exemplo, a indicação de retorno de valor numérico infinito no interpretador **RP-HOPE** é apresentada como "**inf : num**" e no interpretador **MA-HOPE** é apresentado como "**1.#INF : num**". Em relação à apresentação de retorno de valor não numérico ocorre no interpretador **RP-HOPE** a indicação do resultado como "**nan : num**" e no interpretador **MA-HOPE** a indicação do resultado como "**-1.#IND : num**".

Dentro do escopo de diferenças existentes entre os interpretadores há cinco funções existentes no interpretador **RP-HOPE** que não são disponibilizadas no interpretador **MA-HOPE**, sendo:

- **erf(x)** erro de Gauss de "x";
- **erfc(x)** erro complementar de Gauss de "x";
- **acosh(x)** arco cosseno hiperbólico de "x";
- **asinh(x)** arco seno hiperbólico de "x";
- **atanh(x)** arco tangente hiperbólico de "x".

Para que essas funções possam ser usadas no interpretador **MA-HOPE** será necessário fazer suas definições manualmente. Essa tarefa será produzida em momento oportuno no decorrer do estudo desta obra.

Em relação à ação de conversão de tipos de dados e manipulação de dados da linguagem **HOPE** gerenciadas pelos interpretadores **MA-HOPE** e **RP-HOME** são disponibilizadas as funções:

- **ord('x')** apresenta código ASCII do caractere "x";
- **chr(x)** apresenta caractere "x" do código ASCII informado;
- **num2str(x)** converte numero na forma de cadeia numérica;
- **str2num("x")** converte cadeia numérica na forma de número.

Os interpretadores **MA-HOPE** e **RP-HOME** não aceitam a definição de valores negativos com uso do símbolo "-". Para atender a esta necessidade é ideal que seja criada uma função para representar valores negativos. Este efeito será abordado mais adiante nesta obra.

2.4 OPERADORES FUNCIONAIS

A linguagem **HOPE** para suas operações de processamento faz uso de um conjunto de operadores que auxiliam diversas ações aritméticas, relacionais, lógicas e auxiliares.

São operadores aritméticos:

- **+** adição;
- **-** subtração;
- ***** multiplicação;

- `/` divisão com quociente real;
- `<=` define asserção de operações matemáticas;
- `div` divisão com quociente inteiro;
- `mod` resto da divisão.

São operadores relacionais:

- `=` igual a;
- `>` maior que;
- `>=` maior ou igual a;
- `<` menor que;
- `=<` igual a ou menor que (menor ou igual a);
- `/=` diferente de.

São operadores lógicos:

- `not` negação;
- `and` conjunção;
- `or` disjunção inclusiva.

São operadores auxiliares:

- `=>` define corpo de expressão lambda;
- `|` define continuidade de expressão lambda;
- `==` define dados para expressão qualificada;
- `++` atribuição de construtores de tipos definidos;
- `#` separador de argumentos na definição de funções;
- `()` definição de tuplas ou argumentos de funções;
- `[]` definição de listas;
- `:` separador de estruturas de uma função;
- `::` constitui lista de valores;
- `!` define linha de comentário.

As operações de processamento matemático e lógico, além do estabelecimento de ações complementares são amplamente usadas na linguagem **HOPE** para a produção de soluções computacionais dentro do paradigma declarativo funcional.

2.5 ESTRUTURA SINTÁTICA BÁSICA

No paradigma de programação declarativo funcional os programas são escritos na forma de funções. O conceito básico de função utilizado na programação funcional é o mesmo usado na ciência matemática e embasa todas as definições a serem programadas. Neste sentido, os programas escritos na linguagem **HOPE**, ou melhor, dizendo as funções são definidas a partir da estrutura sintática:

```
dec <rótulo> : <argumento1> [# <argumentoN> -> <retorno>];
```

Em que "**dec**" (*declaration*) é o comando que estabelece uma função, rótulo é o nome da função, argumento é o tipo do parâmetro passado a função no caso da definição de variáveis ou simulação de constantes. O operador ":" (dois pontos que significa "recebe") é usado para separar o rótulo de identificação da função e a definição do argumento (que poderá ser o primeiro argumento quando existir mais de argumento). Para os casos em que a função deve receber mais de argumentos e retornar uma resposta a sua ação usa-se o trecho opcional indicado entre parênteses. O operador "#" (trilha que significa "e") é usado para separar aos argumentos a serem usados na função. O símbolo "->" (seta para à esquerda que significa gera ou fornece) indica o valor de retorno da função e o símbolo de ";" (ponto e vírgula) que finaliza a instrução. As indicações "**argumento1**", "**argumentoN**" e "**retorno**" devem ser definidos a partir dos tipos de dados suportados pela linguagem "**num**", "**char**", "**truval/bool**", "**tuple**" e "**list**".

O conjunto de tipos de dados para a definição de argumentos e estabelecimento de retorno pode ser realizado com diferentes tipos de dados. Assim sendo, são possíveis as definições, entre outras variações:

```
dec função : num;  
dec função : num # char -> num;  
dec função : list num # char -> num # num;  
dec função : list num # char -> list char;  
dec função : num # num -> truval;
```

A declaração de funções pode ocorrer a partir do uso de funções internas existentes na linguagem ou a partir de outras funções definidas pelo programador, além de poderem ser operacionalizadas a partir de ações de recursividade.

A declaração inicial de uma função na linguagem *HOPE* é iniciada com o comando "**dec**" que é o construtor de uma função.

```
dec soma : num # num -> num;
```

O corpo operacional dde uma função é definido a partir do uso do comando "**---**" (três traços sucessivos que pode ser entendido como comando **def**). Desta forma, para uma função chamada soma pode-se definir o código básico da operação:

```
--- soma (a, b) <= a + b;
```

O lado direito da definição de uma função com o comando "**---**" após o símbolo "**<=**" determina o padrão de comportamento da função, seus elementos de entrada e saída, neste caso "**a**" e "**b**" indicados à esquerda. A este efeito da-se o nome de *correspondência de padrões* que pode ser implementada na função a partir de funções internas, funções definidas pelo programador ou outras ações de processamento matemático e lógico.

Os argumentos "**a**" e "**b**" definidos junto ao rótulo "**soma**" representam os valores de entrada e a operação "**a + b**" é o resultado de retorno da função. O símbolo "**<=**" que significa "está definido como" efetua a asserção do resultado da operação do lado esquerdo a função definida do lado direito.

Observe o código completo da função "**soma**".

```
dec soma : num # num -> num;  
--- soma (a, b) <= a + b;
```

Para ver a função em ação execute a instrução.

```
soma(1, 2);
```

Será apresentado o resultado.

```
3 : num
```

Os argumentos definidos junto à função "**soma**" e identificados como "**a**" e "**b**" são, na verdade, as definições das variáveis a serem usadas na linguagem **HOPE**. O conceito de variável assume significado um pouco diferente em linguagens funcionais, pois são posições de memória que assumem um valor e uma vez assumido este valor, não poderá mais ser alterado. Linguagens funcionais operam com o uso de **variáveis imutáveis**, não sendo isso diferente na linguagem **HOPE**. Não é possível na linguagem **HOPE** definir variáveis fora do escopo de uma função.

A estrutura de comunicação do programador com o computador por intermédio da linguagem **HOPE** é produzida a partir de caracteres de pontuação como parênteses, vírgula, ponto e vírgula, diversos operadores (já comentados), colchetes, letras, números (dígitos), sublinhado, aspas simples e aspas inglesas.

Como a base da programação **HOPE** são as definições de funções é importante ter em mente as estruturas básicas operacionais de definição dos estilos de funções que podem ser usadas.

- operações aritméticas com o formato:
num # num -> num
- operações relacionais diferentes de igualdade com o formato:
() # (**) -> truval**
- operações relacionais de igualdade com o formato:
(*) # (*) -> truval
- operações lógicas de conjunção ou disjunção inclusiva com o formato:
truval # truval -> truval
- operações lógicas de negação com o formato:
truval -> truval
- operadores de manipulação de listas com o formato:
list (*) operador list (*) -> list (*)

A sinalização "**(**)**" significa que os elementos operacionalizados podem ser dos tipos de dados "**num**" ou "**char**" e a sinalização "**(*)**" significa que os elementos podem ser de qualquer tipo de dados aceitos pela linguagem.

Em relação às possibilidades de uso e definições de argumentos cabe uma parte sobre este tema, pois a estrutura usada na construção do protótipo de funções atende diversas possibilidades.

O argumento de retorno de uma função é sempre o último argumento definido no protótipo da função. Os argumentos de entrada, bem como os argumentos de saída de uma função podem ser definidos a partir da estrutura: valor simples; uma lista de valores; uma tupla de valores; uma lista de tuplas ou uma tupla de listas. Note as possibilidades de retorno de uma função considerando o uso dos tipos de dados "**num**" e "**list**".

```
num
list num
num # num
list (num # num)
list num # list num
```

Para ver a apresentação dessas estruturas que podem ser definidas como argumentos de entrada ou de saída execute as instruções seguintes.

```
>: 1;
>> 1 : num

>: [1, 1, 1];
>> [1, 1, 1] : list num

>: [(1, 1, 1), (1, 1, 1)];
>> [(1, 1, 1), (1, 1, 1)] : list (num # num # num)

>: ([1, 1, 1], [1, 1, 1]);
>> ([1, 1, 1], [1, 1, 1]) : list num # list num
```

Note que a definição de listas de valores é estabelecida entre colchetes e a definição de tuplas é estabelecida entre parenteses.

Atente na sequência para a estrutura de declaração de funções sem uso de argumentos de entrada. As funções que não possuem argumentos de entrada são úteis para a definição de constantes internas (constantes implícitas) ou a definição de variáveis imutáveis.

```
dec função : tipo;  
dec função : list tipo;  
dec função : tipo # tipo;  
dec função : list (tipo # tipo);  
dec função : list tipo # list tipo;
```

A definição de argumentos de entrada fica evidente quando se utiliza como símbolo de separação de seções de argumentos "**->**", onde o que estiver à esquerda são os argumentos de entrada e o que estiver à direita são os argumentos de saída. Observe os exemplos seguintes considerando um argumento de entrada com valor simples e as diversas e possíveis formas de saída.

```
dec função : tipo -> tipo;  
dec função : tipo -> list tipo;  
dec função : tipo -> tipo # tipo;  
dec função : tipo -> list (tipo # tipo);  
dec função : tipo -> list tipo # list tipo;
```

A partir da estrutura sintática básica da morfologia da declaração de uma função na linguagem **HOPE** cabe considerar as possibilidades de definição de argumentos de entrada, ou seja, os argumentos definidos à esquerda do símbolo "**->**", além da definição simples "**tipo**" anteriormente apontada. Assim sendo, são formas válidas de argumentos de entrada os seguintes estilos.

```
tipo -> <retorno>;  
list tipo -> <retorno>;  
tipo # tipo -> <retorno>;  
list tipo # list tipo -> <retorno>;  
tipo # list tipo -> <retorno>;  
list tipo # tipo -> <retorno>;  
tipo -> tipo -> <retorno>;  
list tipo -> list tipo -> <retorno>;  
tipo -> list tipo -> <retorno>;  
list tipo -> tipo -> <retorno>;
```

Quando se usa o símbolo "#" para separar os argumentos de entrada esses deverão ser usados separados por vírgula dentro de parênteses. No caso de uso do símbolo "->" os argumentos de entrada serão separados por espaços em branco sem delimitação entre parênteses, excetuando-se a definição do argumento de saída "**algun retorno**" após o último símbolo "->" que segue os detalhes já apresentados.

A definição dos argumentos de uma função forma o que se chama de *aridade* que é a quantidade de argumentos de entrada de uma função. Por exemplo, uma função com argumentos de entrada na forma "**tipo # tipo**" ou "**tipo -> tipo**" é dita como uma função de *aridade dois*.

A definição de argumentos de entrada aceita além das formas indicadas a definição da recepção de funções (funções de primeira classe) quando são delimitadas como argumentos entre parênteses. Neste sentido, o protótipo de funções que utilizam funções como argumentos podem ser definidas como as formas seguintes, considerando-se entre parênteses a possibilidade ou não de fazer uso do tipo "**list**".

```
[list] tipo # ([list] tipo -> <retorno>) -> <retorno>;  
[list] tipo -> ([list] tipo -> <retorno>) -> <retorno>;
```

Todos os detalhes apresentados para a definição dos argumentos de entrada são válidos na definição dos argumentos de função que por sua vez fazem uso de argumentos de entrada.

Na apresentação deste estudo alguma forma das formas possíveis de definição de argumentos, será em algum momento utilizado.

Página propositalmente em branco

3

Programação funcional básica

Este capítulo apresenta informações iniciais sobre o uso preliminar da linguagem HOPE. Neste sentido, é demonstrado como fazer a definição de constantes e gerar o arquivo de módulo "constants". São indicados e exemplificados o uso de operadores aritméticos e lógicos. São apresentadas informações sobre a definição e uso de tomadas de decisão e execução de recursão (simples e de cauda) para a simulação de laços. É apresentado o conjunto básico de funcionalidades encontradas nos interpretadores da linguagem HOPE em uso. No final é orientado como o programador produzir suas funções colocando-as em um arquivo de módulo.

3.1 DEFINIÇÃO DE CONSTANTE IMPLÍCITA

A linguagem **HOPE** por ser minimalista não possui a definição de valores constantes implícitos em sua estrutura interna. Em particular, o interpretador **RP-HOPE** possui a biblioteca de módulo "**arith.hop**", a qual tem a definição de duas constantes matemáticas: **pi** e **Euler** identificada com o **e**, que quando executada

O arquivo de módulo "**arith**", bem como outros módulos disponíveis para o interpretador **RP-HOPE** para serem usados no interpretador **MA-HOPE** deverão ser adaptados. A principal mudança no código interno de cada arquivo de módulo é modificar o tipo de dado "**bool**" como "**truval**".

Para definir um conjunto de constantes implícitas, como exemplo, que possam auxiliar diversas operações matemáticas é escrito na sequência um conjunto de constantes no mesmo padrão encontrado nas linguagens de programação C/C++.

A diferença na definição de uma função comum e uma função constante, no caso, da linguagem **HOPE**, é a forma de declaração de seu protótipo e posterior definição de seu código operacional. Veja.

```
dec função_comum : num -> num;
dec constante : num;
```

Na declaração da função (indicação de seu protótipo) nomeada com o rótulo "**Função_comum**" é expressa a recepção de um argumento numérico e o retorno de um valor, também, numérico após o símbolo de dois pontos, caracterizando a definição de uma função comum que efetua algum processamento matemático ou lógico. No caso da declaração de uma função constante é declarado apenas o argumento de retorno após o símbolo de dois pontos.

A declaração de funções comuns ou constantes na linguagem *HOPE* pode ser feita de forma isolada como já indicadas ou na forma de grupos seguindo os seguintes estilos.

```
dec func1, func2, func3 : num -> num;
dec func4, func5, func6 : num;
```

A partir dessas considerações segue código de declaração e definição de um pequeno conjunto de funções do tipo constante a ser escrito a partir do *prompt* da linguagem ">:".

```
dec m_1_pi, m_2_pi, m_2_sqrtpi, m_e, m_ln10, m_ln2, m_log10e,
    m_log2e, m_pi, m_pi_2, m_pi_4, m_sqrt1_2, m_sqrt2 : num;
--- m_1_pi    <= 0.318309886183791; ! 1 / (acos(0) * 2)
--- m_2_pi    <= 0.636619772367581; ! 2 / (acos(0) * 2)
--- m_2_sqrtpi <= 1.12837916709551; ! 2 / sqrt(acos(0) * 2)
--- m_e       <= 2.71828182845905; ! exp(1)
--- m_ln10    <= 2.30258509299405; ! log(10)
--- m_ln2     <= 0.693147180559945; ! log(2)
--- m_log10e  <= 0.434294481903252; ! log10(exp(1))
--- m_log2e   <= 1.44269504088896; ! log(exp(1)) / log(2)
--- m_pi      <= 3.14159265358979; ! acos(0) * 2
--- m_pi_2    <= 1.5707963267949; ! acos(0) * 2 / 2
--- m_pi_4    <= 0.785398163397448; ! acos(0) * 2 / 4
--- m_sqrt1_2 <= 0.707106781186547; ! 1 / sqrt(2)
--- m_sqrt2   <= 1.4142135623731; ! sqrt(2)
```

O grupo de funções que representam a definição de constantes matemáticas ao estilo C/C++ são seguidos dos valores de cada constante após o símbolo "<=" e o nome da função antes do símbolo "<=".

Cada linha de código de certa função é precedida do símbolo de exclamação "!" que indica um trecho de comentário a sua direita. Neste caso, é definido como comentário as funções embutidas da linguagem *HOPE* que podem ser usadas para gerar os valores definidos para cada constante.

Para fazer uso de cada função constante, basta indicar seu nome seguido de ponto e vírgula e acionar a tecla "<Enter>".

Após declarar com o comando "**dec**" e definir com o comando "---" as funções constantes anteriores, pode-se ver o código de cada função com o uso do comando.

```
display;
```

Que apresentará cada uma das funções definidas em memória como se tivessem sido declaradas de forma isolada.

```
dec m_sqrt2 : num;
--- m_sqrt2 <= 1.4142135623731;

dec m_sqrt1_2 : num;
--- m_sqrt1_2 <= 0.707106781186547;

dec m_pi_4 : num;
--- m_pi_4 <= 0.785398163397448;

dec m_pi_2 : num;
--- m_pi_2 <= 1.5707963267949;

dec m_pi : num;
--- m_pi <= 3.14159265358979;

dec m_log2e : num;
--- m_log2e <= 1.44269504088896;
```

```
dec m_log10e : num;
--- m_log10e <= 0.434294481903252;

dec m_ln2 : num;
--- m_ln2 <= 0.693147180559945;

dec m_ln10 : num;
--- m_ln10 <= 2.30258509299405;

dec m_e : num;
--- m_e <= 2.71828182845905;

dec m_2_sqrtpi : num;
--- m_2_sqrtpi <= 1.12837916709551;

dec m_2_pi : num;
--- m_2_pi <= 0.636619772367581;

dec m_1_pi : num;
--- m_1_pi <= 0.318309886183791;
```

Os dados constantes da memória, como já comentado, podem ser gravados em um arquivo especial chamado módulo que representa uma biblioteca de funcionalidades da linguagem *HOPE*. Neste sentido, considere gerar um módulo chamado "**constant**" a partir da instrução.

```
save constant;
```

Para fazer um teste encerre a execução do ambiente interativo em uso da linguagem *HOPE* e carregue-novamente. Na sequência, execute a instrução.

```
uses constant;
```

A partir deste momento qualquer uma das constantes definidas podem ser usadas por estarem carregadas na memória. O código carregado pelo arquivo de módulo não é mais apresentado com o uso do comando "**display**". Caso queira ver o código do arquivo pode ser usada a instrução.

```
read "constant.hop";
```

Que apresenta sequencialmente o conteúdo do módulo com a indicação dos símbolos "**\n**" entre as declarações e definições das funções existentes no arquivo de módulo carregado, os quais são usados internamente para pular linha quando o arquivo é carregado ou usado em um programa de editor de textos convencional.

Se o arquivo de módulo "**constant.hop**" for carregado em um editor de textos convencional sua forma de apresentação se parecerá com o estilo.

```
dec m_sqrt2 : num;
dec m_sqrt1_2 : num;
dec m_pi_4 : num;
dec m_pi_2 : num;
dec m_pi : num;
dec m_log2e : num;
dec m_log10e : num;
dec m_ln2 : num;
dec m_ln10 : num;
dec m_e : num;
dec m_2_sqrtpi : num;
dec m_2_pi : num;
dec m_1_pi : num;

--- m_sqrt2 <= 1.4142135623731;

--- m_sqrt1_2 <= 0.707106781186547;

--- m_pi_4 <= 0.785398163397448;

--- m_pi_2 <= 1.5707963267949;
```

```
--- m_pi <= 3.14159265358979;  
  
--- m_log2e <= 1.44269504088896;  
  
--- m_log10e <= 0.434294481903252;  
  
--- m_ln2 <= 0.693147180559945;  
  
--- m_ln10 <= 2.30258509299405;  
  
--- m_e <= 2.71828182845905;  
  
--- m_2_sqrtpi <= 1.12837916709551;  
  
--- m_2_pi <= 0.636619772367581;  
  
--- m_1_pi <= 0.318309886183791;
```

Entre a definição de função comum apresentada no capítulo anterior e a definição da função constante apresentada neste capítulo se tem as bases operacionais básicas para iniciar um estudo mais embasado da linguagem *HOPE*.

3.2 OPERAÇÕES ARITMÉTICAS

Operações matemáticas sobre variáveis e constantes são operacionalizadas matematicamente a partir de certos operadores e funcionalidades aritméticas, destacando-se as ações como adição, subtração, multiplicação, divisão, potenciação e radiciação, entre outras possibilidades. Observe a seguir os resultados obtidos com a linguagem *HOPE* a partir de seu uso como calculadora.

```
>: 5 + 2;  
>> 7 : num
```

```
>: 5 * 2;
```

```
>> 10 : num
```

```
>: 5 - 2;
```

```
>> 3 : num
```

```
>: 5 / 2;
```

```
>> 2.5 : num
```

```
>: 1 + 2 * 3;
```

```
>> 7 : num
```

```
>: (1 + 2) * 3;
```

```
>> 9 : num
```

```
>: (1 + b) * a / 3;
```

```
>> 1 : num
```

```
>: 5 div 2;
```

```
>> 2 : num
```

```
>: 5 mod 2;
```

```
>> 1 : num
```

```
>: pow(2, 3);
```

```
>> 8 : num
```

As operações aritméticas na linguagem *HOPE* não diferenciam dados numéricos inteiros ou reais, podendo-se usá-los em conjunto sem nenhuma dificuldade operacional. No entanto, é importante considerar que no uso de valores reais é necessário que o valor seja expresso de forma completa, não sendo aceitos valores numéricos reais expressos como ".5" ou "1.", devendo-se informar os valores como "0.5" ou "1.0".

3.3 OPERAÇÕES LÓGICAS

Operações lógicas sobre variáveis e constantes são operacionalizadas a partir de certos operadores e funcionalidades lógicas e relacionais. Veja a seguir os resultados obtidos com a linguagem *HOPE* a partir do uso de operações relacionais. Para cada operação indicada é usada linha de comentário da linguagem descrevendo o resultado a ser apresentado pela operação.

Para ilustrar algumas das ações operacionais relacionais serão definidas duas constantes como variáveis imutáveis internas que são usadas em cada uma das expressões lógicas que retornarão um dos valores "false" ou "true". No interpretador *MA-HOPE* será usado "truval" e para *RP-HOPE* será usado "bool".

```
>: dec a : num; ! definicao da variavel imutavel "a" como numerica.
```

```
>: --- a <= 1; ! assercao do valor "1" junto a variavel "a".
```

```
>: dec b : num;
```

```
>: --- b <= 2;
```

A seguir são indicadas as formas apresentadas nos interpretadores *MA-HOPE* e *RP-HOPE* respectivamente.

```
>: a = b; ! igual a
```

```
>> false : truval
```

```
>: a > b; ! maior que
```

```
>> false : truval
```

```
>> a >= b; ! maior ou igual a
```

```
>> false : truval
```

```
>: a < b; ! menor que
```

```
>> true : truval
```

```
>: a <= b; ! menor ou igual a
```

```
>> true : truval
```

```
>: a = b; ! igual a
```

```
>> false : bool
```

```
>: a > b; ! maior que
```

```
>> false : bool
```

```
>> a >= b; ! maior ou igual a
```

```
>> false : bool
```

```
>: a < b; ! menor que
```

```
>> true : bool
```

```
>: a <= b; ! menor ou igual a
```

```
>> true : bool
```



```
>: a /= b; ! diferente de
>> true : truval
```

```
>: a /= b; ! diferente de
>> true : bool
```

Para quem está acostumado com outras linguagens de programação funcionais ou não é importante atentar especialmente para dois dos operadores relacionais usados pela linguagem *HOPE*, sendo "**=<**" para determinar se o primeiro valor da expressão "*é menor igual a*" ao (e não como ocorre em outras linguagens: maior ou igual a) segundo valor da expressão e o operador "**/=**" que represente "*diferente de*", levemente diferente de outras linguagens.

Os interpretadores *MA-HOPE* e *RP-HOPE* possuem internamente definidos no arquivo de módulo "**Standard.hop**" os operadores lógicos "**not**", "**and**" e "**or**", não estando disponível o operador lógico "**xor**" que poderá ser manualmente definido. Observe em seguida a tabela verdade para os operadores lógicos suportados.

Tabela verdade do operador lógico "and"		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Falso
Falso	Verdadeiro	Falso
Falso	Falso	Falso
Tabela verdade do operador lógico "or"		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Verdadeiro
Falso	Verdadeiro	Verdadeiro
Falso	Falso	Falso
Tabela verdade do operador lógico "not"		
Condição		Resultado lógico
Verdadeiro		Falso
Falso		Verdadeiro

Observe a seguir alguns exemplos de uso dos operadores lógicos dos interpretadores *MA-HOPE* e *RP-HOPE*.

```
>: (1 = 1) and (1 = 1);
```

```
>> true : truval
```

```
>: (1 = 1) and (1 = 0);
```

```
>> false : truval
```

```
>: (1 = 0) and (1 = 1);
```

```
>> false : truval
```

```
>: (1 = 0) and (1 = 0);
```

```
>> false : truval
```

```
>: (1 = 1) or (1 = 1);
```

```
>> true : truval
```

```
>: (1 = 1) or (1 = 0);
```

```
>> true : truval
```

```
>: (1 = 0) or (1 = 1);
```

```
>> true : truval
```

```
>: (1 = 0) or (1 = 0);
```

```
>> false : truval
```

```
>: not (1 = 1);
```

```
>> false : truval
```

```
>: not (1 = 0);
```

```
>> true : truval
```

Os operadores lógicos "**and**" e "**or**" são usados quando há a necessidade de se utilizar mais de uma condição para a tomada de certa decisão e o operador "**not**" é usado para negar certa condição.

3.4 TOMADA DE DECISÃO

A linguagem **HOPE** proporciona duas formas de se definir expressões condicionais. Uma de forma indireta (implícita) sem o uso de comando específico para a ação e outra forma direta (explícita) com o uso dos comandos **"if"**, **"then"** e **"else"**.

Para exemplificar ação de tomada de decisão indireta considere a definição de uma função chamada **"mult"** que apresente o resultado da multiplicação de dois valores numéricos diferentes de zero. Se qualquer um dos valores for definido como zero deverá ser retornado como resultado o valor diferente de zero. Sendo os dois valores diferentes de zero deve ser retornado o cálculo de multiplicação dos valores informados.

```
dec mult : num # num -> num;  
--- mult (0, _) <= _;  
--- mult (_, 0) <= _;  
--- mult (a, b) <= a * b;
```

Note o uso do símbolo **"_"** *underline* que representa o uso de um valor curinga, ou seja, de um valor qualquer. Internamente a linguagem **HOPE** considera este símbolo uma variável indefinida. Neste caso, se o primeiro argumento for zero da primeira definição ou o segundo argumento for zero da segunda definição ocorrerá o retorno apenas do valor curinga, que pode ser qualquer valor fornecido.

Observe que as duas primeiras definições são formas implícitas de verificar se certa condição é verdadeira e sendo uma ação será então executada. Se os valores passados forem diferentes de zero será retornado o valor da multiplicação dos argumentos fornecidos.

Veja os resultados apresentados após cada uma das execuções possíveis.

```
>: mult(0, 2);  
>> 2 : num  
  
>: mult(3, 0);  
>> 3 : num  
  
>: mult(3, 2);
```

```
>> 6 : num
```

Não haveria nenhum problema em definir a função com o uso de argumentos explícitos no lugar de um argumento indefinido simbolizado pelo símbolo *underline*. Assim sendo, a forma seguinte também é válida considerando uma função que apresente a adição de dois valores diferentes de zero e a manutenção do valor se qualquer um dos argumentos for zero.

```
dec adic : num # num -> num;  
--- adic (0, b) <= b;  
--- adic (a, 0) <= a;  
--- adic (a, b) <= a + b;
```

Ao fazer um teste da função "**adic**" observar-se-á a apresentação dos resultados de acordo com as condições estabelecidas.

A estrutura de tomada de decisão executada de forma direta usa os comandos "**if**", "**then**" e "**else**" como descrito.

```
if <condição> then <ação verdadeira> else <ação falsa>;
```

Onde "**condição**" é a definição de uma expressão formada pela relação lógica de um argumento relacionado com outro argumento ou a relação lógica definida entre um argumento com um valor constante. Se a condição for verdadeira é executado o trecho "**ação verdadeira**" após o comando "**then**". Não sendo a condição verdadeira é executado o trecho "**ação falsa**" após o comando "**else**".

Como exemplo de tomada de decisão direta considere uma função chamada **verifica** que indica se um valor fornecido é par ou impar. Desta forma, considere o código de função seguinte.

```
dec verifica : num -> list char;  
--- verifica (x) <= if (x mod 2 = 0) then "par" else "impar";
```

O uso dos parênteses na definição do argumento da função é opcional quando a quantidade de argumentos é igual a um. Quando houver mais de um argumento o uso de parênteses é obrigatório. Os parênteses usados na definição da condição também podem ser omitidos. Assim sendo, também é válida a definição da função "**verifica**" indicada a seguir.

```
dec verifica : num -> list char;
```

```
--- verifica x <= if x mod 2 = 0 then "par" else "impar";
```

Para testar essa funcionalidade execute as instruções seguintes observando-se os resultados apresentados.

```
>: verifica(3);  
>> "impar" : list char
```

```
>: verifica(6);  
>> "par" : list char
```

A condição definida para o comando **"if"** com o uso do operador aritmético **"mod"** proporciona a obtenção do valor do resto da divisão do valor do argumento **"x"** em relação ao valor da constante **"2"**. Se o valor retornado for zero isso indica que o valor do argumento **"x"** é par, caso contrário o valor é impar.

O exemplo anterior demonstra o uso de tomada de decisão composta, identificada com o uso do comando **"else"**. A linguagem **HOPE** não oferece recurso para execução de tomada de decisão simples, quando se omite o uso do comando **"else"**.

No entanto, é possível simular o uso de tomada de decisão simples. Assim sendo, considere uma função **"mult10"** que apresente mensagem informando se o valor informado é múltiplo de 10. Não sendo múltiplo de 10 a função não apresenta nenhuma resposta.

```
dec mult10 : num -> list char;  
--- mult10 x <= if x mod 10 = 0 then "multiplo de dez" else "";
```

Ao ser executada a função **"mult10"** poderá ocorrer a apresentação da mensagem prevista ou da saída **"nil : list char"**, como demonstra as formas de uso indicadas a seguir.

```
>: mult10(5);  
>> nil : list char  
  
>: mult10 20;  
>> "multiplo de dez" : list char
```

Observe que a função **"mult10"** é exemplificada de duas formas diferentes. A primeira forma o uso ocorre com parênteses e a segunda forma sem parênteses. Vale

ressaltar que o uso de parênteses é opcional quando a quantidade de argumentos for um. Para casos em que há mais de um argumento o uso de parênteses se torna obrigatório.

O uso da sequência de caracteres nulos com a definição das aspas inglesas sem nenhum caractere após o comando **"else"** gera a apresentação do resultado **"nil : list char"** quando fornecido um valor que não seja múltiplo de **"10"**. Valores múltiplos de **"10"** terão a apresentação da mensagem, informado a ocorrência.

Para auxiliar eventual possibilidade de uso de indicação de mensagem personalizada de erro para alguma ação pode-se lançar mão do comando **"error"** como o exemplo seguinte.

```
dec posit : num -> list char;  
--- posit n <= if n >= 0 then "positivo"  
    else error ("entrada invalida");
```

Agora execute a instrução.

```
>: posit(2);  
>> "positivo" : list char  
  
>: posit(0 - 2);  
user error - forneça valor positivo
```

Observe que a mensagem de erro definida junto ao comando **"error"** é apresentada após a indicação de ser uma mensagem de erro personalizada a partir da informação **"user error"**.

3.5 RECURSÃO COMO LAÇOS

Uma linguagem de programação tem por vezes a necessidade de repetir trechos de código de programa. Isso não é muito diferente em linguagens funcionais. No entanto, linguagens funcionais puras, como é o caso da linguagem **HOPE**, não possuem comandos para a efetivação de laços. A repetição de trechos de código deve ser produzida a partir do uso de ações recursivas.

Devido à característica operacional típica de uma função que é retornar sempre uma resposta a sua ação, é possível desenvolver funções que fazem chamadas a

si mesmas. Esse efeito denomina-se *recursividade* e permite executar o processamento de certo cálculo sem laço. Uma função recursiva é uma estrutura simples que representa um algoritmo de cálculo complexo (MENESES, 2013, p.216).

O uso de recursividade proporciona a escrita de um código mais elegante com alto grau de abstração. No entanto, para que uma função recursiva seja bem definida é importante levar a cabo duas propriedades comentadas por Lipschutz & Lipson (2013, p. 51):

- deve haver argumentos, chamados *valores bases*, para os quais a função em hipótese alguma não faz referência a si mesma;
- cada vez que a função se retira a si mesma, o resultado da função deve ficar mais próximo a resposta esperada a partir do argumento base em uso.

Se essas propriedades não forem consideradas correr-se-á o risco de implementar funções circulares, as quais produzem chamadas infinitas a si mesmas consumindo recursos de memória e não produzindo nenhuma resposta.

O termo recursão pode ser entendido como *ação de recorrer, de reajustar linhas, colunas ou páginas* de acordo com o dicionário Houaiss, sendo que para a ciência matemática e computação interessa a parte ação de recorrer, que em sua essência, como verbo transitivo direto tem por significado percorrer novamente. Tem-se, da forma mais simplista possível, como função recursiva a função que percorre ou chama a si mesma.

Uma função recursiva não pode chamar a si mesma indiscriminadamente, pois se assim o fizesse entraria em um processo infinito de chamar a si mesma. Desta forma, é necessário que a função recursiva tenha a definição de uma condição de encerramento que é a solução mais simples e menor para o problema avaliado. O efeito de recursividade é possível nos casos em que se utilizam algoritmos recursivos como as operações de fatorial, somatório, série de Fibonacci, entre outras. A recursão se aplica efetivamente em situações onde certo problema possui como parte uma instância do mesmo problema em um nível menor (EFOFILOFF, 2016).

O estilo recursivo de função mais popular é o cálculo da fatorial de um número inteiro qualquer. Para tanto, considerando o cálculo da função **fatorial(5)** a qual deve retornar como resposta o valor **120**. Como exemplo, considere o resultado do cálculo da fatorial de um número qualquer indicado em seguida.

```
dec fatorial : num -> num;  
--- fatorial 0 <= 1;  
--- fatorial x <= fatorial (x - 1) * x;
```

A função "**fatorial**" possui como primeira ação a checagem da recepção do valor zero passado como argumento, que neste caso, será retornado como valor "**1**" (primeira linha de código da função). Para valores diferentes de zero será executada a segunda linha de código que multiplica o valor do argumento com o resultado da fatorial deste argumento menos um como identificado pelo trecho "**fatorial (x - 1) * x**". Note que é neste momento que a função "**fatorial**" chama a si mesma com o valor do argumento subtraído de um. Esse efeito correrá até que o valor do argumento seja zero, o que desencadeará o retorno dos valores obtidos a cada chamada da função "**fatorial**". Observe a ilustração seguinte.

```
fatorial(5) = 5 * fatorial(5 - 1)  
            = 5 * fatorial(4)  
            = 5 * 4 * fatorial(4 - 1)  
            = 5 * 4 * fatorial(3)  
            = 5 * 4 * 3 * fatorial(3 - 1)  
            = 5 * 4 * 3 * fatorial(2)  
            = 5 * 4 * 3 * 2 * fatorial(2 - 1)  
            = 5 * 4 * 3 * 2 * fatorial(1)  
            = 5 * 4 * 3 * 2 * 1 * fatorial(1 - 1)  
            = 5 * 4 * 3 * 2 * 1 * fatorial(0)  
            = 5 * 4 * 3 * 2 * 1 * 1  
            = 120
```

Após a obtenção do valor "**120**", chega-se à quinta e última etapa do empilhamento dos valores calculados, devido à passagem de parâmetro de valor do conteúdo 5 para a função "**fatorial(5)**". Neste caso, ocorre o encerramento da função e o retorno do valor "**120**" para o trecho do programa que efetuou a chamada da função. A Figura 3.1 demonstra graficamente a lógica de funcionalidade e de ação e mostra como funciona uma função recursiva. Cada instância de chamada da função recursiva "**fatorial**" ocorre de forma a empilhar cada uma das instâncias da função em operação para depois, no retorno, efetuar a multiplicação sucessiva típica do valor retornado com o valor armazenado na pilha, a fim de calcular a fatorial solicitada.

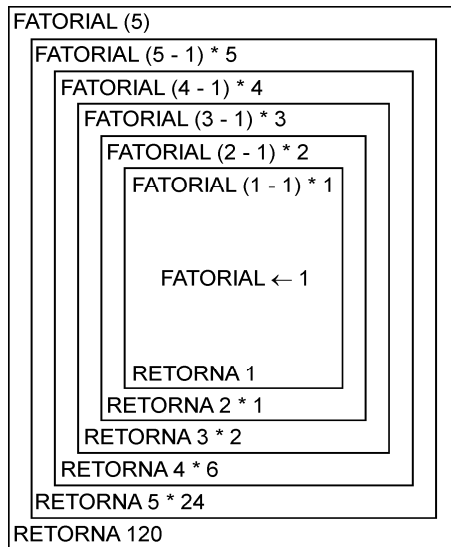


Figura 3.1 - Funcionamento e ação de função recursiva.

O processo de recursividade é considerado muito elegante em programação, pois facilita a abstração e a modularidade no desenvolvimento de funções que podem ser complexas. No entanto, devido ao efeito de empilhamento, essa estratégia pode consumir grande espaço de memória e deve ser pensada sobre outra ótica, como o uso de recursão de cauda.

A recursividade apresentada anteriormente se caracteriza por ser ação de *recursividade direta*. No entanto, existem situações onde esta ação é realizada de forma indireta. A *recursividade indireta* (recursividade de cauda) ocorre quando uma função chama outra função e essa outra função chama a si mesma, ou seja, a função **alfa()** chama função **beta()** que por sua vez chama recursivamente a si mesma criando uma ação de laço entre as funções que deverá possuir em algum ponto uma condição de encerramento da ação recursiva.

A *recursividade em cauda* é uma técnica que garante menor uso de memória por não ser necessário guardar a posição do código onde a chamada da função é realizada na pilha, uma vez que a chamada efetuada pela própria função é a última ação a ser executada. Assim sendo, o resultado final da chamada recursiva em cauda é o resultado final da própria função.

Partindo-se da aplicação do cálculo de fatorial de um valor numérico inteiro qualquer se tem como solução recursiva simples o código.

```

dec fatorbase : num # num -> num;
--- fatorbase (n, parcial) <= if n == 1 then parcial
                                else fatorbase (n - 1, parcial * n);

dec fatorial2 : num -> num;
--- fatorial2 n <= fatorbase (n, 1);

```

Ao ser executada a função "**fatorial2**" o resultado apresentado será o mesmo da execução da função "**fatorial**". No entanto, a forma operacional de uso das funções é completamente diferente uma da outra. Observe a ilustração da figura 3.2.

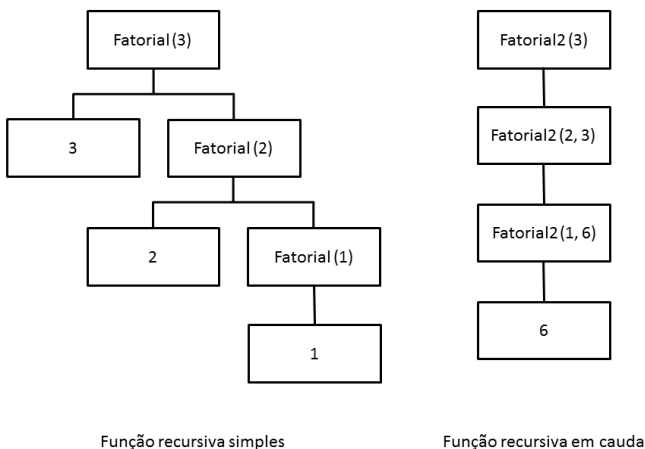


Figura 3.2 - Função simples e cauda - comportamento em memória.

A partir da figura 3.2 é possível ter ideia de como se comporta os tipos de recursividade simples e em cauda. O tempo de execução de ambas as funções é idêntico uma vez que a função recursiva chama a si mesma apenas uma vez por instância.

A função "**fatorial2**" é apenas um recurso de execução e chamada da função recursiva de cauda "**fatorbase**". Note que a função "**fatorbase**" quando chama a si mesma passa dois argumentos, sendo o primeiro argumento o valor da fatorial decrescido de um a cada chamada e o segundo argumento a multiplicação do valor do argumento pelo valor do acumulador parcial. A função **fatorial2** recebe o argumento a ser calculado e faz a chamada da função "**fatorbase**" passando-lhe o argumento de cálculo e o valor limite mínimo para a recursão parar.

Nos casos de execução de funções para obter os cálculos de fatorial ou de somatório o uso da recursão de cauda em relação à recursão simples pode não ser evidente em um primeiro momento. O que não se pode dizer em relação ao cálculo de termos da sequência de Fibonacci.

Considere o cálculo de certo termo da sequência de Fibonacci a partir da função **"fib1"** com recursão simples.

```
dec fib1 : num -> num;
--- fib1 0 <= 1;
--- fib1 1 <= 1;
--- fib1 n <= fib1 (n - 1) + fib1 (n - 2);
```

Ao ser executada a função **"fib1"** obter-se-á o resultado do termo de Fibonacci indicado. Por exemplo, se executada a função **"fib1(5)"** o resultado apresentado será **"8"**. Para ver o trigésimo quinto termo usa-se **"fib1(35)"** e obter-se-á o resultado **"14930352"** que levará aproximadamente **"14"** segundos para ser calculado. Valores maiores que **"35"** ocasionarão um tempo de cálculo muito maior devido ao consumo de memória, como efeito colateral da ação de recursão.

A recursão de cauda é mais eficiente e rápida. Considere o cálculo de certo termo da sequência de Fibonacci a partir da função **"fib2"** com recursão de cauda.

```
dec fibbase : num # num # num -> num;
--- fibbase (0, ant, atual) <= anterior;
--- fibbase (1, ant, atual) <= atual;
--- fibbase (2, ant, atual) <= atual + anterior;
--- fibbase (n, ant, atual) <= fibbase (n - 1, atual, ant + atual);

dec fib2 : num -> num;
--- fib2 n <= fibbase (n, 0, 1);
```

Ao ser executada a função **"fib2"** obter-se-á o resultado quase que instantaneamente. Se executado **"fib2(35)"** o resultado **"14930352"** é imediato, mostrando que a recursão de cauda é mais eficiente que a recursão simples, consumindo muito menos recurso de memória.

Observe que a função **"fib2"** faz uso do auxílio da função **"fib1"** usada para realizar a recursão de cauda. Note que é possível usar o símbolo aspas simples como elemento de definição do nome de uma função na linguagem **HOPE**.

Outro detalhe a ser observado no código da função `"fib2"` é o uso do comando `"where"` que tem por finalidade ser utilizado na simplificação de expressões matemáticas e lógicas. Mais detalhes sobre o comando `"where"` são apresentados no capítulo 4.

3.6 FUNÇÕES INTERNAS

Como descrito a linguagem *HOPE*, disponibilizada por meio dos interpretadores *MA-HOPE* e *RP-HOPE*, possui uma pequena coleção de funções para auxílio de diversas operações matemáticas e para conversão e manipulação básica de dados.

A seguir são descritas e exemplificadas cada uma das funções existentes nos interpretadores, excetuando-se as funções não existentes no interpretador *MA-HOPE* que serão definidas no final deste capítulo.

É importante ressaltar que o conjunto de funções ofertado nos interpretadores não é completo, faltando inclusive algumas funcionalidades básicas, as quais serão produzidas em momento oportuno de modo que ambos os interpretadores tenham os recursos mínimos necessários para maior comodidade e auxílio no uso da linguagem *HOPE*.

Um detalhe operacional importante a ser considerada é que o uso de parênteses após o nome da função para definição de seus argumentos é opcional quando a função opera com um só argumento. Para funções com mais de um argumento o uso de parênteses é obrigatório.

FUNÇÕES PARA MANIPULAÇÃO E CONVERSÃO DE DADOS

As funções de conversão e manipulação de dados servem para tratar dados numéricos e alfanuméricos da linguagem.

Para o tratamento de dados são disponibilizadas funções que identificam o valor do código ASCII (*American Standard Code for Information Interchange*) de um caractere indicado entre aspas simples e ação inversa com o fornecimento do valor ASCII e a indicação do respectivo caractere. Veja alguns exemplos do uso das funções `"ord"` (mostra código numérico do caractere) e `"chr"` (mostra caractere do código numérico).

```
>: ord('A');
```

```
>> 65 : num
```

```
>: ord 'a';
```

```
>> 97 : num
```

```
>: chr(65);
```

```
>> 'A' : char
```

```
>: chr 97;
```

```
>> 'a' : char
```

Para a conversão de dados são disponibilizadas funções que convertem uma sequência alfanumérica em numérica e vice-versa. Veja alguns exemplos do uso das funções **"num2str"** (converte valor numérico em sequência de caracteres alfanumérico) e **"str2num"** (converte sequência de caracteres alfanuméricos em valor numérico).

```
>: num2str(123);
```

```
>> "123" : list char
```

```
>: num2str 456;
```

```
>> "456" : list char
```

```
>: str2num("789");
```

```
>> 789 : num
```

```
>: str2num "135";
```

```
>> 135 : num
```

Se a sequência alfanumérica a ser convertida como numérica possuir caracteres não numéricos estes são desconsiderados, mantendo-se apenas os símbolos numéricos.

FUNÇÕES PARA OPERAÇÕES MATEMÁTICAS

As funções matemáticas disponibilizadas pelos interpretadores *MA-HOPE* e *RP-HOPE* não são documentalmente classificadas por estilos de ação. Neste sentido, para melhor legibilidade essas funções são apresentadas a seguir sub classificadas de acordo com suas categorias de ação: auxiliares, arredondamento, potenciação, logaritmos, trigonométricas e hiperbólicas.

Entre o conjunto de funções matemáticas destacam-se duas funções auxiliares usadas para transformar valores numéricos negativos em positivos e geração de valor sucessor. Veja alguns exemplos do uso das funções "**abs**" (conversão de valor negativo em valor positivo), "**succ**" (para a criação de valor sucessor do valor definido) e "**hypot**" (para calcular o tamanho da hipotenusa de um ângulo reto do triângulo).

```
>: abs(5);
```

```
>> 5 : num
```

```
>: abs 5;
```

```
>> 5 : num
```

```
>: hypot(6, 6);
```

```
>> 8.48528137423857 : num
```

```
>: hypot(3, 4);
```

```
>> 5 : num
```

```
>: succ(1);
```

```
>> 2 : num
```

```
>: succ 2;
```

```
>> 3 : num
```

É importante levar em consideração que os interpretadores *MA-HOPE* e *RP-HOPE* não permitem a definição de valores negativos diretos como, por exemplo, "-5". Neste caso, sugere definir valores com o artifício de cálculo "**0 - 5**". Veja os exemplos a seguir.

```
>: -5;  
>> syntax error
```

```
>: 0 - 5;  
>> -5 : num
```

```
>: abs(0 - 5);  
>> 5 : num
```

Muitas vezes é necessário operar valores com arredondamento numérico, onde o valor numérico deve ser arredondado para o inteiro mais acima ou mais abaixo. Veja alguns exemplos do uso das funções "**ceil**" (arredondamento para inteiro acima) e "**floor**" (arredondamento para inteiro abaixo).

```
>: ceil(1.1);  
>> 2 : num
```

```
>: ceil(1.9);  
>> 2 : num
```

```
>: floor(1.9);  
>> 1 : num
```

```
>: floor(1.1);  
>> 1 : num
```

Para auxiliar operações com potenciação são disponibilizadas três funções, sendo "**exp**" (para apresentar o exponencial de um valor numérico), "**pow**" (para apresentar o resultado de uma potência de um valor numérico) e "**sqrt**" (para calcular a raiz quadrada de um valor numérico). Veja alguns exemplos do uso dessas funções.

```
>: exp(1);  
>> 2.71828182845905 : num
```

```
>: exp(2);  
>> 7.38905609893065 : num
```

```
>: pow(5, 2);  
>> 25 : num
```

```
>: sqrt(5);  
>> 2.23606797749979 : num
```

Para operações logarítmicas os interpretadores *MA-HOPE* e *RP-HOPE* disponibilizam as funções de logaritmo natural e logaritmo na base 10, sendo "**log**" (para calcular logaritmo natural) e "**log10**" (para calcular o logaritmo na base 10 de um valor numérico fornecido). Veja alguns exemplos do uso dessas funções.

```
>: log(0.5);  
>> -0.693147180559945 : num
```

```
>: log 0.5;  
>> -0.693147180559945 : num
```

```
>: log10(0.5);  
>> -0.301029995663981 : num
```

```
>: log10 0.5;  
>> -0.301029995663981 : num
```

Em relação ao uso de operações trigonométricas os interpretadores *MA-HOPE* e *RP-HOPE* disponibilizam algumas funções para algumas operações, destacando-se "**cos**" (para o cálculo do cosseno de um ângulo), "**sin**" (para o cálculo do seno de um ângulo), "**acos**" (para calcular o arco cosseno de um ângulo), "**asin**" (para calcular o arco seno de um ângulo), "**atan**" (para calcular o arco tangente de um ângulo) e "**atan2**" (para calcular o arco tangente do quociente de duas coordenadas). Veja alguns exemplos do uso dessas funções.

```
>: cos(1);  
>> 0.54030230586814 : num
```



```
>: sin(1);  
>> 0.841470984807897 : num
```

```
>: acos(0.5);  
>> 1.0471975511966 : num
```

```
>: asin(0.5);  
>> 0.523598775598299 : num
```

```
>: atan(0.5);  
>> 0.463647609000806 : num
```

```
>: atan2(1, 0.5);  
>> 1.10714871779409 : num
```

Como funções hiperbólicas (funções inversas) os interpretadores *MA-HOPE* e *RP-HOPE* disponibilizam "**cosh**" (para calcular o cosseno hiperbólico), "**sinh**" (para calcular o seno hiperbólico) e "**tanh**" (para calcular a tangente hiperbólica). Veja alguns exemplos do uso dessas funções.

```
>: cosh(1);  
>> 1.54308063481524 : num
```

```
>: sinh(1);  
>> 1.1752011936438 : num
```

```
>: tanh(1);  
>> 0.761594155955765 : num
```

O conjunto de funções internas existentes nos interpretadores *MA-HOPE* e *RP-HOPE* possuem como finalidade auxiliar e facilitar a execução de operações de cálculos que venham a fazer uso desses recursos.

3.7 FUNÇÕES DEFINIDAS PELO PROGRAMADOR

No capítulo anterior foi apontado que o interpretador *MA-HOPE* não possui algumas das funções encontradas no interpretador *RP-HOPE*, sendo: "erf"; "erfc"; "acosh"; "asinh" e "atanh". Desta forma, para resolver este detalhe pode-se criar um arquivo de módulo separado e solicitar que este seja carregado quando de sua necessidade de uso por meio do comando "uses", ou definir tais recursos dentro do arquivo de módulo "Standard.hop" que é carregado automaticamente quando da execução do programa "hope.exe".

Levando em consideração que essas funções devem automaticamente ficar disponíveis como as demais funções apresentadas abra em um editor de texto o arquivo "Standard.hop" do interpretador *MA-HOPE*.

Localize no arquivo o código da função "succ", antes do comando "private" e acrescente após o código da função "succ" e a indicação do comando "private" a sequência de código a seguir, exatamente como definida.

```
dec sign : num -> num;
--- sign 0 <= 0;
--- sign x <= if x > 0 then 1 else 0 - 1;

--- erf 0 <= 0;
--- erf x <= (1 - (((((
    1.061405429 * (1 / (1 + 0.3275911 * abs (x))) +
    0 - 1.453152027) *
    (1 / (1 + 0.3275911 * abs (x)))) + 1.421413741) *
    (1 / (1 + 0.3275911 * abs (x))) + 0 - 0.284496736) *
    (1 / (1 + 0.3275911 * abs (x))) + 0.254829592) *
    (1 / (1 + 0.3275911 * abs (x))) * exp(0 - abs(x) *
    abs (x))) * sign (x);

--- erfc x <= 1 - erf(x);

--- acosh x <= log(x + sqrt (pow (x, 2) - 1));
```

```

--- asinh x <= log(x + sqrt(pow(x, 2) + 1));

--- atanh x <= 1 / 2 * log((1 + x) / (1 - x));

```

Grave o conteúdo inserido no arquivo e a partir deste instante o interpretador **MA-HOPE** está equacionado com o interpretador **RP-HOPE**. No entanto, será necessário fechar o interpretador e reabri-lo para as mudanças possam ser usadas.

Um detalhe a serem considerado em relação ao uso das funções "erf" e "erfc" dos interpretadores são que essas funções retornam seus valores com diferenças a partir da sexta casa decimal.

Isto ocorre pelo fato de estarem sendo usadas constantes numéricas para o cálculo das funções "erf" e "erfc" baseados na fórmula "7.1.26" indicada na obra "**Hand-book of Mathematical Functions: with Formulas, Graphs, and Mathematical Tables**" de Milton Abramowitz e Irene A. Stegun, citada nas referências bibliográficas.

Observe junto aos interpretadores **MA-HOPE** e **RP-HOPE** respectivamente exemplos das funções "erf" e "erfc". Veja alguns exemplos do uso dessas funções.

<pre> >: erf(1) ! MA-HOPE >> 0.84270068974759 : num >: erfc(1) ! MA-HOPE >> 0.15729931025241 : num </pre>	<pre> >: erf(1) ! RP-HOPE >> 0.842700792949715 : num >: erfc(1) ! RP-HOPE >> 0.157299207050285 : num </pre>
--	--

Para a devida ação das funções "erf" e "erfc" junto ao interpretador **MA-HOPE** foi definida a função "sign" (não existente no interpretador **RP-HOPE**) que retorna zero se o valor avaliado for zero, retorna "1" se o valor avaliado é positivo ou retorna "-1" se o valor avaliado for negativo. Caso deseje esta funcionalidade disponível no interpretador **RP-HOPE** esta deverá ser inserida no arquivo de módulo "**Standard.hop**", o que será realizado mais adiante neste tópico.

As funcionalidades estabelecidas para os cálculos dos inversos hiperbólicos do seno (**asinh**), cosseno (**acosh**) e tangente (**atanh**) do interpretador **MA-HOPE** retornará os mesmos valores calculados no interpretador **RP-HOPE**. Veja alguns exemplos do uso dessas funções.

```

>: acosh(2);
>> 1.31695789692482 : num

>: asinh(2);
>> 1.44363547517881 : num

>: atanh(0.5);
>> 0.549306144334055 : num

```

Após os ajustes para que ambos os interpretadores tenham comportamento semelhante será preparado o trecho de código que fornecerá outras funções básicas importantes não existentes, destacando-se:

- **deg2rad(x)** conversão de graus em radianos;
- **rad2deg(x)** conversão de radianos em graus;
- **log2(x)** logaritmo de "x" na base 2;
- **logb(x, b)** logaritmo de "x" na base "b" (qualquer base);
- **minus(x)** valor negativo de "x";
- **nthrt(b, i)** raiz enésima de "b" com índice "i";
- **pred(x)** valor predecessor de "x";
- **round(x)** arredondamento de "x" para baixo ou para cima;
- **tan(x)** tangente de "x".

Observe inicialmente o trecho de código seguinte que deverá ser inserido no arquivo de módulo "**Standard.hop**" dos interpretadores. Após o trecho indicado é fornecida as instruções de como proceder a inserção deste trecho de código em cada um dos interpretadores.

```

dec deg2rad, rad2deg, log2, minus, pred, round : num -> num;

--- deg2rad x <= x * (acos (0) * 2) / 180;

--- rad2deg x <= x * 180 / (acos (0) * 2);

```

```

--- log2 x <= log (x) / log (2);

--- minus x <= 0 - x;

--- pred x <= x - 1;

--- round x <= floor (x + 0.5);

--- tan x <= sin (x) / cos (x);

dec logb, nthrt: num # num -> num;

--- logb (x, b) <= log (x) / log (b);

--- nthrt (x, i) <= pow (x, 1 / i);

```

No caso do interpretador *MA-HOPE* basta seguir como orientado anteriormente. Coloque o trecho de código antes do comando "**private**" e grave o conteúdo.

Já para o interpretador *RP-HOPE* é necessário como usuário "**root**" entrar no diretório "**/usr/local/share/hope/lib**" e editar o arquivo "**Standard.hop**" inserindo o trecho anterior antes do comando "**private**". Aproveite e entre o trecho de código para a função "**sign**" e não se esqueça de gravar as alterações.

Agora é possível obter mais alguns resultados a partir das novas funções definidas tendo-se em mãos um pacote mínimo essencial para a efetivação de alguns cálculos matemáticos. Veja alguns exemplos do uso dessas funções.

```

>: deg2rad(45); ! 45 graus em radianos
>> 0.785398163397448 : num

>: rad2deg(0.785398163397448); ! 0.785398163397448 em graus
>> 45 : num

>: log2(5); ! logaritmo 5 de base 2
>> 2.32192809488736 : num

```

```
>: logb(2, 7); ! logaritmo 2 de base 7  
>> 0.356207187108022 : num
```

```
>: minus 7; ! numero negativo  
>> -7 : num
```

```
>: sign(0);  
>> 0 : num
```

```
>: sign(9);  
>> 1 : num
```

```
>: sign(minus 8); ! minus 8 = -8  
>> -1 : num
```

```
>: nthrt(2, 3); ! raiz cubica de 3  
>> -1 : num
```

```
>: pred(2);  
>> 1 : num
```

```
>: round(1.1); ! arredonda para baixo  
>> 1 : num
```

```
>: round(1.9); ! arredonda para cima  
>> 2 : num
```

```
>: tan(1);  
>> 1.5574077246549 : num
```

Outra providência a ser tomada é a inserção das constantes criadas e armazenadas no arquivo "**constant.hop**" após o código da função "**succ**" definida no arquivo "**Standard.hop**". Após este procedimento o arquivo de módulo contendo as constantes pode ser apagado.

Agora, os interpretadores *MA-HOPE* e *RP-HOPE* estão ajustados e compatibilizados permitindo que o trabalho nos próximos detalhes a serem apresentados seja homogêneo e não haja diferenças a serem consideradas.

3.8 TIPOS DE DADOS DECLARADOS

Anteriormente foi apresentado o conjunto de tipos de dados primitivos que são suportados pela linguagem *HOPE* disponíveis nos interpretadores *MA-HOPE* e *RP-HOPE*, destacando-se "**num**", "**char**", "**truval/bool**", "**tuple**" e "**list**".

Os tipos de dados primitivos atendem diversas necessidades, mas por vezes não são suficientes para atender ao que se deseja fazer. Nesses casos é possível criar um tipo de dado próprio por intermédio do comando "**data**" que possui a estrutura sintática seguinte.

```
data <rótulo> == <valor1> ++ <valor2> ++ <valorN>;
```

Onde "**rótulo**" é o nome do tipo e "**valor1**", "**valor2**" e "**valorN**" são os valores padrão do tipo criado. O comando "**data**" é usado no início da declaração de um programa ou de uma função. O símbolo "**==**" (igualdade dupla significa que "é definido como") que representa o construtor do dado definido e o símbolo "**++**" (adição dupla significa o conceito de "ou").

Por exemplo, considerando um tipo de dado chamado "**resposta**" com a possibilidade de retornar como valor de sua ação "**sim**", "**nao**" ou "**talvez**" pode ser definido como:

```
data resposta == sim ++ nao ++ talvez;
```

Para verificar a existência do tipo de dado definido "**resposta**" basta executar as instruções indicadas em seguida e observar a apresentação do nome do tipo de dado ao lado de cada um dos valores previstos.

```
>: resposta sim;
>> sim : resposta

>: resposta nao;
>> nao : resposta
```

```
>: resposta talvez;
>> talvez : resposta
```

A partir da definição de um tipo de dado é possível definir uma função que venha a utilizá-lo. Por exemplo, uma função que receba uma das respostas previstas e procede ações relacionadas a cada uma das respostas.

```
dec pergunta : resposta -> list char;
--- pergunta (sim) <= "entao voce sabe tudo";
--- pergunta (talvez) <= "entao voce esta com duvida";
--- pergunta (nao) <= "inocente, nao sabe de nada";
```

Na sequência veja as instruções que demonstram o uso da função "pergunta" e as mensagens de retorno dependendo do valor usado do tipo "resposta".

```
>: pergunta sim;
>> "entao voce sabe tudo" : list char

>: pergunta talvez;
>> "entao voce esta com duvida" : list char

>: pergunta nao;
>> "inocente, nao sabe de nada" : list char
```

O comando "**data**" permite criar tipos de dados que possuam constantes internas como respostas para representar sua ação.

Dentro do contexto de definição de tipos de dados abstratos há a possibilidade de fazer uso do comando "**type**" que permite criar sinônimos para os tipos de dados existentes. Esta estratégia possibilita para certos projetos definir nomes personalizados de tipos de dados que sejam mais condizentes com os dados sendo operacionalizados.

Considere definir como sinônimo para o tipo de dado lógico da linguagem "**truval**" (**MA-HOPE**) ou "**bool**" (**RP-HOPE**) o rótulo de identificação "**logico**". Para tanto, considere o uso do comando "**type**" de acordo com a sintaxe.

```
type <tipo sinônimo> == <tipo primitivo>;
```


Onde "**tipo sinônimo**" é o tipo definido a partir do "**tipo primitivo**" da linguagem. Assim sendo, execute uma das instruções.

```
type logico == truval; ! Interpretador MA-HOPE
type logico == bool;  ! Interpretador RP-HOPE
```

Na sequência basta criar uma função que faça uso do tipo de dado definido como sinônimo. Desta forma, considere uma função chamada "**decide**", a qual indica se dado valor é verdadeiro quando o valor igual a "**1**", sendo diferente de "**1**" o programa deve retornar o valor como falso. Observe as instruções a seguir.

```
dec decide : num -> logico;
--- decide n <= if n = 1 then n = 1 else n /= 1;
```

Em seguida execute as instruções.

```
>: decide 1;
>> true : logico

>: decide 2;
>> true : logico
```

Observe a indicação do termo "**logico**" como tipo de dado dos valores retornados como "**true**" ou "**false**".

HOPE possui para a definição de tipos de dados os comandos "**abstype**" ou "**mu**", onde "**abstype**" é usado para definir tipos de dados abstratos associados e o "**mu**" é usado para simplificar a definição de tipos de dados.

O comando "**abstype**" cria um rótulo identificador de tipo que pode ser usado como sinônimo de um tipo de dado válido da linguagem disponível para posteriores definições. Dados abstratos devem ser usados na declaração de outros dados com os comandos "**data**" e "**type**" para ser usado com o comando "**dec**" a partir da definição sintática.

```
abstype <tipo abstrato> <tipo concreto>;
```

Onde "**tipo abstrato**" é o tipo a ser definido e "**tipo concreto**" é o uso de um tipo de dado suportado pela linguagem. Como exemplo considere a definição do tipo de

dado "**set**" a partir do tipo concreto "**pos**" que permite usar o tipo abstrato como um tipo padrão da linguagem.

```
abstype set pos;
```

Após definir o tipo de dado abstrato é necessário associar este tipo a um tipo padrão da linguagem, o que é feito com a instrução.

```
type set num == list num;
```

O tipo de dado "**set**" declarado a partir de "**abstype set pos**" deve ser usado em conjunto do tipo de dados "**list**". Em seguida execute as instruções.

```
dec a : set num;
--- a <= [1, 2, 3, 4, 5];

>: a;
>> [1, 2, 3, 4, 5] : set num
```

Após a execução dos comandos anteriores observe a apresentação da lista de valores indicada com o sendo do tipo "**set num**".

Em relação ao comando "**mu**" este é usado na declaração de ações de simplificação na definição de tipos quando em uso tipos de dados recursivos como os exemplos seguintes.

```
type set num == num # set num;
type meu_tipo alpha beta == alpha # meu_tipo beta alpha;
```

Os tipos de dados apresentados podem com o uso do comando "**mu**" ser simplificados com as instruções.

```
type set num == mu x => num # x;
type meu_tipo alpha beta == mu x => alpha # (beta # x);
```

Por exemplo, a instrução "**type set num == mu x => num # x**" informa exatamente a mesma coisa que a instrução "**type set num == num # set num**". Neste contexto o comando "**mu**" diz que a variável "**x**" é usada como sinônimo do tipo de dado

declarado a sua esquerda colocando, neste caso, o tipo de dado "**set num**" a direita do símbolo "**=>**", onde "**x**" é definido ("**num # x**").

3.9 FUNÇÃO POLIMÓRFICA

A linguagem *HOPE* exige que cada função em um programa seja declarada com o comando "**dec**" e definida com o comando **---**. A declaração de uma função é realizada com o comando "**dec**", possuindo além do nome da função a relação de argumentos de entrada e saída.

Embora essa abordagem evite erros de tempo de execução, pois qualquer incoerência na definição ou uso da função é prontamente apontada exigindo sua pronta correção causa grande desvantagem na tipificação dos dados usados na declaração de uma função, pois uma função definida para uso com um tipo de dado não pode ser usada para executar ação semelhante com outro tipo de dado. Por exemplo, considerando uma função chamada "**tamanho**" que apresente a quantidade de elementos de uma lista como indicada em seguida.

```
dec tamanho : list num -> num;
--- tamanho nil <= 0;
--- tamanho (x :: xs) <= 1 + tamanho xs;
```

Observe que a função "**tamanho**" está operando com valores do tipo "**num**" em uma lista como argumento de entrada devolvendo um valor numérico do tipo "**num**" com o resposta da quantidade de elementos exustentes. Assim, execute a instrução seguinte.

```
tamanho [1, 2, 3, 4, 5];
```

Veja que é apresentada a resposta;

```
5 : num
```

Indicando que a lista fornecida possui **5** elementos. No entanto se, ocorrer o uso de uma lista de caracteres alfabéticos ocorrerá a apresentação de um erro como indicado a seguir.

```
>: tamanho ['a', 'b', 'c', 'd', 'e'];
tamanho ("abcde")
tamanho : list num -> num
"abcde" : list char
type error - argument has wrong type
```

A mensagem de erro é composta de três linhas, sendo a primeira a indicação do dado fornecido na forma "**tamanho ("abcde")**", a segunda linha indica o formato da função e a terceira linha indica o dado fornecido e seu tipo. Observe que a terceira linha mostra o fornecimento de um dado do tipo "**list char**", o qual deveria se um dado do tipo "**list num**", como indicado na segunda linha.

Para que a ação possa ser usada com qualquer tipo de dado deve-se estabelecer uma função do tipo polimórfica que opere com qualquer tipo de dado. Para que isso seja possível é necessário fazer uso do comando "**typevar**" com a indicação do nome de um tipo de dado polimórfico.

```
typevar teste;
```

O tipo de dato "**teste**" passa a ser um tipo de dado variável que pode ser usado na forma de curinga substituindo qualquer tipo de dado primitivo da linguagem dentro da operação de uma função polimórfica. Desta forma, considere as próximas instruções.

```
dec tamanho2 : list teste -> num;
--- tamanho2 nil <= 0;
--- tamanho2 (x :: y) <= 1 + tamanho2 y;
```

Em seguida execute as instruções indicadas e observe os resultados apresentados.

```
>: tamanho2 ['a', 'b', 'c', 'd', 'e'];
>> 5 : num

>: tamanho2 [1, 2, 3, 4, 5];
>> 5 : num
```

Veja que a função "**tamanho2**" diferentemente da função "**tamanho**" consegue operar com estruturas de valores de tipos de dado primitivos diferentes.

É interessante pontuar que no arquivo de módulo "**Standard.hop**" encontra-se a definição de tipos de dados curingas chamados "**alpha**", "**beta**" e "**gamma**" para atender necessidade polimórficas.

Página propositalmente em branco

4

Programação funcional avançada

Este capítulo apresenta informações mais avançadas em relação ao capítulo anterior, destacando-se o uso de funções pré-fixadas e pós-fixadas; operações iniciais com listas e tuplas como estruturas de dados; funções de ordem superior; funções anônimas (expressões lambda); simplificação de expressões e definição avançada de módulos.

4.1 FUNÇÕES PRÉ-FIXADA E PÓS-FIXADA

Linguagens funcionais, como **HOPE**, normalmente consideram qualquer elemento operacional da linguagem como sendo função. Desta forma, funções pré-fixadas e pós-fixadas são recursos que permitem estabelecer a construção de operadores aritméticos, lógicos ou auxiliares.

Funções pré-fixadas são definidas com o comando **"infix"** e as funções pós-fixadas são definidas com o comando **"infixr"**. Esses comandos definem o símbolo (que pode ser um rótulo ou um ou mais caracteres especiais de pontuação) a ser usado e a precedência do símbolo na operação a ser realizada. Os valores de precedência permitidos são de **"1"** a **"9"** do menor para a maior precedência.

A diferença entre os comandos **"infix"** e **"infixr"** é a ordem em que o operador definido é processado.

Para exemplificar, considere a definição de um operador aritmético de multiplicação chamado **"mult"** (como função infixa à esquerda) para realizar a operação entre dois valores numéricos definidos. Observe as instruções seguintes.

```
infix mult : 8;
dec mult : num # num -> num;
--- a mult b <= if b = 0 then 0 else a mult (b - 1) + a;
```

O comando "**infix**" permite que se use a função "**mult**" como operador infixo binário com a definição de prioridade "**8**". A prioridade é usada para determinar qual operação deve ser executada primeiro. Observe que a instrução seguinte apresenta como resultado o valor "**7**".

```
1 + 2 mult 3;
```

Como não há nos interpretadores **HOPE** em uso o recurso para uso de operador lógico "**xor**", observe os detalhes dessa definição.

```
infix xor : 1;
dec xor : truval # truval -> truval;
--- true xor true <= false;
--- true xor false <= true;
--- false xor true <= true;
--- false xor false <= false;
```

O operador lógico "**xor**" caracteriza-se por realizar comparações lógicas de disjunção exclusiva conforme a tabela verdade seguinte.

Tabela verdade do operador lógico "xor"		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Falso
Verdadeiro	Falso	Verdadeiro
Falso	Verdadeiro	Verdadeiro
Falso	Falso	Falso

Observe os resultados apresentados a partir de algumas ações condicionais.

```
>: (1 = 1) xor (1 = 1);
>> false : truval
```



```
>: (1 = 1) xor (1 = 0);  
>> true : truval
```

```
>: (1 = 0) xor (1 = 1);  
>> true : truval
```

```
>: (1 = 0) xor (1 = 0);  
>> false : truval
```

Como exemplo de uso do comando "**infixr**" considere a definição de um operador aritmético representado por "******" com a finalidade de calcular a potência de uma base elevada a um índice. Para tanto observe as instruções.

```
infixr ** : 7;  
dec ** : num # num -> num;  
--- (**) <= pow;
```

A partir dessa definição basta executar a instrução.

```
2 ** 3;
```

Para ser apresentado o valor "**8**" como resultado da operação de potenciação.

4.2 OPERAÇÕES BÁSICAS COM ESTRUTURAS

Linguagens funcionais operam coleções de dados, matematicamente chamadas de conjuntos representadas na linguagem **HOPE** como estruturas formadas por listas ou tuplas. Uma lista é formada por um conjunto de valores de mesmo tipo de dado delimitados entre colchetes e uma tupla é um conjunto de dados de tipos diferentes delimitados entre parênteses. Desta forma, listas são estruturas homogêneas com dados de mesmo tipo e tuplas são estruturas heterogêneas podendo ser composta com dados de tipos diferentes.

As estruturas de dados, sejam listas ou tuplas, são formatadas em linguagens de programação funcionais com cabeça (**head**) e cauda (**tail**). A cabeça se refere ao

primeiro elemento, à esquerda, da estrutura e a cauda aos demais elementos componentes a partir do primeiro elemento existente.

O uso de listas é mais amplo que o uso de tuplas na linguagem *HOPE*, pois apesar das listas serem mais restritas em relação à exigência de seus elementos serem do mesmo tipo de dados, são elas que permitem abstrair na linguagem o uso da teoria de conjuntos estuda na ciência matemática.

Tuplas são adequadas para representar estruturas de dados que quando definidas não necessitam ter seus elementos processados fora do escopo da função. O estilo de definição de tuplas segue o formato.

```
<tipo> # <tipo> [# <tipo> # ... # <tipo>]
```

Onde "**tipo**" pode ser a indicação dos tipos de dados "**char**" ou "**num**", considerando-se eventualmente o uso de algum tipo de dado definido pelo próprio programador.

Há na linguagem *HOPE* uso extensivo da estrutura de dado tupla, de forma até implícita, quando da definição de um conjunto de argumentos como valores de entrada para certas funções quando se torna necessário na definição da função determinar os argumentos dentro de parênteses. Observe a declaração e definição da estrutura operacional da função "**mult**" exemplificada no capítulo anterior.

```
dec mult : num # num -> num;  
--- mult (0, _) <= _;  
--- mult (_, 0) <= _;  
--- mult (a, b) <= a * b;
```

Note que na função "**mult**" a declaração "**dec mult : num # num -> num**" faz uso de uma tupla de dados do tipo "**num**" na declaração dos argumentos de entrada como "**num # num**". Essa declaração exige que na definição dos padrões de correspondência sejam estabelecidos os argumento entre parênteses "**(0, _)**", "**(_, 0)**" e "**(a, b)**".

Uma função pode usar tuplas para receber argumentos e retornar seus resultados na forma de tupla. Como exemplo, considere uma função simples que apresenta como resultado na forma de tupla o quadrado de dois valores fornecidos.

```
dec quadrado_dois : num # num -> num # num;  
--- quadrado_dois (a, b) <= (pow (a, 2), pow (b, 2));
```

Ao ser executada a função "**quadrado_dois**" com o uso dos argumentos **5** e **9** ocorrerá a resposta dos valores retornados dentro de uma tupla.

```
>: quadrado_dois(5, 9);  
>> (25, 81) : num # num
```

A partir dessas informações considere uma função que receba uma tupla com valores da hora, minuto e segundo e retorne o valor do tempo em segundos.

```
dec segundos : num # num # num -> num;  
--- segundos (h, m, s) <= h * 3600 + m * 60 + s;
```

Ao ser executada a função "**segundos**" e fornecidos os valores das horas, minutos e segundos ocorrerá a apresentação do valor do tempo em segundos. Observe as instruções seguintes para a definição dos segundos do horário "**13h16min55**".

```
>: segundos(13, 16, 55);  
>> 47815 : num
```

A operação inversa apresentará o tempo indicado em segundos separado os elementos de tempo em horas, minutos e segundos em uma tupla.

```
dec horario : num -> num # num # num;  
--- horario t <= (t div 3600, t mod 3600 div 60, t mod 3600 mod 60);
```

Ao ser executada a função "**horario**" e fornecido o valor em segundos ocorrerá a apresentação do valor do tempo em horas minutos e segundos na forma de tupla. Observe as instruções seguintes para a definição do horário a partir do tempo em segundos "**47815**".

```
>: horario(47815);  
>> (13, 16, 55) : num # num # num
```

Listas, diferentemente das tuplas podem ter seus elementos processados para gerar outras estruturas de listas fora do escopo da função que as manipulam, simu-

lando, por exemplo, o uso de conjunto domínio, contradomínio e imagem. O estilo de definição de listas segue o formato.

```
list <tipo>
```

Onde "**tipo**" pode ser a indicação dos tipos de dados "**char**" ou "**num**", considerando-se eventualmente o uso de algum tipo de dado definido pelo próprio programador.

A definição de uma tupla na linguagem **HOPE** ocorre a partir da declaração de uma função com a forma:

```
dec <rótulo> : <tipo> -> <tipo> # <tipo> [# <tipo> # ... # <tipo>];
```

A definição de uma lista na linguagem **HOPE** ocorre a partir da declaração de uma função com a forma:

```
dec <rótulo> : <tipo> -> list <tipo>;
```

Para o tratamento de listas a linguagem **HOPE** possui duas operações padrão, sendo:

- A primeira operação padrão ocorre com o uso do operador "**::**" (operador **cons**, representado por dois pontos duplos que efetua a entrada de um valor na cabeça de uma lista) que tem por finalidade produzir uma nova lista inserindo um elemento a uma lista existente de mesmo tipo de dado. Se executada a instrução "**10 :: [20, 30, 40]**" obter-se-á como resultado "**[10, 20, 30, 40] : list num**". É importante observar que a operação não adiciona "**10**" à lista "**[20, 30, 40]**", mas produz uma nova lista cujo primeiro item (cabeça da lista) tem o mesmo valor que o primeiro operando definido antes do operador "**::**", cujo a cauda da lista possui o mesmo valor definido após o operador "**cons**".
- A segunda operação padrão ocorre com o uso da função "**nil**" que permite criar listas vazias ou determinar o último elemento de uma lista não vazia. Toda lista possui seu último elemento definido como "**nil**". Desta forma a lista "**[1, 2, 3]**" é internamente definida como "**1 :: (2 :: (3 :: nil))**".

HOPE suporta o uso de listas aninhadas. Por exemplo, a lista `["abc", "def"]` caracteriza-se por ser uma lista formada por listas, ou seja, será apresentado como resposta desta ação o resultado `["abc", "def"] : list (list char)`, onde a indicação `list (list char)` mostra uma lista forma por outra lista de caracteres. Observe alguns exemplos de definição de listas.

```
>: [1, 2, 3, 4, 5];
>> [1, 2, 3, 4, 5] : list num

>: ['a', 'b', 'c', 'd', 'e'];
>> "abcde" : list char

>: HOPE;
>> "HOPE" : list char

>: [true, false];
>> [true, false] : list truval
```

Os dados exclusivamente de uma lista (não de uma tupla) podem ser apresentados separados por linhas a partir do uso do comando `write` ao invés de serem apresentados todos dentro de uma linha delimitada entre colchetes e separados por vírgulas. Observe a execução e resposta da instrução seguinte.

```
>: write([1, 2, 3, 4, 5]);
1
2
3
4
5
>:
```

Note que os valores da lista são apresentados linha a linha antes da indicação do *prompt* de comando do ambiente interativo *HOPE* em uso.

O comando `write` pode ser usado para apresentação de mensagens isoladas por ser uma mensagem considerada internamente uma lista de caracteres. Observe o exemplo a seguir.

```
>: write("Programacao HOPE");  
Programacao HOPE>:
```

Note que no caso de uma mensagem esta é apresentada antes do símbolo de *prompt*, diferentemente de uma sequência de dados numéricos que gera uma nova linha a cada número apresentado.

Caso queira que o símbolo de *prompt* seja apresentado depois da mensagem é necessário fazer uso do símbolo de código *escape* de controle para geração de nova linha `"\n"` como indicado na instrução.

```
>: write("Programacao HOPE\n");  
Programacao HOPE  
>:
```

Observe em seguida a apresentação de listas de caracteres e de valores lógicos.

```
>: write(['A', 'B', 'C', '\n']);  
ABC  
>:
```

```
>: write([true, false]);  
true  
false  
>:
```

Além do símbolo de código *escape* de controle `"\n"` para mudança de linha há um pequeno conjunto de outros símbolos como indicados.

- `\n` pula linha;
- `\t` tabulação horizontal;
- `\'` insere aspas simples na cadeia de caracteres;

- `\"` insere aspas inglesas na cadeia de caracteres;
- `\b` volta uma posição;
- `\r` retorno de cursor na mesma linha;
- `\\` apresenta barra invertida;

Para facilitar a construção de listas é possível criar funções que façam automaticamente o preenchimento de elementos a partir da execução dessa função. Observe as instruções seguintes.

```
dec listad : num -> list num;  
--- listad n <= if n = 0 then nil else n :: listad (n - 1);
```

Na sequência execute a instrução seguinte e observe o resultado apresentado.

```
>: listad(5);  
>> [5, 4, 3, 2, 1] : list num
```

Note que fornecido o valor "5" para a função "listad" apresenta os valores em ordem decrescente. Observe que para contar de "5" a "1" a função pega o valor do argumento e decrementa "1" até que o valor do argumento seja zero e neste caso coloca o valor "nil" no final da lista para indicar sua finalização. A construção da lista é produzida pelo trecho do código "`n::listad(n - 1)`".

Para definir a criação de uma lista crescente é necessário considerar uma estratégia diferente de geração da lista. Assim sendo, execute as instruções a seguir.

```
dec listac : num -> list num;  
--- listac n <= if n = 0 then nil else listac (n - 1) <> [n];
```

Na sequência execute a instrução seguinte e observe o resultado apresentado.

```
>: listac(5);  
>> [1, 2, 3, 4, 5] : list num
```

Observe o uso do operador "<>" (*union*) que tem por finalidade unir duas ou mais listas em uma só lista. Desta forma o operador "<>" usa duas listas para compor uma só. No exemplo indicado como crescente ocorre que a cada chamada recursi-

va da função `"listac"` o valor de `"[n]"` isolado é acrescido a lista processada pela função gerando uma sequência inversa.

4.3 FUNÇÕES DE ORDEM SUPERIOR

Uma função de ordem superior ou função de alta ordem é um recurso que pode receber como argumento uma função ou gerar como resultado uma função. O uso de funções de ordem superior é uma das prerrogativas do paradigma declarativo funcional, podendo ser fornecidas pela própria linguagem ou desenvolvidas a partir da necessidade da programação.

O exemplo a seguir demonstra o uso de função comum chamada `"soma_raizes"` que soma as raízes quadradas de dois valores inteiros delimitados entre os argumentos `"i"` (início) e `"f"` (fim) que são obtidos a partir do uso da função interna da linguagem `"sqrt"`.

```
dec soma_raizes : num # num -> num;
--- soma_raizes (i, f) <= if i > f then 0
                        else sqrt i + soma_raizes (1 + i, f);
```

Para verificar a ação da função `"soma_raizes"` execute a instrução seguinte.

```
>: soma_raizes(1, 5);
>> 8.38233234744176 : num
```

Veja que o resultado apresentado é o somatório das raízes quadradas dos valores inteiros de `"1"` a `"5"`. Note que a função `"soma_raizes"` e a função `"sqrt"` não estão sendo usadas como funções de primeira ordem, são apenas funções comuns, sendo necessário extrair a raiz quadrada de cada valor para em seguida produzir o somatório de cada um dos valores calculados (neste caso, as raízes quadradas).

E se quiser fazer o somatório dos valores pares entre dois limites? Seria necessário criar outra função de somatório para atender a esta necessidade. E se fosse usada apenas uma função para calcular o somatório das raízes quadradas, dos quadrados ou qualquer outra ação entre os limites de dois valores fornecidos ao estilo `"somatorio(1, 5, acao)"`, onde `"acao"` será a definição de uma função de ordem superior aplicada a ação desejada.

Funções de ordem superior são maneiras de se definir abstrações para a realização de certas operações sem se preocupar como essas ações são efetivamente feitas, uma vez que já estão definidas. Neste sentido, é possível definir uma função chamada "**somatorio**" que possa ser usada de forma mais inteligente que a função "**soma_raizes**".

A seguir é declarada uma função, com toque genérico, chamada "**somatorio**" que fará uso de alguma outra função definida passada a ela como argumento. Desta forma, observe o código da função a seguir.

```
dec somatorio : num # num # (num -> num) -> num;
--- somatorio (i, f, func) <= if i > f then 0
                                else func i + somatorio (1 + i, f, func);
```

Para verificar a ação da função "**somatorio**" execute a instrução seguinte.

```
>: somatorio(1, 5, sqrt);
>> 8.38233234744176 : num
```

A função "**somatorio**" faz uso da estrutura de argumentos "**num # num # (num -> num) -> num**", onde o trecho marcado entre parênteses "**(num -> num)**" refere-se a definição do terceiro argumento que representa a recepção de uma função de ordem superior com um argumento simples de entrada e um valor de retorno. Internamente na correspondência de padrão da função "**somatorio**" é definida a estrutura de uso da função "**(i, f, func)**", onde o argumento "**i**" refere-se ao valor inicial do limite da sequência numérica, o argumento "**f**" refere-se ao valor final do limite da sequência numérica e "**func**" refere-se ao uso de uma função de ordem superior.

A função "**somatorio**" poderá ser usada para diversas ações sobre certa sequência de valores. Por exemplo, para apresentar o somatório dos quadrados dos valores de "**1**" a "**5**" será definida uma função simples de primeira ordem chamada "**quadrado**".

```
dec quadrado : num -> num;
--- quadrado n <= pow (n, 2);
```

Agora execute a instrução.

```
>: somatorio(1, 5, quadrado);  
>> 55 : num
```

Observe que a função "**somatorio**" pode ser usada genericamente para a obtenção da soma de diversas operações.

4.4 FUNÇÕES ANÔNIMAS

Uma função anônima ocorre com a definição de uma função sem nome de identificação. Este tipo de função é também chamada função *lambda*. Na linguagem *HOPE* funções anônimas são usadas dentro de funções nominais como ações de *call-back*, sendo identificadas a partir de expressões lambda declaradas internamente dentro de ações seguindo a sintaxe.

```
lambda <argumento> => <ação>;
```

Onde "**argumento**" pode ser a definição de um ou mais elementos delimitados como tupla e "**ação**" é a definição de expressões matemáticas ou lógicas, podendo ser inclusive obtidas a partir da definição funções na forma mais clássica existente na linguagem *HOPE*. O símbolo "**=>**" no caso de definição de expressões lambda é usado para determinar a recursão da função ao invés de usar o símbolo "**<=**".

A maneira mais simples (e até inútil) de definir uma expressão lambda é considerar uma função sem nome que apresente a soma de dois valores numéricos e usá-la de forma isolada. Observe a instrução a seguir com o uso de uma expressão lambda e do resultado obtido a partir da execução da expressão.

```
>: (lambda (a, b) => a + b) (1, 2);  
>> 3 : num
```

Apesar de ser uma forma não muito adequada de uso de expressão lambda o exemplo demonstra que a soma ocorre a partir da recepção de dois argumentos "**a**" e "**b**" identificados anonimamente pelo comando "**lambda**".

Para um exemplo mais consistente no uso de expressões lambda considere a declaração e definição de uma função chamada "**mapa**" que permite iterar todos os

elementos de uma lista, aplicando-lhes uma regra para obter outra lista com os valores processados a partir da iteração. Observe as instruções a seguir.

```
dec mapa : list num # (num -> num) -> list num;  
--- mapa (nil, func) <= nil;  
--- mapa (i :: f, func) <= func i :: mapa (f, func);
```

A partir da definição da função "mapa" é possível aplicar diversas regras para obter o conjunto imagem do conjunto domínio usado como fonte do mapeamento. Observe a ação para criar um conjunto imagem formado pelos triplos dos valores dos elementos existentes no conjunto domínio.

```
>: mapa ([0, 1, 2, 3, 4, 5], lambda n => 3 * n);  
>> [0, 3, 6, 9, 12, 15] : list num
```

Assim como a definição de funções comuns as funções definidas a partir de expressão lambda podem ser operacionalizadas com diferentes padrões, separando cada padrão com o símbolo "|" (*pipe*).

Observe a instrução seguinte que apresenta o valor "100" no conjunto imagem se no conjunto domínio existir algum valor "0". Para valores diferentes de "0" será apresentado o dobro de cada valor do conjunto domínio.

```
>: mapa ([0, 1, 2, 3, 0, 4], lambda 0 => 100 | n => 2 * n);  
>> [100, 2, 4, 6, 100, 8] : list num
```

Funções anônimas são funções definidas para serem usadas como funções de primeira ordem.

4.5 SIMPLIFICAÇÃO DE EXPRESSÕES

Expressões são formas de exprimir ações matemáticas e lógicas em programação de computadores. Por ser a linguagem *HOPE* pertencente ao paradigma declarativo funcional seus programas consistem em expressões. Tudo o que é programado em *HOPE* é uma expressão.

Há diversas situações operacionais em expressões precisam ser simplificadas para maior legibilidade. Para essas situações a linguagem *HOPE* possui alguns comandos auxiliares a este tipo de operação, sendo: "**letrec**", "**let**", "**in**", "**whererec**" e "**where**".

Como exemplo considere uma função chamada "**quad_soma1**" que apresente o resultado do quadrado da soma de dois valores numéricos passados como argumentos (BAYLEI, 1985), sem fazer uso da função "**pow**". Se fornecidos os valores "**2**" e "**3**" a função deve retornar o resultado "**25**". Uma possibilidade de solução para o problema é realizar o cálculo como demonstrado a seguir.

```
dec quad_soma1 : num # num -> num;  
--- quad_soma1 (a, b) <= (a + b) * (a + b);
```

Agora execute a instrução.

```
>: quad_soma1(2, 3);  
>> 25 : num
```

Em uma linguagem de programação imperativa, uma maneira de simplificar expressões como a expressão "**(a + b) * (a + b)**" é definir sua operação como demonstrado em seguida.

```
x ← a + b  
escreva x * x
```

Algo semelhante é possível de ser realizado com a linguagem *HOPE* ao utilizar os comandos "**let**" e "**in**". Observe o código da função "**quad_soma2**".

```
dec quad_soma2 : num # num -> num;  
--- quad_soma2 (a, b) <= let x == a + b in x * x;
```

Agora execute a instrução.

```
>: quad_soma2(2, 3);  
>> 25 : num
```

O comando "**let**" diz que a variável "**x**" a sua frente, sua correspondência de padrão, é definida por meio do símbolo "**==**" como o local de controle para a soma de

"**a + b**" e determina a do resultado do cálculo do quadrado de "**a + b**" após o comando "**in**".

A simplificação de expressões com os comandos "**let**" e "**in**" segue como forma sintática o estilo.

```
let <padrão> == <expressão1> in <expressão2>;
```

Onde "**padrão**" é o ponto de recepção de retorno do resultado operacionalizado na "**expressão2**" a partir da definição dos elementos definidos na "**expressão1**".

Esta ação é similar a executar dentro da função de cálculo a definição de uma função anônima como a seguir.

```
(lambda <padrão> => <expressão2>) <expressão1>
```

Desta forma, observe as instruções a seguir para declaração e definição da função "**quad_soma3**".

```
dec quad_soma3 : num # num -> num;  
--- quad_soma3 (a, b) <= (lambda x => x * x) (a + b);
```

Agora execute a instrução.

```
>: quad_soma3(2, 3);  
>> 25 : num
```

A ação de simplificação de expressões em **HOPE** pode ser escrita de uma segunda maneira utilizando-se o comando "**where**". Observe a seguir o código para a função "**quad_soma4**".

```
dec quad_soma4 : num # num -> num;  
--- quad_soma4 (a, b) <= x * x where x == a + b;
```

Agora execute a instrução.

```
>: quad_soma4(2, 3);  
>> 25 : num
```

O comando "**where**" diz como a expressão, neste caso, "**x * x**" deve retornar seu resultado para cada um dos "**x**" que representa o resultado de "**a + b**".

A simplificação de expressões com o comando "**where**" segue como forma sintática o estilo.

```
<expressão1> where <padrão> == <expressão2>;
```

Onde "**padrão**" é o ponto de recepção de retorno do resultado operacionalizado na "**expressão2**" sobre os elementos da "**expressão1**".

A simplificação de expressões em *HOPE* pode ser feita com auxílio dos comandos "**letrec**" e "**whererec**" homônimos dos comandos "**let**" e "**where**" com a diferença de serem mais restritivos para sua operação, apesar de produzirem o mesmo resultado que os comandos "**let**" e "**where**".

Para exemplificar o uso do comando "**where**" considere o código da função seguinte chamada "**quad_soma5**".

```
dec quad_soma5 : num # num -> num;
--- quad_soma5 (a, b) <= letrec x == a + b in x * x;
```

Agora execute a instrução.

```
>: quad_soma5(2, 3);
>> 25 : num
```

Esta ação é similar a executar dentro da função de cálculo a definição de uma função anônima como a seguir.

```
(lambda <padrão> => <expressão2>) (fix(lambda <padrão>
=> <expressão1>))
```

Note a definição de uso da função "**fix**" que deverá ser codificada exatamente como se apresenta o código seguinte (PATERSON, 2000).

```
dec fix : (alpha -> alpha) -> alpha;
--- fix f <= f (fix f);
```

Desta forma, observe as instruções a seguir para declaração e definição da função "quad_soma6".

```
dec quad_soma6 : num # num -> num;  
--- quad_soma6 (a, b) <= (lambda x => x * x) (fix (lambda x  
                                         => (a + b)));
```

Agora execute a instrução.

```
>: quad_soma6(2, 3);  
>> 25 : num
```

Em contra ponto ao comando "letrec" há o comando "whererec". Observe a seguir o código para a função "quad_soma7".

```
dec quad_soma7 : num # num -> num;  
--- quad_soma7 (a, b) <= x * x whererec x == a + b;
```

Agora execute a instrução.

```
>: quad_soma4(2, 3);  
>> 25 : num
```

Fica evidente, com esta apresentação, que a linguagem *HOPE* disponibiliza quatro maneiras de se fazer uso de ações de simplificação de expressões, sendo que muitas vezes é mais cômodo operar com os comandos "where / whererec", sendo este mais popular que os comandos "let / letrec".

4.6 DEFINIÇÃO DE MÓDULOS

Módulos são para a linguagem *HOPE* o que as bibliotecas são para linguagens como *Pascal*, *C* e *C++*. Desta forma, um módulo é um conjunto de funções que podem ser definidos e deixados prontos para o uso a qualquer momento.

O módulo principal da linguagem nos interpretadores *MA-HOPE* e *RP-HOPE* é chamado "Standard.hop" e concentra as funcionalidades padrão para uso quando do carregamento em memória do ambiente interativo.

Anteriormente os módulos "**Standard.hop**" dos interpretadores *HOPE* foram ajustados para atenderem necessidades mínimas e se compatibilizarem. Além dessa tarefa foi criado de forma bem simples o módulo "**constant.hop**" que foi posteriormente inserido no módulo "**Standard.hop**".

Apesar do tema já ter sido apresentado de forma simples é importante conhecer alguns detalhes a mais. Os módulos *HOPE* administrados pelos interpretadores *MA-HOPE* e *RP-HOPE* são formados por três níveis de encapsulamento: público privado externo e privado interno.

Para usar o modo publico de encapsulamento dentro de um módulo não é necessário nenhuma ação operacional, basta manter a declaração (comando "**dec**") e a definição da função (comando "**---**") fora da seção privada do módulo.

Para usar o modo privado interno manter a declaração (comando "**dec**") e a definição da função (comando "**---**") deve ser estabelecida após a definição de seção privada com o comando "**private**".

Para usar o modo privado externo manter a declaração (comando "**dec**") deve estar na parte pública do módulo e a definição da função (comando "**---**") deve estar dentro da seção privada com o comando "**private**".

A estrutura de definição de um módulo pode fazer uso de todo o conjunto de comandos da linguagem já apresentados. Observe a seguir uma estrutura típica básica de módulos que pode ser considerada do ponto de vista dos interpretadores *MA-HOPE* e *RP-HOPE*.

```
uses <módulo1> [, <módulo2>, <móduloN>];

dec <função pública> : <argumentos> -> <retorno>;
--- <função pública> <variável argumento> <= <operação>;

dec <função privada externa> : <argumentos> -> <retorno>;

private;

uses <módulo1> [, <módulo2>, <móduloN>];
```



```
--- <função privada externa> <variável argumento> <= <operação>;

dec <função privada interna> : <argumentos> -> <retorno>;
--- <função privada interna> <variável argumento> <= <operação>;
```

Um arquivo de módulo pode usar internamente diversas linhas contendo o comando **"uses"** para efetuar a chamada de outros arquivos de módulo. Desta forma, é possível a um módulo usar os recursos de outro módulo para implementar suas ações. Essa estratégia permite a definição de grupos de funções em diversos módulos que podem ser combinados de diversas formas.

É importante considerar que a definição de funções dentro do arquivo de módulo deve ser feita seguindo uma estrutura **bottom-up**, ou seja, as funções devem ser definidas na ordem em que devem ser usadas.

Como exemplo, considere a criação de um arquivo de módulo chamado **"math.hop"** contendo algumas funcionalidades que serão úteis para o estudo do próximo capítulo.

O arquivo de módulo deve ser definido em um programa de edição de textos simples que opere com a gravação de seu conteúdo em formato ASCII.

No caso do sistema operacional FreeBSD o arquivo de módulo deve estar no mesmo diretório em que o interpretador é chamado ou ser colocado no diretório de instalação das bibliotecas **"/usr/local/share/hope/lib"**. Se o sistema operacional for o Windows pode-se deixar o arquivo de módulo junto ao diretório onde o interpretador foi descompactado.

Atente para o conteúdo do arquivo de módulo simples com as instruções de declaração e definição de funções a ser gravado no arquivo **"math.hop"**. Neste arquivo de módulo serão definidas funções com seus nomes em inglês para manter homogeneidade estrutural com a linguagem.

```
!! Módulo .....: math.hop
!! Finalidade ...: Funções matemática gerais.
!! Autor .....: Augusto Manzano
!! Versão .....: 1.0
!! Data .....: 28/06/2018
```



```
--- factfn (i, f, fn) <= if i > f then 1
                        else fn i * factfn (1 + i, f, fn);
--- fact n <= product (1, n);
--- sum n <= sumrng (1, n);
```

Após gravar o arquivo de módulo encerre a execução do ambiente e em seguida abra-o novamente. Na sequência execute a instrução.

```
uses math;
```

As funcionalidades do módulo "**math.hop**" são carregadas para a memória e podem então ser utilizadas. Observe a seguir as instruções exemplificando o uso das funções definidas no arquivo de módulo "**math.hop**".

```
>: sum(5);
>> 15 : num

>: fact(5);
>> 120 : num

>: max(3, 7);
>> 7 : num

>: min(5, 9);
>> 5 : num

>: add(2, 3);
>> 5 : num

>: sub(2, 3);
>> -1 : num

>: mult(2, 3);
>> 6 : num
```

```
>: divf(5, 2);
```

```
>> 2.5 : num
```

```
>: sumrng (1, 5);
```

```
>> 15 : num
```

```
>: product(1, 5);
```

```
>> 120 : num
```

```
>: sumfn(1, 5, one);
```

```
>> 15 : num
```

```
>: factfn (1, 5, one);
```

```
>> 120 : num
```

```
>: sumfn(1, 5, sqrt);
```

```
>> 8.38233234744176 : num
```

```
>: factfn (1, 5, sqrt);
```

```
>> >> 10.9544511501033 : num
```

```
>: sumfn(1, 5, fact);
```

```
>> 153 : num
```

```
>: sumfn(1, 5, sum);
```

```
>> 35 : num
```

```
>: factfn (1, 5, fact);
```

```
>> 34560 : num
```

```
>: factfn (1, 5, sum);
```

```
>> 2700 : num
```

Após efetuar os testes de todas as funções e das possibilidades de combinação das funções que são de primeira ordem note que todas as funções estão disponíveis do arquivo de módulo, pois a declaração foi mantida pública, apesar da definição estar definida como privada.

Para verificar a funcionalidade não permitida de uma função declarada e definida como privada considere o próximo código para um arquivo de módulo chamado "**teste.hop**" que apresenta o resultado da fatorial de um valor qualquer a partir do uso de recursão de cauda..

```
!! Módulo .....: teste
!! Finalidade ...: Função fatorial com recursividade de cauda.
!! Autor .....: Augusto Manzano
!! Versão .....: 1.0
!! Data .....: 28/06/2018

dec factorial : num -> num;

private;

dec factbase : num # num -> num;
--- factbase (n, p) <= if n =< 1 then p
                        else factbase (n - 1, p * n);

--- factorial n <= factbase (n, 1);
```

Após criar e gravar o arquivo de módulo execute a instrução.

```
uses teste;
```

Agora execute as instruções a seguir e observe os resultados apresentados.

```
>: factorial(5);
>> 120 : num
```

```
>: factbase(1, 5);
semantic error - factbase: undefined variable
```

Observe que ao ser chamada a função "**factorial**" está apresenta o resultado de sua operação. Veja que esta função faz uso da função de base "**factbase**" definida como privada. A tentativa de fazer uso da função "**factbase**" gera a apresentação da mensagem de erro "**semantic error - factbase: undefined variable**". Note que a mensagem de erro ocorre como se a função "**factbase**" não estivesse carregada na memória. E de fato, não está.

Note atentamente a definição do código do arquivo de módulo. Veja que de forma pública está definida apenas a função "**factorial**", mas tanto seu código de ação como o código da função "**factbase**" são definidos como recursos privados. Pelo fato da função "**factbase**" se privada só pode ser usada por uma função pública que a use. Neste caso a função "**factbase**" é uma função privada interna.

Entre os interpretadores usados nesta obra e o projeto original da linguagem há como apontado pelos autores dos interpretadores a existência de algumas diferenças existentes, apesar de não indicarem claramente quais são essas diferenças. Uma das diferenças percebidas é em relação a definição do arquivo de módulo que na forma original tem as funções declaradas de forma endentada com espaço dois entre os comandos "**module**" e "**end**". Além, dessa evidente diferença, há a ausência do uso do símbolo de ponto na finalização dos comandos da linguagem, como apontado no artigo *HOPE: An experimental applicative language* de Burs-tall, MaeQueen e Sannella na apresentação da linguagem *HOPE*.

Um modelo típico de definição de arquivo de módulo, respeitando a forma original da linguagem *HOPE* e do estilo usado pelos interpretadores é baseado na forma.

```
module <nome do módulo>;
  dec <função> : <argumento> -> <argumento>;
  --- <função> [(]<padrão>[)] <= <ação>;
end
```

Os comandos "**module**" e "**end**" apesar de poderem ser usados, sem nenhum problema, nos interpretadores *MA-HOPE* e *RP-HOPE* não possuem nesses ambientes nenhum efeito direto, estando presentes apenas para manter compatibilidade com código escrito para outras implementações da linguagem.

5

Programação com listas

Este capítulo apresenta detalhes relacionados ao uso e manipulação de listas que é o modo mais popular e comum de tratamento de dados em linguagens funcionais. Neste sentido, é indicado como operacionalizar ações básicas e contextualizadas a partir de exemplos baseados e adaptados da documentação gerada por Ross Paterson e Piyush Maheshwari indicados nas referências bibliográficas deste trabalho, além de funções escritas exclusivamente para essas demonstrações. São apresentados neste capítulo assuntos relacionados a manipulação: de listas ao estilo de conjuntos, conjuntos e funções, mapeamento, filtragem, redução, transposição entre outros assuntos relacionados. As funções apresentadas possuem em si muito mais informação do que as explicações dadas. Tire um tempo para observar atentamente e vagorosamente cada um dos códigos mostrados.

5.1 LISTA COMO ESTRUTURA DE DADOS

Uma lista em linguagens funcionais é formada basicamente por dois componentes estruturais: a cabeça (*head*) que determina o primeiro elemento da lista e a cauda (*tail*) formada do segundo ao último elemento da lista.

Este tópico apresenta uma pequena coleção de funcionalidades encontradas em diversas linguagens funcionais como *Haskell*, *Elixir*, *Julia*, *OCaml* entre outras que não são encontradas diretamente na linguagem *HOPE*. Pelo fato de ser *HOPE* uma linguagem minimalista ela fornece apenas a infraestrutura básica de manipulação de dados deixando a cargo de seu usuário a tarefa de construir as ferramentas de manipulação necessárias.

Isso pode parecer para alguns uma desvantagem, mas é de extremo valor para o iniciante que ao aprender como construir as funcionalidades necessárias tem a rara oportunidade de colocar em prática diversos elementos da programação funcional, solidificando bases e entendendo melhor como produzir melhores programas funcionais, inclusive em outras linguagens do paradigma funcional.

Utilize este capítulo como reforço e fundamento de fixação dos conceitos de lógica de programação funcional, pois para o desenvolvimento dos detalhes aqui apresentados estão sendo usados todos os detalhes apresentados anteriormente.

Antes de iniciar este capítulo deixe carregado na memória o arquivo de módulo `"math.hop"`. Para tanto, use a instrução.

```
uses math;
```

A definição de uma lista vazia na linguagem *HOPE* é estabelecida a partir da instrução, que fora usada para definir a estrutura de constantes implícitas.

```
dec A : list num;  
--- A <= [];
```

De maneira simplificada a definição de uma constante implícita ou mesmo de uma variável imutável nos interpretadores usados pode ser escrita a partir da instrução.

```
A : list num; A <= [];
```

Em seguida execute a instrução.

```
>: A;  
>> nil : list num
```

Ao ser informado o nome da lista como `"A"` e acionando a tecla `"<Enter>"` ocorre a apresentação do valor `"nil"` indicando em se tratar de uma lista vazia.

Devido as características operacionais da linguagem *HOPE* a função `"A"` representado uma lista vazia pode ser usada para armazenar algum valor temporariamente na memória, não mantendo os valores armazenados para uso posterior. Lembre-se de que *HOPE* é uma linguagem experimental e acadêmica, não sendo seu objetivo ser uma linguagem para uso comercial ou industrial.

As funcionalidades deste capítulo são indicadas apenas para operações com listas numéricas. Assim sendo, para realizar ações de entrada de valores na lista "A" considere as instruções seguintes.

```
dec insert : num # list num -> list num;
--- insert (a, nil) <= a :: nil;
--- insert (a, x :: xs) <= if a < x
                           then a :: x :: xs
                           else x :: insert (a, xs);
```

Observe que a função "insert" efetua a receptação de dois argumentos polimórficos (pode ser usado com qualquer tipo de dado existente ou definido na linguagem). O primeiro argumento é um valor que será inserido na lista indicada no segundo argumento e fornecerá como retorno (indicação após o símbolo "->") uma lista.

A primeira linha da função "insert (a, nil) <= a :: nil" informa que a inserção do elemento "a" com a próxima posição da lista marcada como nula ("nil") junto a correspondência de padrão ("insert (a, nil)") deve ser aceita e que uma nova posição "nil" deve ser definida. Esta operação ocorrerá se a lista usada estiver totalmente vazia ou quando encontra a primeira posição vazia após o último elemento existente na lista.

A segunda linha (e principal ação da função) é usada quando a lista possui mais de um elemento definido. Isto é estabelecido no trecho "insert (a, x :: xs)" que identifica a correspondência de padrão da função, sendo "a" o elemento a ser inserido na lista "x :: xs" (normalmente listas em linguagens funcionais são identificadas com "x" marcando a cabeça e "xs" marcando a cauda da lista). Na sequência, após o símbolo "<=" é estabelecida ação de inserção de um novo elemento classificado em ordem crescente se for numérico ou ascendente se for alfanumérico junto aos demais elementos já existentes. Se o elemento "a" inserido na lista for menor que o elemento "x" que está na cabeça da lista o elemento "a" é colocado a frente dos demais elementos ("a :: x :: xs"), caso contrário o elemento "a" será inserido após o início da fila ("x :: insert (a, xs)") buscando-se uma nova posição a frente do restante da lista se assim for necessário por meio da ação de recursão "insert (a, xs)".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```

>: insert(1, A);
>> [1] : list num

>: insert(2, insert(1, A));
>> [1, 2] : list num

>: insert(2, insert(3, insert(1, A)));
>> [1, 2, 3] : list num

>: insert(4, insert(2, insert(3, insert(1, A))));
>> [1, 2, 3, 4] : list num

```

Note que não importa a ordem estabelecida pela função **"insert"** para os elementos da lista, sua apresentação será sempre automaticamente classificada como crescente/ascendente.

Apesar da função **"insert"** poder ser usada para representar a definição de listas é, muitas vezes, mais prático, devido às características operacionais da linguagem produzir listas de forma fechada como indicado a seguir.

```

dec B : list num;
--- B <= [1, 2, 3];

```

O tratamento de listas em linguagens funcionais pode ser definido a partir de operações lógicas como as instruções, lembrando que **"true : truval"** e **"false : truval"** são apresentados no uso do ambiente *MA-HOPE* e **"true : bool"** e **"false : bool"** no uso do ambiente *RP-HOPE*.

```

>: insert(2, insert(3, insert(1, A))) = B;
>> true : truval

>: [2, 3, 1] = B;
>> false : truval

>: [2, 3, 1] = [1, 2, 3];
>> false : truval

```

```
>: [1, 2, 3] = [1, 2, 3];  
>> true: truval
```

```
>: [1, 1] = [1];  
>> false : truval
```

Uma maneira de preencher o conteúdo de listas é fazer uso de uma ação em que se define o valor inicial, final e incremento de definição da sequência de valores. No entanto, a linguagem *HOPE* não possui tal funcionalidade, o que exige que seja escrita uma função para esta finalidade como a seguinte.

```
dec range : num # num # num -> list num;  
--- range (s, f, i) <= if s > f  
    then []  
    else s :: range (s + 1 + (i - 1), f, i);
```

A função "**range**" efetua a recepção de três argumentos respectivamente definidos para determinar o começo "**s**" (*start*), fim "**f**" (*finish*) e incremento "**i**" (*increase*) de uma contagem retornando uma lista com os valores estabelecidos.

Se o valor do início da contagem da função "**range**" for maior que o valor do fim da contagem uma lista vazia é retornada. Caso contrário a lista de valores será gerada com o trecho "**s :: range (s + 1 + (i - 1), f, i)**", em que a chamada recursiva da função "**range**" usa para determinar os próximos valores o cálculo "**s + 1 + (i - 1), f, i**", onde o valor do próximo início é determinado pelo trecho de código "**s + 1 + (i - 1)**" quando para "**s**" é gerado um "novo" valor.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: range(1, 5, 1);  
>> [1, 2, 3, 4, 5] : list num
```

```
>: range(0, 10, 3);  
>> [0, 3, 6, 9] : list num
```

```
>: range(1, 10, 4);  
>> [1, 5, 9] : list num  
  
>: range(1, 3, 1) = B;  
>> true : truval  
  
>: range(1, 3, 1) = [1, 2, 3];  
>> true : truval
```

Uma lista pode necessitar de outras operações para seu gerenciamento além das, até então, apresentadas, como a que seguem.

Uma ação que pode ser estabelecida em uma lista é a inversão dos valores existentes. Para esta ação é indicada a função **"reverse"** a seguir.

```
dec reverse : list num -> list num;  
--- reverse [] <= [];  
--- reverse (x :: xs) <= reverse xs <> [x];
```

A função **"reverse"** recebe uma lista como argumento e retorna uma lista como resultado. Se a lista estiver vazia será retornado uma lista vazia, mas se a lista passada contiver elementos **"reverse (x :: xs)"** ação executada fará a reversão dos elementos com **"reverse xs <> [x]"**, onde será feita a união dos elementos da cauda **"xs"** com o elemento da cabeça **"x"** da lista por meio do operador **"<>"**.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: reverse([1, 2, 3, 4, 5]);  
>> [5, 4, 3, 2, 1] : list num  
  
>: reverse(range(0, 8, 2));  
>> [8, 6, 4, 2, 0] : list num
```

Assim como é possível inserir elementos em uma lista é possível retirar elementos de uma lista definida. Para esta ação considere a função **"remove"**.

```
dec remove : num # list num -> list num;
--- remove (a, []) <= [];
--- remove (a, x :: xs) <= if a = x
                           then xs
                           else x :: remove (a, xs);
```

A função "**remove**" opera a partir de dois argumentos o primeiro o valor a ser removido e o segundo a lista que terá um elemento removido, tendo como retorno uma lista com o elemento removido.

A primeira definição da função mostra que se removido um elemento de uma lista vazia o resultado será uma lista vazia, mas a segunda definição a partir da correspondência de padrão "**(a, x :: xs)**" verifica se o valor do elemento, neste caso, "**a**" é igual ao elemento da cabeça da lista "**x**", sendo os valores iguais a decisão retorna a cauda da lista com o restante existente. Não sendo a condição verdadeira é indicado o elemento da cabeça da lista com o processamento da recursão e repetição da ação até que a primeira definição em algum momento ocorra.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: remove(1, [1, 2, 3, 4, 5]);
>> [2, 3, 4, 5] : list num

>: remove(3, [1, 2, 3, 4, 5]);
>> [1, 2, 4, 5] : list num

>: remove(3, range(1, 9, 2));
>> [1, 5, 7, 9] : list num
```

Dentre a coleção de recursos de gerenciamento de listas são funcionalidades importantes a detecção da cabeça da lista (função "**head**"), sua cauda (função "**tail**"), o primeiro elemento (função "**first**") e último elemento (função "**last**"). Observe as três funções seguintes.

```
dec head : list num -> num;
--- head (x :: xs) <= x;
```

```
dec tail : list num -> list num;
--- tail (x :: xs) <= xs;

dec first : list num -> num;
--- first (x :: xs) <= x;

dec last : list num -> num;
--- last [x] <= x;
--- last (x :: xs) <= last xs;
```

A função "**head**" retorna o elemento "**x**" da cabeça da lista "**x :: xs**" indicada. Já a função "**tail**" retorna a cauda "**xs**" da lista "**x :: xs**". A função "**last**" devolve o elemento "**x**" se a lista tiver apenas o elemento "**x**" na medida em busca por recursão o último elemento da cauda (**last xs**).

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: head([1, 2, 3, 4, 5]);
>> 1 num

>: tail([1, 2, 3, 4, 5]);
>> [2, 3, 4, 5] : list num

>: first([1, 2, 3, 4, 5]);
>> 1 num

>: last([1, 2, 3, 4, 5]);
>> 5 num
```

Outra funcionalidade comum é um recurso que apresente a quantidade de elementos existentes em uma lista a partir de uma função chamada "**length**". Atente para as instruções seguintes.

```
dec length : list num -> num;
--- length [] <= 0;
--- length (x :: xs) <= succ (length xs);
```

A função "**length**" recebe como argumento uma lista e retorna um valor numérico como resposta, retornando o valor "**0**" se a lista estiver vazia, caso contrário a função recursivamente executa a função interna "**succ**" que retorna o valor sucessor de cada uma das chamadas recursivas da função. A função "**succ**" está sendo usada como um acumulador das vezes em que a ação recursiva foi executada pela função "**length**" até chegar ao final da lista.

A última linha de definição da função "**length**" poderá ser codificada como.

```
--- length (x :: xs) <= length xs + 1;
```

Para a realização de teste execute a instrução seguinte observando os resultados gerados após cada ação.

```
>: length([1, 2, 3, 4, 5]);
>> 5 num
```

O gerenciamento de listas pode necessitar de funcionalidades que informe se certo valor existe ou não em uma lista, bem como pode indicar em que posição da ordinal da lista certo valor se encontra. Para solucionar essas operações considere as instruções a seguir para uso das funções "**member**" e "**find**". Atente para uso do tipo de dado "**bool**" no lugar do tipo "**truval**" se estiver em uso o ambiente **RP-HOME**.

```
dec member: num # list num -> truval;
--- member (_, nil) <= false;
--- member (a, x :: xs) <= if a = x
                           then true
                           else member (a, xs);
```

```

dec find : num # list num -> num;
--- find (a, []) <= if a > length []
    then error "valor muito alto"
    else error "valor muito baixo";
--- find (a, x :: xs) <= if a = x
    then 0
    else find (a, xs) + 1;

```

A função "**member**" retorna o valor "**true**" quando o valor do argumento "**a**" é igual ao valor da cabeça da lista "**x**", caso contrário efetua chamada recursiva para fazer a mesma verificação na próxima posição da cauda.

A função "**find**" retorna a posição cardinal quando o argumento "**a**" é igual ao valor da cabeça da lista representado pela variável "**x**". Caso contrário, a função efetua nova chamada recursiva acrescentando ao valor encontra mais "**1**" sobre a posição da cauda da lista e posicionado a próxima posição.

A finalização da execução da função "**find**" retorna a posição em que certo valor existe na lista, ou retorna a mensagem "**valor muito alto**" se um valor maior que o valor da lista for informado ou retorna a mensagem "**valor muito baixo**" se um valor menor que o valor da lista for informado. Note que para a apresentação das mensagens está sendo utilizada a função "**error**" com a indicação da mensagem desejada. A função "**error**" deve ser utilizada sempre com os comandos "**if**", "**then**" e "**else**".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```

>: member(3, [1, 2, 3, 4, 5]);
>> true : truval

>: member(9, [1, 2, 3, 4, 5]);
>> false : truval

>: find(3, [8, 7, 6, 5, 4, 3, 2, 1]);
>> 5 num

```



```
>: find(9, [8, 7, 6, 5, 4, 3, 2, 1]);  
user error - valor muito alto
```

```
>: find(0, [8, 7, 6, 5, 4, 3, 2, 1]);  
user error - valor muito baixo
```

```
>: find(minus 3, [8, 7, 6, 5, 4, 3, 2, 1]);  
user error - valor muito baixo
```

Observe que a função "**member**" indica com resultado "**true**" ou "**false**" se o valor indicado existe ou não dentro de certa lista. Já a função "**find**" indica o valor da posição em que se encontra certo valor, retornando um valor negativo quando o valor pesquisado não é existente na lista.

A partir da função "**find**" que apresenta um valor a partir de uma posição válida da lista pode-se realizar ação inversa com a função "**show**" que a partir da indicação de algum valor da lista válido mostra sua posição na lista. Assim, considere o código a seguir.

```
dec show : list num # num -> num;  
--- show ([], a) <= if a >= length []  
    then error "índice muito alto"  
    else error "índice muito baixo";  
--- show (x :: xs, 0) <= x;  
--- show (x :: xs, a) <= show (xs, a - 1);
```

A função "**find**" faz sua ação de pesquisa a partir do decremento de "**1**" sob o valor do argumento "**a**" a cada recursão que quando chegar a "**0**" fará a apresentação da variável "**x**" contendo o valor do elemento da posição indicada.

Se o valor do índice for maior ou igual ao tamanho da lista será retornada a mensagem de erro "**índice muito alto**". Caso contrário, se a recursão não encontrar o elemento pesquisado ocorrerá a apresentação da mensagem "**índice muito baixo**".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: show(3, [3, 2 ,4, 5, 1]);
>> 5 : num

>: show(4, [3, 2, 4, 5, 1]);
>> 1 : num

>: show(0, [3, 2, 4, 5, 1]);
>> 3 : num

>: show(5, [3, 2, 4, 5, 1]);
user error - índice muito alto

>: show(minus 1, [3, 2, 4, 5, 1]);
user error - índice muito baixo
```

Em complemento as ações já produzidas são interessantes ter em mãos funcionalidades para detectar o maior valor de uma lista (função "**list_max**"), o menor valor de uma lista (função "**list_min**"), o maior e menor valor de uma lista (função "**list_max_min**") e o menor e maior valor de uma lista (função "**list_min_max**"). Observe o conjunto de instruções a seguir.

```
dec list_max : list num -> num;
--- list_max [] <= error "sem maior valor: lista vazia";
--- list_max ([a]) <= a;
--- list_max (x :: xs) <= if x > list_max xs
                           then x
                           else list_max xs;

dec list_min : list num -> num;
--- list_min [] <= error "sem menor valor: lista vazia";
--- list_min ([a]) <= a;
--- list_min (x :: xs) <= if x < list_min xs
                           then x
                           else list_min xs;
```

```
dec list_min_max : list num -> list num;
--- list_min_max [] <= error "sem menor e maior valor: lista vazia";
--- list_min_max (x :: xs) <=
    [list_min (x :: xs), list_max (x :: xs)];

dec list_max_min : list num -> list num;
--- list_max_min [] <= error "sem maior e menor valor: lista vazia";
--- list_max_min (x :: xs) <=
    [list_max (x :: xs), list_min (x :: xs)];
```

Do conjunto de funções produzidas para o gerenciamento de informações relacionadas a manipulação de listas as funções de máximo e mínimo são as mais simples em sua concepção lógica.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: list_max [3, 2, 1, 6, 7, 9, 8, 0];
>> 9 : num

>: list_min [3, 2, 1, 6, 7, 9, 8, 0];
>> 0 : num

>: list_min_max [3, 2, 1, 6, 7, 9, 8, 0];
>> [0, 9] : list num

>: list_max_min [3, 2, 1, 6, 7, 9, 8, 0];
>> [9, 0] : list num
```

Já foram definidas funções para efetuar a inserção (função "**insert**") e remoção (função "**remove**") de valores em uma lista, além de outras funcionalidades. Dentro deste escopo é ideal possui uma função que efetue a substituição de algum valor da lista por outro valor fornecido. Desta forma, considere o código a seguir.

```

dec replace : list num # num # num # num -> list num;
--- replace (lst, p, e, c) <= lst;
--- replace (x :: xs, p, e, c) <= if c = p
                                then e :: (replace(xs, p, e, c + 1))
                                else x :: (replace(xs, p, e, c + 1));

```

A função "**remove**" opera a troca de valores dentro da lista a partir da indicação dos argumentos: lista, posição de troca, novo valor e início de operação como contador sempre definido como "0".

Para a realização de teste execute a instrução seguinte observando os resultados gerados após cada ação.

```

>: replace([1, 2, 3, 4, 5], 3, 9, 0);
>> [1, 2, 3, 9, 5] : list num

```

Observe que o valor da posição "3" identificado como "4" é substituído pelo valor "9".

Outro recurso que pode ser implementado é a definição de uma função que repita determinado elementos certo número de vezes. Para esta ação basta criar uma função com nome "**replicate**". Veja as instruções a seguir.

```

dec replicate : num # num -> list num;
--- replicate (a, r) <= if a <= 0
                        then []
                        else r :: replicate (a - 1, r);

```

A função "**replicate**" verifica inicialmente se o valor do argumento "a" é menor ou igual a zero e sendo deve retornar a lista definida na memória. Caso contrário a função criar a lista com o valor definido junto ao argumento "r".

Para a realização de teste execute a instrução seguinte observando os resultados gerados após cada ação.

```

>: replicate(10, 3);
>> [3, 3, 3, 3, 3, 3, 3, 3, 3, 3] : list num

```

A ação estabelecida para o gerenciamento de listas necessita ainda de duas funções especiais para como "**sort**" para colocar a lista em ordem crescente e "**uniq**" para evitar a existência de valores repetidos em uma lista.

A função "**sort**" a seguir usa para seu processamento o algoritmo de classificação **merge sort** sendo necessário para sua ação o auxílio de uma função suplementar chamada "**split**" que tem por finalidade particionar uma lista. Desta forma considere o conjunto a seguir de instruções.

```
dec split : list num -> list num # list num;
--- split [] <= ([], []);
--- split (x :: xs) <= (x :: zs, ys)
    where (ys, zs) == split xs;

dec sort' : list num # list num -> list num;
--- sort' ([], ys) <= ys;
--- sort' (xs, []) <= xs;
--- sort' (x :: xs, y :: ys) <= if x < y
    then x :: sort' (xs, y :: ys)
    else y :: sort' (x :: xs, ys);

dec sort : list num -> list num;
--- sort [] <= [];
--- sort [x] <= [x];
--- sort (x1 :: x2 :: xs) <=
    sort' (sort (x1 :: ys), sort (x2 :: zs))
    where (ys, zs) == split xs;
```

Para a realização de teste execute a instrução seguinte observando os resultados gerados após cada ação.

```
sort([9, 1, 8, 2, 7, 3, 6, 4, 5, 0]);
>> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] : list num
```

Para apresentar os valores em ordem decrescente utilize a função "**reverse**" em conjunto com a função "**sort**". Observe a instrução.

```
reverse(sort([9, 1, 8, 2, 7, 3, 6, 4, 5, 0]));
>> [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] : list num
```

Para permitir apenas a definição de valores únicos em uma lista usa-se a função **"uniq"** com o código a seguir.

```
dec uniq' : num -> list num -> list num;
--- uniq' x [] <= [];
--- uniq' x (y :: ys) <= if x = y
                        then uniq' x ys
                        else y :: uniq' y ys;

dec uniq : list num -> list num;
--- uniq [] <= [];
--- uniq (x :: xs) <= x :: uniq' x xs;
```

A função **"uniq"** usa como declaração de argumentos de sua operação um estilo diferente do que foi mostrado até aqui: **"num -> list num -> list num"**. Neste estilo **"num -> list num"** é internamente tratado com os argumentos separados com apenas espaço em branco na forma **"x (y :: ys)"** e não com o uso de vírgula como a forma **"num # num"**. É importante estar atendo a esses detalhes para o devido uso da passagem de parâmetros junto aos argumentos.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: [1, 1, 2, 2, 3, 3, 4, 4, 5, 5];
>> [1, 1, 2, 2, 3, 3, 4, 4, 5, 5] : list num

>: uniq([1, 1, 2, 2, 3, 3, 4, 4, 5, 5]);
>> [1, 2, 3, 4, 5] : list num
```

Observe o efeito da função **"uniq"** sobre uma lista de valores repetidos.

Além das operações apresentadas pode-se fazer uso de funções que operam com partes chamadas **"take"**, **"drop"** e **"slice"**, codificadas em seguida.

```

dec take : num # list num -> list num;
--- take (_, []) <= [];
--- take (n, x :: xs) <= if n > 0
                        then x :: take (n - 1, xs)
                        else [];

dec drop : num # list num -> list num;
--- drop (_, []) <= [];
--- drop (n, x :: xs) <= if n - 1 > 0 then drop (n - 1, xs) else xs;

dec slice : num # num # list num -> list num;
--- slice (c, f, lst) <= drop(c, take(f, lst));

```

A partir da definição das funções "**take**" e "**drop**" é possível extrair certa quantidade de elementos nas extremidades da lista. A extremidade esquerda ou inicial é calculada pela função "**take**" e a porção final ou a direita é calculada pela função "**drop**".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```

>: take(5, [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]);
>> [1, 2, 3, 4, 5] : list num

>: drop(5, [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]);
>> [6, 7, 8, 9, 0] : list num

>: slice(4, 9, [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]);
>> [5, 6, 7, 8, 9] : list num

```

A função "**take**" é usada quando se deseja obter a quantidade de elementos iniciais de uma lista, a função "**drop**" permite obter os elementos de uma lista a partir de uma posição informada e a função "**slice**" permite fatiar um pedaço de uma lista.

A partir das definições das funções anteriores tem-se um conjunto básico de funções para o gerenciamento de listas similares a outras linguagens de programação funcionais.

5.2 OPERAÇÕES BÁSICAS COM CONJUNTOS

A coleção de funcionalidades apresentadas, neste capítulo, oferece uma série de possibilidades operacionais para o uso de listas, mas não apresentam nenhum recurso para o tratamento de conjuntos do ponto de vista matemático.

Conjunto é em essência uma coleção de elementos que possuem características comuns (contexto matemático). A linguagem **HOPE** proporciona como ferramenta de manipulação de conjuntos os recursos listas e tuplas, sendo listas a forma mais prática de aliar os conceitos matemáticos e computacionais.

Um conjunto, do ponto de vista matemático, é uma coleção de elementos delimitados entre chaves, tendo seus elementos separados por vírgulas associados a uma letra maiúscula. Em **HOPE** o conjunto será representado por uma coleção de valores delimitados entre colchetes (assim como ocorre em outras linguagens de programação com suporte a programação funcional), separados por vírgula e associados por meio de função a uma variável imutável definida com letras minúsculas.

Um conceito importante no estudo da teoria de conjuntos é a relação de pertinência que indica de determinado elemento pertence ou não pertence a certo conjunto. Essa ação não é oferecida diretamente pela linguagem **HOPE**, mas já é possível de ser utilizada a partir da definição anterior da função **"member"** que retorna **"true"** se um elemento existe na lista, ou seja, está contido no conjunto ou **"false"** quando o elemento não está contido.

No estudo da teoria de conjuntos é previsto quatro operações com conjuntos, sendo: união, intersecção, diferença e igualdade entre conjuntos. Para atender, especificamente essas ações será necessário escrever essas funcionalidades. Assim sendo, são definidas as funções **"union"**, **"intersect"**, **"diff"** e **"equality"**.

A união entre os conjuntos **"A"** e **"B"** é formada por todos os elementos pertencentes ao conjunto **"A"** ou ao conjunto **"B"**, ou seja, **"A U B = {x / x ∈ A ou x ∈ B}"**.

```
dec union : list num # list num -> list num;
--- union (a, []) <= a;
--- union ([], b) <= b;
--- union (a, x :: b) <= x :: union (a, b);
```


A função "**union**" considera os argumentos "**a**" e "**b**" respectivamente como a representação dos conjuntos "**A**" e "**B**", a variável "**x**" representa particularmente os elementos existentes nos conjuntos em uso. Se na ação de união um dos conjuntos é indicado como vazio ("**[]**") ocorre o retorno da união indicando apenas o conjunto não vazio. Isso é definido junto às instruções "**union (a, []) <= a;**" e "**union ([], b) <= b;**". A terceira linha de código estabelece recursivamente a ação de união entre os conjuntos indicados. A ação de união entre conjuntos pode ser realizada na linguagem **HOPE** por meio do uso do operador "**<>**" com o formato de instrução "**A <> B**".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: union([1, 2, 3], []);
>> [1, 2, 3] : list num

>: union([], [4, 5, 6]);
>> [4, 5, 6] : list num

>: union([1, 2, 3], [4, 5, 6]);
>> [4, 5, 6, 1, 2, 3] : list num
```

A intersecção entre os conjuntos "**A**" e "**B**" é formada pelos elementos comuns aos conjuntos "**A**" e "**B**", ou seja, " **$A \cap B = \{x / x \in A \text{ e } x \in B\}$** ".

```
dec intersect : list num # list num -> list num;
--- intersect (a, []) <= [];
--- intersect ([], b) <= [];
--- intersect (a, x :: b) <= if member (x, a)
                             then x :: intersect (a, b)
                             else intersect (a, b);
```

A função "**intersect**" considera os argumentos "**a**" e "**b**" respectivamente como a representação dos conjuntos "**A**" e "**B**", a variável "**x**" representa particularmente os elementos existentes nos conjuntos em uso. Se na ação de intersecção um dos conjuntos é indicado como vazio ("**[]**") ocorre o retorno da intersecção indicando vazio. Isso é definido junto às instruções "**intersect (a, []) <= [];**" e "**inter-**

`sect ([], b) <= [];`. A terceira linha de código estabelece recursivamente a ação de intersecção entre os conjuntos indicados. Note que é verificado se o elemento "x" está contido no conjunto "A" ("`member (x, a)`") e sendo esta condição verdadeira o elemento é colocado no conjunto de saída da função e nova recursão é processada ("`x :: intersect (a, b)`"). Caso o elemento "x" não esteja contido no conjunto "A" é solicitada nova recursão ("`intersect (a, b)`") para verificar se há mais elementos que sejam iguais e estejam nos conjuntos "A" e "B".

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: intersect([1, 2, 3], []);
>> nil : list num

>: intersect([], [4, 5, 6]);
>> nil : list num

>: intersect([1, 2, 3, 4], [3, 4, 5, 6]);
>> [3, 4] : list num
```

A diferença entre os conjuntos "A" e "B" é formada pelos elementos que pertencem ao conjunto "A", mas não pertencem ao conjunto "B". Havendo elementos iguais entre os conjuntos estes são desconsiderados. Não são considerados outros elementos que existam no conjunto "B", ou seja, " $A - B = \{x / x \in A \text{ e } x \notin B\}$ ".

```
dec diff : list num # list num -> list num;
--- diff (a, []) <= a;
--- diff ([], b) <= [];
--- diff (a :: x, b) <= if member (a, b)
                        then diff (x, b)
                        else a :: diff (x, b);
```

A função "`diff`" considera os argumentos "a" e "b" respectivamente como a representação dos conjuntos "A" e "B", a variável "x" representa particularmente os elementos existentes nos conjuntos em uso. Se na diferença o conjunto "B" estiver vazio o resultado será a consideração de todos os elementos do conjunto "A", mas estando o conjunto "A" vazio, mesmo tendo o conjunto "B" elementos o resultado

será um conjunto vazio. A terceira linha de código estabelece recursivamente a ação de diferença entre os conjuntos indicados. Note que é verificado se o elemento do conjunto "A" está contido no conjunto "B" ("**member (a, b)**") e sendo esta condição verdadeira a ação recursiva da diferença é realizada entre o elemento "x" e o conjunto "B" repetindo recursivamente a ação anterior ("**diff (x, b)**"). Quando a condição não é verdadeira o elemento avaliado é colocado junto ao conjunto "B" ("**diff (x, b)**").

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: diff([1, 2, 3, 4, 5], []);  
>> [1, 2, 3, 4, 5] : list num  
  
>: diff([], [1, 2]);  
>> nil : list num  
  
>: diff([1, 2, 3, 4, 5], [1, 2]);  
>> [3, 4, 5] : list num  
  
>: diff([1, 2, 3, 4, 5], [1, 2, 6, 7]);  
>> [3, 4, 5] : list num
```

A igualdade entre os conjuntos ocorre quando os elementos de um conjunto são exatamente iguais aos elementos de outros conjuntos independentemente da ordem de disposição dos elementos dentro dos conjuntos, ou seja, "**A = B**". Pelo fato de ser a funcionalidade de detecção de igualdade uma função lógica atente para uso do tipo de dado "**bool**" no lugar do tipo "**truval**" se estiver em uso o ambiente *RP-HOME*.

```
dec equality : list num # list num -> truval;  
--- equality (a, []) <= false;  
--- equality ([], b) <= false;  
--- equality (a :: as, b :: bs) <=  
    if sort (a :: as) = sort (b :: bs) then true else false;
```

A função **"equality"** retorna a resposta **"true"** para dois conjuntos informados se estes possuírem os mesmos elementos não importando a ordem interna dos elementos nos conjuntos. Se indicados conjuntos vazios o retorno será **"false"**. Para que a função **"equality"** possa gerar o devido resultado é importante que os elementos dos conjuntos sejam classificados, pois apenas a verificação condicional de dois conjuntos como **"[1, 2, 3] == [3, 2, 1]"** resulta **"false"**, sendo que do ponto de vista de igualdade de conjuntos o resultado é **"true"**.

Para a realização de alguns testes execute as instruções seguintes observando os resultados gerados após cada ação.

```
>: equality([1, 2, 3], [3, 2, 1]);  
>> true : truval  
  
>: equality([1, 2, 3], [3, 2, 1, 0]);  
>> false : truval
```

A partir dessa coleção de funcionalidades para a manipulação de conjuntos é possível executar algumas operações de tratamento de conjuntos do ponto de vista do estudo da teoria de conjuntos.

5.3 PROCESSAMENTO DE CONJUNTOS

A partir da visão sobre a manipulação de listas e sua relação com conjuntos incluindo-se as ações de união, interseção, diferença e igualdade é possível expandir as ações que podem ser realizadas sobre conjuntos com o desenvolvimento de funcionalidades voltadas para mapeamento, filtragem, redução e transposição de elementos existentes em conjuntos. São apresentadas, neste tópico, as funções **"map"**, **"filter"**, **"reduce"** e **"zip"** usadas com funções de ordem superior.

Parte das ações apresentadas já foi apresentada anteriormente. No entanto, foram apresentadas de forma contextualizada apenas para justificar algumas operações para auxiliar o entendimento da funcionalidade operacional da linguagem **HOPE**.

A partir da visão básica e contextual sobre conjuntos e sua relação com a linguagem **HOPE**, é adequado expandir sua aplicação conceituando-se o significado de função tanto do ponto de vista matemático como do ponto de vista computacional:

- Computacionalmente, uma função, pode ser entendida como sendo uma rotina de programa ou a definição de uma expressão aritmética ou lógica que a partir de um argumento de entrada fornece sempre um valor de saída como resposta de sua ação. Essa definição não é diferente para nenhum paradigma de programação, podendo-se acrescentar que no caso do paradigma declarativo funcional o conceito de função fica mais próximo do contexto matemático.
- Matematicamente, uma função é uma maneira de expressar dois valores existentes em diferentes conjuntos. Desta forma, todos os elementos do primeiro conjunto, chamado domínio, devem ser relacionados aos valores do segundo conjunto, contradomínio. No entanto, pode acontecer de existir no segundo conjunto valores que não estejam relacionados com os elementos do primeiro conjunto.

Algebricamente uma função é definida como $f: A \rightarrow B$, onde "efe" representa a relação existente entre (seta para à direita) os elementos dos conjuntos "A" e "B". A partir dessa relação tem-se a definição de uma regra de operação representada por $y = f(x)$ que será representada computacionalmente como $f(x) = y$.

A partir das definições $f: A \rightarrow B$ e $y = f(x)$ tem-se como domínio o conjunto "A" e como contradomínio o conjunto "B". A indicação *domínio* (conjunto "A") refere-se ao conjunto que possui os valores independentes, os quais determinam a partir da regra algébrica $y = f(x)$ os valores do conjunto *contradomínio* (conjunto "B") formados pelos valores dependentes.

Pode existir no conjunto contradomínio algum valor que não tenha relação com um valor do conjunto domínio, ou seja, não pertencente ao conjunto imagem definido a partir da função existente entre os conjuntos. Assim sendo, o conjunto imagem é por sua natureza um subconjunto do conjunto contradomínio possuindo os valores que correspondem diretamente aos valores do conjunto domínio.

A ação conhecida por *mapeamento* visa aplicar uma função a todos os elementos existentes em um conjunto domínio gerando a partir dessa ação um conjunto contradomínio com o resultado da operação definida como função. Para esta ação é definida a função "map" indicada em seguida.

```
dec map : list num # (num -> num) -> list num;
--- map ([], func) <= [];
--- map (x :: lst, func) <= func x :: map (lst, func);
```

É importante relembrar e ressaltar que o protótipo da função **"map"** indica o uso dos argumentos de entrada lista e função para gerar como resposta uma função. Neste sentido o primeiro argumento estabelece o conjunto domínio, o segundo argumento estabelece a função e o terceiro argumento, argumento de retorno, estabelece o conjunto contradomínio com a imagem da função utilizada no mapeamento.

A função **"map"** retorna como resposta um conjunto vazio se um conjunto vazio se fornecido como argumento. Para esta versão a função **"map"** aceita a recepção de um argumento com função que use um argumento de entrada e um de saída identificado como **"(num -> num)"**.

A figura 5.1 mostra estruturalmente como uma função de mapeamento opera sua ação sobre os conjuntos **"A"** (domínio) e **"B"** (contradomínio) envolvidos a partir da função **$f(x) = x3$** .

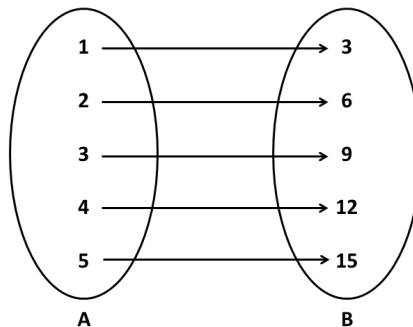


Figura 5.1 – Conjuntos domínio e contradomínio da função $f(x) = x3$.

No sentido de corroborar a imagem da figura 5.1 execute as instruções seguintes, observando o resultado gerado pela ação.

```
>: map([1, 2, 3, 4, 5], \ x => x * 3);  
>> [3, 6, 9, 12, 15] : list num  
  
>: map([1, 2, 3, 4, 5], lambda x => x * 3);  
>> [3, 6, 9, 12, 15] : list num
```

Apesar de ter sido brevemente comentado cabe aqui ressaltar o uso do símbolo **"\"** como sinônimo de uso do comando **"lambda"**.

A função "**filter**" tem definido com o primeiro argumento a função que aplicará ação lógica sobre o conjunto indicado como segundo argumento. O terceiro argumento indica o retorno de um conjunto criado a partir da ação de filtragem definida no primeiro argumento.

Note que a função "**filter**" retorna um conjunto vazio se um conjunto vazio for fornecido como argumento. Sendo informado um conjunto com elementos com a definição de uma função ocorre a verificação lógica de sua ação ("**if func x**") que sendo verdadeira acrescenta o elemento "**x**" a recursão da filtragem. Se condição for falsa ocorre nova filtragem a partir do próximo elemento.

A figura 5.2 mostra estruturalmente como uma função de filtragem opera sua ação sobre o conjunto indicado a partir da função $f(x) = x - 2 \lfloor x / 2 \rfloor \mid x \sim= 0$.

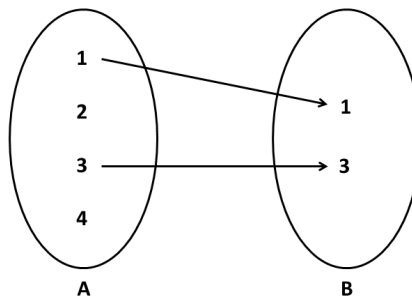


Figura 5.2 – Filtragem com função $f(x) = x - 2 \lfloor x / 2 \rfloor \mid x \sim= 0$.

No sentido de corroborar a imagem da figura 5.2 execute as instruções seguintes, observando o resultado gerado pela ação (todos os valores que não sejam pares).

```
>: filter(\ x => x mod 2 /= 0, [1, 2, 3, 4]);
>> [1, 3] : list num
```

Tomando por base o conjunto domínio $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ para obter o conjunto imagem a partir da filtragem formada pelos valores do domínio que sejam pares. Observe as seguintes instruções.

```
>: filter(\ x => x mod 2 = 0, range(1, 10, 1));
>> [2, 4, 6, 8, 10] : list num
```


Veja alguns outros exemplos de filtragem.

```
>: filter(\ x => x > 5, range(1, 10, 2));  
>> [7, 9] : list num  
  
>: filter(\ x => x > 2 and x < 8, range(1, 10, 1));  
>> [3, 4, 5, 6, 7] : list num  
  
>: filter(\ x => 10 mod x /= 0, range(1, 10, 1));  
>> [3, 4, 6, 7, 8, 9] : list num
```

A ação conhecida por **redução** visa aplicar uma função a todos os elementos existentes em um conjunto domínio gerando a partir dessa ação um único valor como resposta. Uma redução recebe dois argumentos a partir dos dois primeiros elementos de um conjunto e aplica essa ação iterativamente aos próximos elementos usando a função de auxilio indicada. O resultado dos dois elementos anteriores é aplicado iterativamente ao próximo elemento até o final do conjunto. O resultado apresentado será a redução aplicada a todo o conjunto.

Nas operações de redução os conjuntos imagem e contradomínio são diferentes. O conjunto domínio não pode ser um conjunto vazio, mas o conjunto contradomínio poderá ter um único elemento ou ser um conjunto vazio, sendo definido a partir das instruções. Para esta ação é definida a função "**reduce**" indicada em seguida.

```
dec reduce : list num # (num # num -> num) # num -> num;  
--- reduce ([], func, st) <= st;  
--- reduce (x :: xs, func, st) <= func (x, reduce (xs, func, st));
```

O protótipo da função "**reduce**" indica a recepção do primeiro argumento definido como conjunto de entrada, o segundo argumento marca o uso de uma função que opere com dois argumentos de entrada e um argumento de saída (função de redução), o terceiro argumento é usado para definir um valor inicial para a ação de redução e o último argumento caracteriza-se por ser a resposta da ação da função "**reduce**".

Se for informado um conjunto vazio para função "**reduce**" será dado como retorno o valor do argumento "**st**" (argumento **start**). O argumento "**st**" é usado para auxiliar a ação da função definida junto ao segundo argumento.

Observe que a ação de redução fica evidenciada na terceira linha de código da função onde a função passada como argumento atua sobre cada elemento do conjunto como indicado no trecho "**func (x, reduce (xs, func, st))**".

A figura 5.3 mostra estruturalmente como uma função de redução opera sua ação sobre o conjunto indicado a partir da função $f(x) = \sum x$.

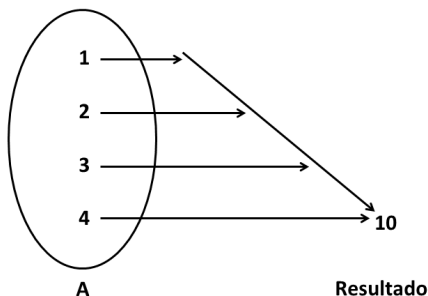


Figura 5.3 – Filtragem de conjunto com função $f(x) = \sum x$.

No sentido de corroborar a imagem da figura 5.3 execute as instruções seguintes, observando o resultado gerado pela ação.

```
>: reduce([1, 2, 3, 4], add, 0);  
>> 10 : num
```

A operação de adição executada pela função "**add**" foi desenvolvida anteriormente nesta obra e faz parte do arquivo de módulo "**math.hop**". Note que a função "**add**" é processada elemento a elemento do conjunto, neste caso "**[1, 2, 3, 4]**", tendo como valor inicial para o processamento a indicação "**0**".

A função "**reduce**" apresentada aceita o uso de qualquer função que opere com dois argumentos. Veja os próximos exemplos.

```
>: reduce([1, 2, 3, 4], mult, 1);  
>> 24 : num
```

```
>: reduce([1, 2, 3, 4], sub, 0);  
>> -2 : num
```

```
>: reduce([1, 2, 3, 4], divf, 1);  
>> 0.375 : num  
  
>: reduce([1, 2, 3, 4], max, 1);  
>> 4 : num  
  
>: reduce([1, 2, 3, 4], min, 1);  
>> 1 : num  
  
>: reduce(reverse([1, 2, 3, 4]), pow, 1);  
>> 262144 : num
```

A linguagem *HOPE* possui um recurso que pode ser útil em algumas operações funcionais, destacando-se ações nas operações de redução. Trata-se do comando "**nonop**" que pode ser usado para definir um operador da linguagem como se fosse uma função de primeira ordem. O comando "**nonop**" diz que o operador em uso não é um operador, e sim uma função. Veja alguns exemplos.

```
>: reduce([1, 2, 3, 4], nonop *, 1);  
>> 24 : num  
  
>: reduce([1, 2, 3, 4], nonop -, 0);  
>> -2 : num  
  
>: reduce([1, 2, 3, 4], nonop /, 1);  
>> 0.375 : num  
  
>: reduce([1, 2, 3, 4], nonop +, 0);  
>> 10 : num  
  
>: reduce(reverse([1, 2, 3, 4]), nonop **, 1);  
>> 262144 : num
```

A ação conhecida por *transposição* visa criar um conjunto formado com os elementos intercalados de outros dois conjuntos. A transposição ocorrerá com a jun-

ção do primeiro valor do primeiro conjunto com o primeiro valor do segundo conjunto, depois é realizada a junção do segundo elementos do primeiro conjunto com o segundo elemento do segundo conjunto e assim por diante até que algum dos elementos de algum dos conjuntos acabe. Se os conjuntos a serem transpostos forem de tamanho diferentes a transposição usará sempre como base o conjunto com menor número de elementos.

Nas operações de transposição, normalmente, os conjuntos envolvidos na operação são idênticos. Para esta ação é definida a função "**zip**" indicada em seguida.

```
dec zip : list num # list num -> list (num # num);  
--- zip ([], b) <= [];  
--- zip (a, []) <= [];  
--- zip ((x :: a), (y :: b)) <= (x, y) :: (zip (a, b));
```

Na efetivação da ação de transposição se algum dos conjuntos for vazio o resultado será um conjunto vazio. Se os conjuntos tiverem elementos a transposição fará a distribuição intercalada dos elementos de cada conjunto em um único conjunto a partir do trecho "**(x, y) :: (zip (a, b))**".

A figura 5.4 mostra estruturalmente como uma função de transposição opera.

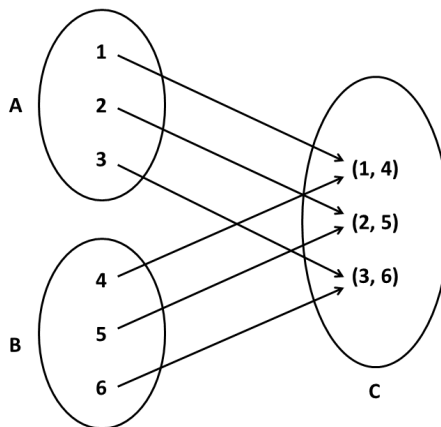


Figura 5.4 – Transposição de conjuntos.

No sentido de corroborar a imagem da figura 5.4 execute as instruções seguintes, observando o resultado gerado pela ação.

```
>: zip([1, 2, 3], [4, 5, 6]);  
>> [(1, 4), (2, 5), (3, 6)] : list (num # num)
```

Note que a operação de transposição faz a distribuição intercalada dos elementos um a um combinando-os em um conjunto resultante da ação.

Veja alguns outros exemplos de transposição.

```
>: zip([1, 2, 3], [4, 6]);  
>> [(1, 4), (2, 6)] : list (num # num)  
  
>: zip([1, 2, 3], [4, 6, 5, 7]);  
>> [(1, 4), (2, 6), (3, 5)] : list (num # num)
```

Os exemplos anteriores mostram os resultados que são obtidos quando se opera com conjuntos de tamanho diferentes.

5.4 RELAÇÕES DE INCLUSÃO

As principais operações com uso de conjuntos são a união, a intersecção e a diferença já apresentados. Outro conceito importante no estudo da teoria de conjuntos é a verificação se um conjunto está ou não contido em outro conjunto por meio da definição da relação de inclusão.

Assim como as demais operações sobre conjuntos a linguagem **HOPE** não possui internamente uma função que realiza esta tarefa. Desta forma, é necessário considerar o desenvolvimento de uma função chamada "**subset**". Atente para uso do tipo de dado "**bool**" no lugar do tipo "**truval**" se estiver em uso o ambiente **RP-HOME**.

```

dec subset' : list num # list num -> truval;
--- subset' (a, b) <= true;
--- subset' (x :: y, b) <= if member (x, b)
                           then subset' (y, b)
                           else false;

dec subset : list num # list num -> truval;
--- subset ([], b) <= false;
--- subset (a, []) <= false;
--- subset ([], []) <= true;
--- subset (a, b) <= subset' (a, b);

```

A função "**subset**" necessita validar inicialmente as condições das ações: *contido* e *não contido*. Observe que as três primeiras linhas ajustam as condições dessa validação. Se nenhuma dessas condições iniciais ocorre a quarta linha faz a chamada da função auxiliar "**subset'**" que efetua o restante da ação.

As condições de validação inicial são definidas separadamente por não poderem ser usadas em conjunto com a validação inicial da função "**subset'**".

apresentada é uma solução preguiçosa, pois apesar de realizar sua principal operação não leva em consideração a verificação e validação de uso de conjuntos vazios.

Observe as instruções a seguir.

```

>: subset([1, 2, 3], [1, 2, 3, 4, 5]);
>> true : truval

>: subset([1, 2, 6], [1, 2, 3, 4, 5]);
>> false : truval

>: subset([], []);
>> false : truval

```

```
>: subset([1, 2], []);  
>> false : truval  
  
>: subset([], [1, 2]);  
>> false : truval  
  
>: subset([1,3], [1,2,3,4,5]);  
>> true : truval  
  
>: subset([1, 2, 3, 4, 5], [1, 2, 3]);  
>> false : truval
```

Dos exemplos indicados o primeiro mostra que o conjunto "[1, 2, 3]" está contido no conjunto "[1, 2, 3, 4, 5]", o segundo exemplo apresenta que o conjunto "[1, 2, 6]" não está contido no conjunto "[1, 2, 3, 4, 5]" e o terceiro exemplo indica que o conjunto "[1, 2, 3, 4, 5]" não está contido no conjunto "[1, 2, 3]".

5.5 MÓDULO MYLIST.HOP

Ao longo deste capítulo foram apresentadas diversas funções para o gerenciamento de listas na forma de conjuntos. Desta forma, este conjunto de operações pode ser definido em um arquivo de módulo chamado "**mylist.hop**".

O arquivo de módulo "**mylist.hop**" deve conter as funcionalidades essenciais para as ações apresentadas neste capítulo. Para o interpretador **RP-HOPE** troque o tipo de dado "**truval**" por "**bool**".

O código seguinte mostra a definição do arquivo de módulo "**mylist.hop**" com seção a privada contendo as funcionalidades.

```
!! Módulo .....: mylist.hop  
!! Finalidade ...: Funções matemática gerais.  
!! Autor .....: Augusto Manzano  
!! Versão .....: 1.0  
!! Data .....: 05/07/2018
```

```
uses math;

dec diff      : list num # list num -> list num;
dec drop      : num # list num -> list num;
dec equality   : list num # list num -> truval;
dec filter    : (num -> truval) # list num -> list num;
dec find      : num # list num -> num;
dec first     : list num -> num;
dec head      : list num -> num;
dec insert    : num # list num -> list num;
dec intersect : list num # list num -> list num;
dec last      : list num -> num;
dec length    : list num -> num;
dec list_max  : list num -> num;
dec list_max_min : list num -> list num;
dec list_min  : list num -> num;
dec list_min_max : list num -> list num;
dec map       : list num # (num -> num) -> list num;
dec member    : num # list num -> truval;
dec range     : num # num # num -> list num;
dec reduce    : list num # (num # num -> num) # num -> num;
dec remove    : num # list num -> list num;
dec replace   : list num # num # num # num -> list num;
dec replicate : num # num -> list num;
dec reverse   : list num -> list num;
dec show      : list num # num -> num;
dec slice     : num # num # list num -> list num;
dec sort      : list num -> list num;
dec subset    : list num # list num -> truval;
dec tail      : list num -> list num;
dec take      : num # list num -> list num;
dec union     : list num # list num -> list num;
dec uniq      : list num -> list num;
```



```
dec zip          : list num # list num -> list (num # num);

private;

--- insert (a, nil) <= a :: nil;
--- insert (a, x :: xs) <= if a < x
                           then a :: x :: xs
                           else x :: insert (a, xs);

--- range (s, f, i) <= if s > f
                       then []
                       else s :: range (s + 1 + (i - 1), f, i);

--- reverse [] <= [];
--- reverse (x :: xs) <= reverse xs <> [x];

--- remove (a, []) <= [];
--- remove (a, x :: xs) <= if a = x
                           then xs
                           else x :: remove (a, xs);
--- head (x :: xs) <= x;

--- tail (x :: xs) <= xs;

--- first (x :: xs) <= x;

--- last [x] <= x;

--- last (x :: xs) <= last xs;

--- length [] <= 0;
--- length (x :: xs) <= succ (length xs);
```

```
--- member (_, nil) <= false;
--- member (a, x :: xs) <= if a = x
                             then true
                             else member (a, xs);
--- find (a, []) <= if a > length []
                    then error "valor muito alto"
                    else error "valor muito baixo";
--- find (a, x :: xs) <= if a = x
                        then 0
                        else find (a, xs) + 1;

--- show ([], a) <= if a >= length []
                   then error "indice muito alto"
                   else error "indice muito baixo";
--- show (x :: xs, 0) <= x;
--- show (x :: xs, a) <= show (xs, a - 1);

--- list_max [] <= error "sem maior valor: lista vazia";
--- list_max ([a]) <= a;
--- list_max (x :: xs) <= if x > list_max xs
                          then x
                          else list_max xs;

--- list_min [] <= error "sem menor valor: lista vazia";
--- list_min ([a]) <= a;
--- list_min (x :: xs) <= if x < list_min xs
                          then x
                          else list_min xs;

--- list_min_max [] <= error "sem menor e maior valor: lista vazia";
--- list_min_max (x :: xs) <=
    [list_min (x :: xs), list_max (x :: xs)];
```

```
--- list_max_min [] <= error "sem maior e menor valor: lista vazia";
--- list_max_min (x :: xs) <=
    [list_max (x :: xs), list_min (x :: xs)];

--- replace (lst, p, e, c) <= lst;
--- replace (x :: xs, p, e, c) <= if c = p
    then e :: (replace(xs, p, e, c + 1))
    else x :: (replace(xs, p, e, c + 1));

--- replicate (a, r) <= if a <= 0
    then []
    else r :: replicate (a - 1, r);

dec split : list num -> list num # list num;
--- split [] <= ([], []);
--- split (x :: xs) <= (x :: zs, ys)
    where (ys, zs) == split xs;

dec sort' : list num # list num -> list num;
--- sort' ([], ys) <= ys;
--- sort' (xs, []) <= xs;
--- sort' (x :: xs, y :: ys) <= if x < y
    then x :: sort' (xs, y :: ys)
    else y :: sort' (x :: xs, ys);

--- sort [] <= [];
--- sort [x] <= [x];
--- sort(x1 :: x2 :: xs) <= sort' (sort (x1 :: ys), sort (x2 :: zs))
    where (ys, zs) == split xs;
```

```
dec uniq' : num -> list num -> list num;
--- uniq' x [] <= [];
--- uniq' x (y :: ys) <= if x = y
                        then uniq' x ys
                        else y :: uniq' y ys;

--- uniq [] <= [];
--- uniq (x :: xs) <= x :: uniq' x xs;

--- take (_, []) <= [];
--- take (n, x :: xs) <= if n > 0
                        then x :: take (n - 1, xs)
                        else [];

--- drop (_, []) <= [];
--- drop (n, x :: xs) <= if n - 1 > 0 then drop (n - 1, xs) else xs;

--- slice (c, f, lst) <= drop(c, take(f, lst));

--- union (a, []) <= a;
--- union ([], b) <= b;
--- union (a, x :: b) <= x :: union (a, b);

--- intersect (a, []) <= [];
--- intersect ([], b) <= [];
--- intersect (a, x :: b) <= if member (x, a)
                        then x :: intersect (a, b)
                        else intersect (a, b);
```

```
--- diff (a, []) <= a;
--- diff ([], b) <= [];
--- diff (a :: x, b) <= if member (a, b)
                        then diff (x, b)
                        else a :: diff (x, b);

--- equality (a, []) <= false;
--- equality ([], b) <= false;
--- equality (a :: as, b :: bs) <=
    if sort (a :: as) = sort (b :: bs) then true else false;

--- map ([], func) <= [];
--- map (x :: lst, func) <= func x :: map (lst, func);

--- filter (func, []) <= [];
--- filter (func, x :: lst) <= if func x
                              then x :: filter (func, lst)
                              else filter (func, lst);

--- reduce ([], func, st) <= st;
--- reduce (x :: xs, func, st) <= func (x, reduce (xs, func, st));

--- zip ([], b) <= [];
--- zip (a, []) <= [];
--- zip ((x :: a), (y :: b)) <= (x, y) :: (zip (a, b));

dec subset' : list num # list num -> truval;
--- subset' (a, b) <= true;
--- subset' (x :: y, b) <= if member (x, b)
                          then subset' (y, b)
                          else false;
```

```
--- subset ([], b) <= false;  
--- subset (a, []) <= false;  
--- subset ([], []) <= true;  
--- subset (a, b) <= subset' (a, b);
```

Note que uma das primeiras ações efetuadas no arquivo de módulo "**mylist.hop**" é o carregamento do arquivo de módulo "**math.hop**". Desta forma, os recursos definidos no módulo "**math**" são inicialmente e automaticamente carregados com a chamada do módulo "**mylist**".

6

Programação complementar

Este capítulo apresenta recursos que podem ser incorporados a arquivos de módulos e recursos existentes nos arquivos de módulo do interpretador RP-HOME, além de outros assuntos que não foram anteriormente abordados.

6.1 CURRIFICAÇÃO

A programação funcional prevê o desenvolvimento de programas fundamentados sobre funções. O uso de funções pode tornar a programação muito verbosa na medida em que há a necessidade de passar diversos argumentos a uma função podendo tornar a estrutura de uso complexa.

A ação de currficação (*currying*) visa estabelecer a uma função que possui aridade "N" a possibilidade de fazer uso de argumentos com aridade menor. A técnica foi desenvolvida por Gottlob Frege em 1893 e Moses Schönfinkel na década de 1920 e ampliada por Haskell Curry durante a década de 1960, tornando a técnica mais conhecida e dando origem ao termo *currying*.

A ação de *currying* ocorre quando se divide uma função que utiliza um número grande de argumentos em uma série de funções que fazem parte dos argumentos das funções divididas.

No caso particular da linguagem *HOPE* a currficação não é diretamente definida, mas pode ser implementada a partir de uma função auxiliar que executa a currficação sobre uma função não currficada. É importante levar em consideração a aridade da função a ser currficada, desta forma é necessário definir funções currficadas diferentes para aridades diferentes, excetuando-se as funções de uma aridade que não necessitam ser currficadas.

Como exemplo considere a função "**soma**" que retorna o resultado da soma de dois argumentos inteiros indicados como entrada codificada em seguida.

```
dec soma : num # num -> num;
--- soma (a, b) <= a + b;
```

A função "**soma**" recebe dois argumentos como entrada (aridade 2) e retorna como resposta a soma dos argumentos fornecidos como entrada. Assim sendo, o uso da função "**soma(1, 2)**" retorna do resultado "**3**". Para ação currificada a função pode ser usada como "**soma 1 2**" para retornar "**3**".

Para fazer um teste sobre currficação considere a definição do código da função "**soma**" anterior e em seguida a codificação da função "**@@**".

```
dec @@ : (num # num -> num) -> num -> num -> num;
--- @@ func <= \ x => \ y => func (x, y);
```

A partir das definições das funções "**soma**" e "**@@**" execute a instrução.

```
>: @@ soma 1 2;
>> 3 : num
```

Se executada a instrução.

```
>: soma 1 2;
soma 1
soma : num # num -> num
1 : num
type error - argument has wrong type
```

Ocorrerá a apresentação da mensagem indicando o erro "**type error - argument has wrong type**", o qual informa erro no tipo do argumento.

Este erro ocorre uma vez que a função "**soma**" dos dois argumentos passados pega apenas o primeiro argumento "**soma 1**" e o segundo argumento é desconsiderado. A indicação de "**soma : num # num -> num**" informa a estrutura de argumentos da função. A indicação "**1 : num**" mostra o resultado gerado pela execução da função.

A função currificada "@@" exige que o uso da função "soma" seja realizado sem o uso de parênteses (antes obrigatórios), simplificando o uso. Neste caso, se os parênteses forem definidos para a função "soma" ocorrerá erro no tipo de argumento informado.

Para realizar este recurso, a função "@@" faz uso de três argumentos de entrada, sendo o primeiro argumento a recepção de uma função de primeira ordem delimitada entre parênteses com aridade 2 e um retorno simples "(num # num -> num)". O argumento "(num # num -> num)" permite que seja usado como argumento uma função que opere com aridade 2 e retorno de valor simples.

O segundo e terceiros argumentos da função currificada "-> num -> num" são usados para receber sem o uso de vírgulas os dois valores indicados para a função a ser processada. O último argumento da função currificada caracteriza-se por ser o valor de retorno da função "@@".

Na definição da função "@@" encontra-se o uso de duas funções lambda e a recepção da função passada como argumento "func <= \ x => \ y => func (x, y)", onde "func <=" representa o retorno da função passada como argumento, o trecho "\ x =>" representa a ação do segundo argumento e o trecho "\ y =>" representa o terceiro argumento. O trecho "func (x, y)" é usado para fundamentar a estrutura da função passada.

O uso da instrução "@@ soma 1 2" passa primeiramente para a função "@@" o valor "soma 1 2" que é recebido pelo argumento "(num # num -> num)", o segundo valor "1" e o terceiro valor "2" são passados para os argumento "-> num -> num", de modo que a função "@@" obtenha os argumentos necessários para sua operação.

A função "@@" pode ser usada para a ação de qualquer função com aridade 2. Observe a instrução seguinte para calcular a potência da base "2" elevado ao índice "3".

```
>: @@ pow 2 3;  
>> 8 : num
```

O uso do símbolo "@" é escolhido pelo fato deste símbolo não ser usado para nenhuma outra operação da linguagem *HOPE*.

Caso queira realizar ações currificadas com funções de aridade diferente de 2 é necessário considerar uma estrutura adequada para um número maior de aridade. Para tanto, considere uma função chamada "@@@@" que simplifique o uso da função

"**soma_quad**" que apresenta o resultado da soma dos quadrados de três valores fornecidos.

Codifique primeiro a função "**soma_quad**".

```
dec soma_quad : num # num # num -> num;  
--- soma_quad (a, b, c) <= pow(a, 2) + pow(b, 2) + pow(c, 2);
```

Na sequência codifique a função "**@@@**".

```
dec @@@ : (num # num # num -> num) -> num -> num -> num -> num;  
--- @@@ func <= \ x => \ y => \ z => func (x, y, z);
```

A partir das definições das funções "**soma_quad**" e "**@@@**" execute a instrução.

```
>: @@@ soma_quad 1 2 3;  
>> 14 : num
```

A partir das duas propostas apresentadas torna-se mais fácil criar estruturas curri-ficadas para diversas outras funções. Sugere-se incluir as funções "**@@**" e "**@@@**" no arquivo de módulo "**math.hop**".

6.2 FUNÇÕES COMPLEMENTARES

Neste tópico são apresentadas algumas funções simples que podem ser úteis e assim auxiliar algumas tarefas essenciais com a linguagem.

Uma das funções mais simples e úteis é a que apresente um valor numérico na forma percentual. Assim sendo, observe o código seguinte que poderá ser inserido no arquivo de módulo "**math.hop**".

```
dec percent : num -> num;  
--- percent n <= n / 100;
```

Execute a instrução.

```
>: percent 45;  
>> 0.45 : num
```

Para ações relacionadas ao uso de listas podem ser consideradas mais alguns exemplares de ação como definir par de valores, distribuição de um valor constante com os demais elementos de uma lista existente, extrair o número de elementos de uma lista a partir de seu início e gerar lista com valores pseudo aleatórios.

Para definir uma função que crie um par de valores na forma de tupla considere o código seguinte que poderá ser inserido no arquivo de módulo **"mylist.hop"**.

```
dec mkpair : num # num -> num # num;  
--- mkpair (x, y) <= (x, y);
```

Execute a instrução.

```
>: mkpair(1, 2);  
>> (1, 2) : num # num
```

Para definir a distribuição de um valor constante em uma lista considere o código seguinte que poderá ser inserido no arquivo de módulo **"mylist.hop"**.

```
dec dist : num # list num -> list (num # num);  
--- dist (x, []) <= [];  
--- dist (x, y :: ys) <= (x, y) :: dist (x, ys);
```

Execute a instrução.

```
>: dist(1, [2, 4, 6, 8]);  
>> [(1, 2), (1, 4), (1, 6), (1, 8)] : list (num # num)
```

Para definir a criação de uma lista com valores pseudo aleatórios (forma muito simples) adaptado de Maheshwari que poderá ser inserido no arquivo de módulo **"math.hop"**.

```
dec rand : num -> list num;  
--- rand n <= if n = 0  
    then []  
    else n * 115279 mod 3581 :: rand (n - 1);
```

Execute a instrução.

```
>: rand 7;  
>> [1228, 541, 3435, 2748, 2061, 1374, 687] : list num
```

A função "**rand**" proposta gera sempre os mesmos valores para a mesma quantidade de elementos definidos. Não há na linguagem **HOPE**, a partir dos interpretadores usados, até onde foi possível chegar uma maneira de simular a geração de números aleatórios com valores diferentes para uma mesma quantidade fornecida.

6.3 MANIPULAÇÃO DE CARACTERES

Diversas linguagens de programação pertencentes aos paradigmas imperativos e declarativos possuem um conjunto de funcionalidades voltadas ao tratamento de caracteres ASCII, menos a linguagem **HOPE**. No entanto, o conjunto de arquivos de módulo do interpretador **RP-HOPE** possui definido o módulo "**ctype.hop**" escrito por Ross Paterson contendo algumas funcionalidades para o tratamento isolado de caracteres. O arquivo de módulo "**ctype.hop**" pode ser usado no interpretador **MA-HOPE** desde que o tipo de dado "**bool**" seja alterado para o tipo "**truval**".

A seguir são apresentadas funções adaptadas das funções existentes no arquivo de módulo "**ctype.hop**", atendendo a didática utilizada neste trabalho. Do conjunto de funções existentes duas recebem como argumento de entrada um caractere e retornam um caractere, as demais funções recebem como argumento de entrada um caractere e retornam uma resposta lógica: "**bool**" no interpretador **RP-HOPE** e "**truval**" no interpretador **MA-HOPE**. O conjunto apresentado está focado para o interpretador **MA-HOPE**.

O primeiro grupo de funções relaciona-se as funcionalidades que verificam se o caractere informado é maiúsculo "**isupper**" ou minúsculo "**islower**".

```
dec isupper, islower : char -> truval;  
--- isupper c <= c >= 'A' and c <= 'Z';  
--- islower c <= c >= 'a' and c <= 'z';
```

Observe as instruções de teste.

```
>: isupper 'a';  
>> false : truval
```

```
>: isupper 'A';  
>> true : truval
```

```
>: islower 'a';  
>> true : truval
```

```
>: islower 'A';  
>> false : truval
```

A partir da verificação do formato do caractere alfabético é possível fazer uso de funcionalidades que transformam caracteres alfabéticos em formato maiúsculo e minúsculo com as funcionalidades "**tolower**" e "**toupper**".

```
dec tolower, toupper : char -> char;  
--- tolower c <= if isupper c then chr(ord c + 32) else c;  
--- toupper c <= if islower c then chr(ord c - 32) else c;
```

Observe as instruções de teste.

```
>: toupper 'a';  
>> 'A' : char
```

```
>: tolower 'A';  
>> 'a' : char
```

As funcionalidades "**isupper**" e "**islower**" verificam se o caractere do argumento é alfabético e são bases para as funcionalidades "**tolower**" e "**toupper**" que utilizam o valor "**32**" que é a diferença entre caracteres maiúsculos e minúsculos na tabela ASCII, que pode ser verificada com a função "**isasci**".

```
dec isasci : char -> truval;  
--- isasci c <= ord c < 128;
```

Observe as instruções de teste.

```
>: isascii 'a';  
>> true : truval
```

```
>: isascii 'A';  
>> true : truval
```

```
>: isascii '△';  
>> true : truval
```

```
>: isascii 'é';  
>> false : truval
```

Os caracteres "△" e "é" podem ser gerados respectivamente com o uso das teclas de atalho "<Alt> + <1> + <2> + <7>" e "<Alt> + <1> + <3> + <0>", onde deve ser usada a tecla "<Alt>" esquerda e o teclado numérico.

O próximo conjunto de funções é usada para verificar se os caracteres informados são alfabéticos, numéricos e alfanuméricos.

```
dec isalpha, isdigit, isalnum : char -> truval;  
--- isalpha c <= isupper c or islower c;  
--- isdigit c <= c >= '0' and c <= '9';  
--- isalnum c <= isalpha c or isdigit c;
```

Observe as instruções de teste.

```
>: isalpha 'A';  
>> true : truval
```

```
>: isalpha '1';  
>> false : truval
```

```
>: isdigit 'A';  
>> false : truval
```

```
>: isdigit '1';
>> true : truval

>: isalnum 'A';
>> true : truval

>: isalnum '1';
>> true : truval

>: isalnum '.';
>> false : truval
```

A partir da identificação de caracteres alfabéticos, numéricos e alfanuméricos, cabe considerar funcionalidades para a identificação de uso de caracteres imprimíveis ou não.

```
dec isspace, ispunct, isctrl, isgraph : char -> truval;
--- isspace c <= c = ' ' or c = '\t' or c = '\n';
--- isgraph c <= ' ' =< c and c =< '~';
--- ispunct c <= isgraph c and c /= ' ' and not (isalnum c);
--- isctrl c <= isascii c and not (isgraph c);
```

Observe as instruções de teste.

```
>: isspace ' ';
>> true : truval

>: isspace 'A';
>> false : truval

>: ispunct '.';
>> true : truval

>: ispunct 'A';
>> false : truval
```

```
>: isgraph 'A';
>> true : truval

>: isgraph ' ';
>> true : truval

>: isgraph '~';
>> true : truval

>: isgraph 'Ç';
>> false : truval

>: isctrl 'A';
>> false : truval

>: isctrl '^X';
>> true : truval
```

A função "**isspace**" é usada para detectar se o caractere informado é um espaço em branco e a função "**ispunct**" detecta se o caractere em uso é um símbolo de ponto. Tanto o espaço em branco como qualquer símbolo de pontuação não é considerado um caractere alfanumérico.

A função "**isgraph**" é usada para detectar se o caractere analisado é ou não imprimível do ponto de vista da tabela ASCII e a função "**isctrl**" é usada para verificar se o caractere em uso é um caractere que utilize a ação da tecla "**<Ctrl>**" em conjunto com um caractere alfabético.

Falta apenas considerar a função "**isxdigit**" que tem por finalidade detectar se o caractere indicado é ou não um valor hexadecimal.

```
dec isxdigit : char -> truval;
--- isxdigit c <=
    isdigit c or toupper c >= 'A' and toupper c <= 'F';
```


Observe as instruções de teste.

```
>: isxdigit 'a';  
>> true : truval  
  
>: isxdigit 'A';  
>> true : truval  
  
>: isxdigit 'b';  
>> true : truval  
  
>: isxdigit 'e';  
>> true : truval  
  
>: isxdigit 'f';  
>> true : truval  
  
>: isxdigit 'g';  
>> false : truval  
  
>: isxdigit '6';  
>> true : truval
```

Caso deseje manter a versão apresentada das funcionalidades de manipulação de caracteres como arquivo de módulo execute a instrução "**save mychars;**".

6.4 MANIPULAÇÃO DE CADEIAS

Uma cadeia (*string*) caracteriza-se por ser um conjunto composto de caracteres alfanuméricos, alfabéticos ou numéricos (não usados de forma matemática) delimitados entre aspas inglesas. O tratamento dispensado no capítulo 5 sobre a manipulação de listas de dados numéricos se aplica a dados do tipo cadeia e são em essência muito parecidos.

A linguagem *HOPE*, como para outros recursos operacionais computacionais, não fornece um módulo com funcionalidades para esse tratamento de dados. Os únicos recursos encontrados são as funcionalidades "words" e "unwords" do arquivo de módulo "words.hop" que possui dependência dos arquivos de módulos "ctype" e "list" que quando executadas no *RP-HOPE* geram as seguintes saídas.

```
>: words("Teste de escrita com HOPE");
>> ["Teste", "de", "escrita", "com", "HOPE"] : list (list char)

>: unwords(["Teste", "de", "escrita", "com", "HOPE"]);
>> "Teste de escrita com HOPE" : list char
```

Para fazer uso do arquivo de módulo no interpretador *MA-HOPE* é necessário mudar o tipo de dado "bool" para "truval".

Para a manipulação de sequências de dados gerais é necessário usar como apoio técnicas de mapeamento, filtragem e redução. Para a manipulação de dados baseados em formato alfanumérico as técnicas de mapeamento e filtragem serão utilizadas como suporte as ações que deverão ser processadas em toda a sequência de caracteres existente no *string*.

Para a manipulação de caracteres alfanuméricos são, na maior parte das vezes, utilizadas as técnicas de mapeamento e filtragem. Desta forma, considere as funcionalidades base seguintes.

```
dec str_map : list char # (char -> char) -> list char;
--- str_map (nil, func) <= nil;
--- str_map (i :: f, func) <= func i :: str_map (f, func);

dec str_filt : (char -> truval) # list char -> list char;
--- str_filt (func, []) <= [];
--- str_filt (func, x :: lst) <= if func x
                                then x :: str_filt (func, lst)
                                else str_filt (func, lst);
```

A partir da definição da função "str_map" é possível desenvolver uma série de funcionalidades com manipulação de sequências de caracteres. No entanto, para

que isso seja possível é necessário estar com o arquivo de módulo `"mychar.hop"` ou `"ctype.hop"` carregados em memória a partir do comando `"uses"`.

A funcionalidade `"strupper"` é responsável por apresentar uma sequência de caracteres alfabéticos em formato maiúsculo. Observe o código seguinte.

```
dec strupper : list char -> list char;  
--- strupper s <= str_map(s, toupper);
```

Execute a instrução.

```
>: strupper("programacao de computadores");  
>> "PROGRAMACAO DE COMPUTADORES" : list char
```

A funcionalidade `"strlower"` é responsável por apresentar uma sequência de caracteres alfabéticos em formato minúsculo. Observe o código seguinte.

```
dec strlower : list char -> list char;  
--- strlower s <= str_map(s, tolower);
```

Execute a instrução.

```
>: strlower("PROGRAMACAO DE COMPUTADORES");  
>> "programacao de computadores" : list char
```

A funcionalidade `"strcap"` é responsável por apresentar em maiúsculo o primeiro caractere de uma sequência de caracteres alfabéticos. Observe o código seguinte.

```
dec strcap : list char -> list char;  
--- strcap [] <= [];  
--- strcap (x :: xs) <= toupper x :: str_map (xs, tolower);
```

Execute a instrução.

```
>: strcap ("programacao de computadores");  
>> "Programacao de computadores" : list char
```

A próxima função remove espaços em branco de uma sequência de caracteres que pode ser chamada `"strrmvspc"`. Observe o código seguinte.

```
dec strrmvspc : list char -> list char;
--- strrmvspc s <= str_filt (\ c => isalpha c, s);
```

Execute as instruções.

```
>: strrmvspc("teste de escrita");
>> "testedeescrita" : list char

>: strrmvspace(strupper("teste de escrita"));
>> "TESTEDEESCRITA" : list char
```

Uma possibilidade operacional no uso de cadeias com caracteres é a definição de uma função que troque caracteres existentes em uma cadeia por outro caractere informado. Observe a seguir o código para a função **"strreplace"**.

```
dec strreplace : char -> char -> list char -> list char;
--- strreplace ca ct s <= str_map(s, \ s => if s = ca
                                     then ct
                                     else s);
```

Execute a instrução.

```
>: strreplace 'e' 'i' "teste de escrita";
>> "tisti di iscrita" : list char
```

A função seguinte chamada **"concat"** tem por finalidade efetuar a concatenação de duas sequências de caracteres formando uma única sequência. Observe o código seguinte.

```
dec concat : list char # list char -> list char;
--- concat (a, []) <= a;
--- concat ([], b) <= b;
--- concat (a, b) <= a <> b;
```

Execute as instruções.

```
>: concat("TESTE ", "ESCRITA");
>> "TESTE ESCRITA" : list char

>: concat("TESTE ", concat("DE ", "ESCRITA"));
>> "TESTE DE ESCRITA" : list char
```

Uma funcionalidade popular na manipulação de caracteres alfanuméricos é a função **"substr"** que para ser implementada necessitará de duas outras funções chamadas **"strtake"** e **"strdrop"**. Observe os códigos seguintes.

```
dec strtake : num # list char -> list char;
--- strtake (_, []) <= [];
--- strtake (n, x :: xs) <= if n > 0
                               then x:: strtake (n - 1, xs)
                               else [];

dec strdrop : num # list char -> list char;
--- strdrop (_, []) <= [];
--- strdrop (n, x :: xs) <= if n - 1 > 0
                               then strdrop (n - 1, xs)
                               else xs;

dec substr : num # num # list char -> list char;
--- substr (c, f, lst) <= strdrop (c, strtake (f, lst));
```

Execute as instruções.

```
>: strtake(5, "TESTE DE ESCRITA");
>> "TESTE" : list char

>: strdrop(5, "TESTE DE ESCRITA");
>> " DE ESCRITA" : list char
```

```
>: substr(6, 9, "TESTE DE ESCRITA");
>> "DE " : list char

: substr(6, 8, "TESTE DE ESCRITA");
> "DE" : list char
```

A reversão de uma cadeia de caracteres pode ser implementada a partir da função "**strrev**" com o código seguinte.

```
dec strrev : list char -> list char;
--- strrev [] <= [];
--- strrev (x :: xs) <= strrev xs <> [x];
```

Execute a instrução.

```
>: strrev("teste de escrita");
>> "atircse ed etset" : list char
```

A partir da função "**strrev**" pode-se criar outra função que apresente a quantidade final de caracteres de uma sequência. Observe o código da função "**strlast**" seguinte.

```
dec strlast : num # list char -> list char;
--- strlast (_, []) <= [];
--- strlast (n, lst) <= strrev (strtake (n, strrev lst));
```

Execute a instrução.

```
>: strlast(5, "TESTE DE ESCRITA");
>> "CRITA" : list char
```

Há outras funcionalidades para diversos tratamentos de dados que podem, a partir dos exemplos indicados, serem produzidas por você. É devido a esta característica operacional da linguagem **HOPE** que ela é adequada para a aprendizagem da programação funcional. Grave as funções apresentadas neste tópico no arquivo de módulo "**mychars.hop**".

6.5 CRIPTOGRAFIA CAESAR

Neste tópico é apresentada ações referentes ao uso de ações criptográficas com a linguagem **HOPE** a partir das cifra de Caesar. Antes de tratar o método de cifra-gem é necessário considerar que os textos nas ações de cifra-gem são normalmente grafados com caracteres maiúsculos e as mensagens são escritas inicialmente sem espaços em branco.

O algoritmo Caesar usado pelo imperador romano Júlio César para se comunicar com seus generais tem sua origem no ano 50 a.C., usa uma estratégia de cifra-gem muito simples baseada em um único alfabeto cifrante (monoalfabético) em que ocorre o deslocamento de uma letra do alfabeto três posições à frente (monográfica) para cifrar e três posições atrás para decifrar mensagens curtas, sendo ideal para a escrita de pequenos textos.

A figura 6.1 mostra a regra de cifra-gem de Caesar, contendo na linha superior (em minúsculo) o caractere a ser cifrado e na linha inferior (em maiúsculo) o caractere cifrado.

Alfabeto																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Figura 6.1 - Regra de cifra-gem: Caesar.

A linha superior da figura 6.1 possui o alfabeto convencional e a linha inferior possui o alfabeto cifrado em relação ao alfabeto convencional.

A mensagem **"ATACAR BASE INIMIGA AO AMANHECER"** sem espaços em branco fica definida como **"ATACARBASEINIMIGAAOAMANHECER"** quando cifrada com a cifra Caesar será cifrada como **"DWDFDUEDVHLQLPLJDDRPDQKHFHU"**.

Para usar a cifra de Caesar são indicadas as funções **"ecaesar"** e **"dcaesar"** que para serem aplicadas devem ter em memória o carregamento do arquivo de módulo **"mychars.hop"**. Observe os códigos seguintes.

```
dec ecaesar : list char -> list char;
--- ecaesar [] <= [];
--- ecaesar (x :: xs) <= chr ((ord x - 10) mod 26 + 65)
                        :: ecaesar xs;
```

```
dec dcaesar : list char -> list char;
--- dcaesar [] <= [];
--- dcaesar (x :: xs) <= chr ((ord x + 10) mod 26 + 65)
    :: dcaesar xs;
```

Execute as instruções.

```
>: ecaesar(strrmvspc(strupper("atacar base inimiga ao amanhecer")));
>> "DWDFDUEDVHLQLPLJDDRDPDQKHFHU" : list char

>: dcaesar "DWDFDUEDVHLQLPLJDDRDPDQKHFHU";
>> "ATACARBASEINIMIGAAOAMANHECER" : list char
```

Caso deseje manter os espaços em branco na mensagem é necessário fazer a cifragem e decifragem a partir de outra abordagem como são apresentadas as funções "**ecaesarchar**" e "**dcaesarchar**". Neste sentido, observe o código seguinte para a definição das funções auxiliares de cifragem.

```
dec ecaesarchar : char -> char;
--- ecaesarchar c <= if ord c = 32
    then c
    else (chr ((ord c - 10) mod 26 + 65));

dec dcaesarchar : char -> char;
--- dcaesarchar c <= if ord c = 32
    then c
    else (chr ((ord c + 10) mod 26 + 65));
```

A partir da definição das funções que cifram e decifram cada caractere de uma mensagem é necessário definir as funções que farão a ação sobre a mensagem. Observe os códigos a seguir.

```
dec estrcaesar : list char -> list char;
--- estrcaesar s <= str_map(strupper s, ecaesarchar);
```



```
dec dstrcaesar : list char -> list char;  
--- dstrcaesar s <= str_map(strupper s, dcaesarchar);
```

Execute as instruções.

```
>: estrcaesar "ATACAR BASE INIMIGA AO AMANHECER";  
>> "DWDFDU EDVH LQLPLJD DR DPDQKHFHU" : list char  
  
>: estrcaesar "atacar base inimiga ao amanhecer";  
>> "DWDFDU EDVH LQLPLJD DR DPDQKHFHU" : list char  
  
>: dstrcaesar "DWDFDU EDVH LQLPLJD DR DPDQKHFHU";  
>> "ATACAR BASE INIMIGA AO AMANHECER" : list char  
  
>: dstrcaesar "dwdfdu edvh lqlpljd dr dpdqkhfhu";  
>> "ATACAR BASE INIMIGA AO AMANHECER" : list char
```

Um dos detalhes da programação funcional é criar pequenas funções especializadas em resolver um pequeno problema e fazer uso destas funções para auxiliar na solução de outros problemas. Neste sentido, como exemplo, considere a função "**ecaesarchar**" que utiliza como apoio a função "**toupper**" que por sua vez faz uso da função "**isupper**".

Página propositalmente em branco

Apêndice

TABELA ASCII

A representação dos 256 caracteres utilizados por computadores eletrônicos obedece à estrutura de uma tabela de caracteres oficial chamada ASCII (*American Standard Code for Information Interchange* - Código Americano Padrão para Intercâmbio de Informações), a qual deve ser pronunciada como *ásquí* ou em inglês como *ass key* (SINGH, 2011, p. 269) e não *asqui dois*, como algumas pessoas inadvertidamente falam.

A tabela ASCII, Figura A.1, foi desenvolvida entre os anos de 1963 e 1968 com a participação e a colaboração de várias companhias de comunicação norte-americanas, com o objetivo de substituir o até então utilizado código de *Baudot*, o qual usava apenas cinco *bits* e possibilitava a obtenção de trinta e duas combinações diferentes. Muito útil para manter a comunicação entre dois teleimpressores (telegrafia), conhecidos no Brasil como rede TELEX operacionalizada pelos CORREIOS para envio de telegramas. O código de *Baudot* utilizava apenas os símbolos numéricos e letras maiúsculas, tornando-o impróprio para o intercâmbio de informações entre computadores.

O código ASCII padrão permite a utilização de 128 símbolos diferentes (representados pelos códigos decimais de 0 até 127). Nesse conjunto de símbolos estão previstos o uso de 96 caracteres imprimíveis, tais como números, letras minúsculas, letras maiúsculas, símbolos de pontuação e caracteres não imprimíveis, como retorno do carro de impressão (*carriage return* - tecla <Enter>), retrocesso (*backspace*), salto de linha (*line feed*), entre outros.

Da possibilidade de uso dos 256 caracteres, faz-se uso apenas dos 128 primeiros, o que deixa um espaço para outros 128 caracteres estendidos endereçados pelos códigos decimais de 128 até 255. A parte da tabela endereçada de 128 até 255 é

emu8086).

A figura A.1 mostra, respectivamente, a tabela ASCII padrão, códigos de 0 até 127, e a parte estendida, códigos de 128 até 255, utilizada para a representação de caracteres especiais pela empresa IBM quando da criação do seu padrão de microcomputadores durante a década de 1980, denominado padrão IBM-PC. A tabela apresentada mostra o valor numérico decimal e o caractere correspondente. Lembre-se de que cada um desses caracteres é representado internamente em um computador eletrônico como valores binário, cada caractere é representado por um conjunto de 8 bits, formando o chamado *byte*.

Página propositalmente em branco

B

Referências bibliográficas

ABRAMOWITZ, M. & STEGUN, I. A. **Handbook of Mathematical Functions: with Formulas, Graphs, and Mathematical Tables (Dover Books on Mathematics)**. 9a. ed., New York: Dover Publications, 1965.

BAILEY, R. **A HOPE Tutorial: Using one of the new generation of functional languages**. BYTE, Peterborough, v. 10, n. 8, p. 235-258, aug, 1985.

BURSTALL, R. M., MACQUEEN, D. B. & SANNILA D. T. **HOPE: An experimental applicative language**. Dissertação em ciência da computação - Universidade de Endiburgo. Escócia, p. 136-146. 1978.

FEOFILOFF, P. **Recursão e algoritmos recursivos**. São Paulo: USP, 2016. Disponível em: <<https://www.ime.usp.br/~pf/algoritmos/aulas/recu.html>>. Acesso em: 20 jul. 2018.

INCE, D. & ANDREWS, D. **The Software Life Cycle**. London: Butterworth, 1990.

LIPSCHUTZ, S. & LIPSON, M. **Matemática Discreta**. 3. ed. Porto Alegre: Bookman, 2013.

MAHESHWARI, P. **Improving task scheduling for larger grain execution of parallel functional programs**. Brisbane: Griffith University, 1993. Disponível em: <<https://zapdf.com/improving-task-scheduling-for-larger-grain-execution-of-para.html>>. Acesso em: 17 mai. 2018.

MENESES, P. B. **Matemática discreta para computação e informática**. 4. ed. Porto Alegre: Bookman, 2013.

PATERSON, R. **A HOPE interpreter: Reference**. 2000. Disponível em: <http://www.stolyarov.info/static/hope/ref_man.pdf>. Acesso em: 13 jul. 2018.

REYNOLDS, J. C. **Theories of programming languages**. New York: Cambridge University Press, 1998.

SEBESTA, R. W. **Conceitos de linguagens de programação**. 11a. ed., Porto Alegre: Bookman, 2018.

SINGH, S. **O livro dos códigos: A ciência do sigilo, do antigo Egito à criptografia quântica**. São Paulo: Record, 2011.

VICKI, V. **O Código de César**. 2008. Disponível em: <<http://www.numaboa.com.br/criptografia/124-substituicao-simples/165-codigo-de-cesar>>. Acesso em: 6 jul. 2018.

Um arquivo contendo a maior parte do código criado na obra pode ser obtido com o download do arquivo:

<http://www.mediafire.com/file/7htx04y6150auph/modulosHope.zip/file>

